

Software Testing and Quality Assurance

Lecture 1

Fabian Fagerholm

fabian.fagerholm@aalto.fi



Aalto University
School of Science



CS-E4960 Software Testing and Quality Assurance – Learning Outcomes

- You know and can define the **essential concepts of testing and software quality**.
- You understand the **objectives of software testing** and the **significance of testing and quality assurance** as part of software engineering.
- You know **different ways of organizing testing, common testing techniques** as well as other quality practices.
- You can select and apply **appropriate quality practices** in different situations and understand the **strengths and weaknesses of the practices**.
- You understand the purposes that **testing tools and test automation** can be used for and the typical challenges of test automation.

CS-E4960 Software Testing and Quality Assurance – Items from the study guide

Content

- Basics of software testing, concepts, testing techniques and reviews. Test planning, management and tools. The role of testing in quality assurance and quality assurance as part of software process.

Assessment Methods and Criteria

- Lecture attendance, individual assignments, group assignments.

Study Material

- Delivered to the students when the course begins.

Chapters from online books +
PDFs (links in MyCourses).

Target group and prerequisites

- **Target group**

- Software and Service Engineering major / minor
- Information Networks
- Computer Science
- EIT Digital / ICT Innovation
- Others?

- **Prerequisites**

- Basics of SE (CS-C3150 Software Engineering)
- Some understanding of software modelling (CS-C3180 Software Design and Modelling, UML, or similar)
- For several assignments:
 - Some advanced skills in computer usage: installing and configuring development software, solving technical problems
 - Basic programming knowledge (Python)
 - Basic internet technologies
 - Some experience in software development

What is this course about?

Can you spot the flaw?

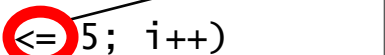
```
void main () {  
    int i = 1, c = 0;  
  
    while (i = 1) {  
        // do stuff  
        if (c > 10) {  
            i--;  
            c = 0;  
        }  
    }  
}
```


In C, the comparison operator is `==`. The `=` operator is the assignment operator.

Consequence:
The loop will never end.

Can you spot the flaw?

```
int array[] = new int[5];  
for (int i = 0; i <= 5; i++)  
    System.out.println(array[i]);
```

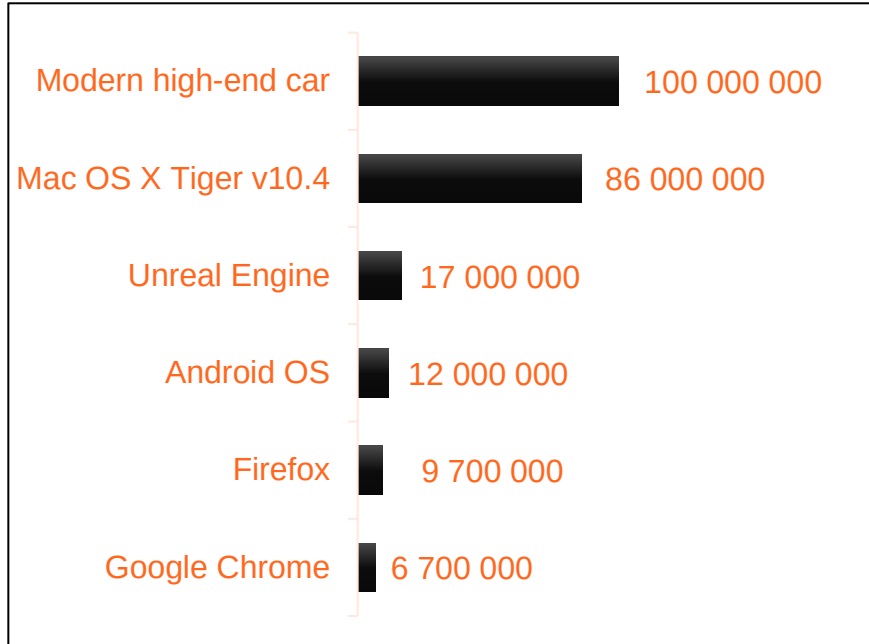


The array has 5 elements, but the counter *i* runs from 0 to 5 – six positions.

Consequence:
The program will crash, e.g.

```
Exception in thread "main"  
java.lang.ArrayIndexOutOfBoundsException:  
Index 5 out of bounds for  
length 5
```

Can you spot the flaw?



Lines of code. (Indicative estimates from various sources.)

- Software systems often consist of millions of lines of code
- An end-user application typically uses a software development framework with millions of lines of code
- How do you know where the fault is once an incident has occurred?
- How can you prevent faults in the first place?

They didn't spot the flaw...

- First NASA Space Shuttle orbital launch (1981)
 - A software modification caused timing problems between primary and backup systems → safety system stopped the launch
 - The shuttle launch was delayed by two days while software experts were looking for the cause
 - The system was extremely complex with both synchronous and asynchronous real-time modules
 - Lesson learned: quality requires significant and continuous effort because of the realities of software development
 - (Reading: Garman (1981))



They didn't spot the flaw...

- First Ariane 5 launch (1996)
 - Reused inertial reference system (SRI) software from Ariane 4 system
 - Ariane 4 has a different velocity during liftoff
 - 64-bit floating point value conversion to 16-bit integer value → operand error
 - The SRI turned the rocket nozzles to an extreme position during liftoff → spacecraft disintegrated
 - Causes:
 - Specification and design errors in the SRI software
 - Inadequate analysis and testing of SRI software
 - Lesson learned: you must follow through on quality assurance
 - (Extra reading: ESA (1996))



Unknown Author (CC BY 3.0)



DLR/Thilo Kranz (CC-BY 3.0)

Launch video:
https://www.youtube.com/watch?v=gp_D8r-2hwk

They didn't spot the flaw...

Nordea bank (2016)

- New reporting system for investment trades taken into use 18 February 2016
- A programming error caused the system to interpret trades as already reported
- ~1.8 million trades were not reported to the Financial Supervisory Authority (~11% of all trades for 6 months)
- The error was not detected during system testing
- Fined 400 000 €

<https://www.arvopaperi.fi/uutiset/nordea-maksaa-fivan-400-000-euron-sakon-mukisematta/7a2f0833-6391-33b8-87f0-9aded3eec499> (in Finnish)

Finnish Tax Authority (2019)

- A programming error in external supplier's mass printing / sending software while sending 27 000 tax reports
- A small portion of the reports were sent to the wrong recipient
- Sensitive information was leaked → data protection breach
- Apparently, the breach was small enough not to warrant disciplinary action

<https://yle.fi/uutiset/3-10915662> (in Finnish)

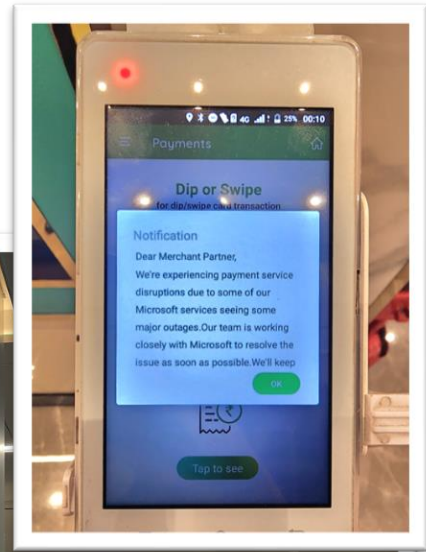
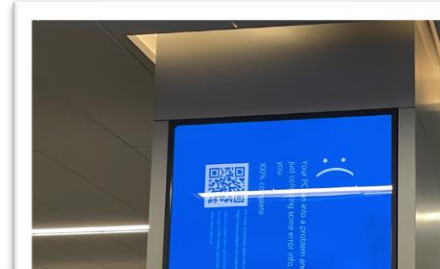
<https://yle.fi/uutiset/3-10918492> (in Finnish)

They didn't spot the flaw...

CrowdStrike (2024)

- CrowdStrike renewed its threat definition channel file format (21 fields vs. old 20 fields)
- A faulty channel file (old format) was deployed automatically, causing CrowdStrike's operating system kernel module to crash
- Worldwide, millions of Windows-based computers with CrowdStrike installed crashed and could not properly restart
- Only the “happy path” of the software was tested
- No regression tests, only valid test data
- Distributed to all customers simultaneously, no staggered rollout
- Estimated financial damage: at least 10 billion USD (10 000 million)

Reading: CrowdStrike (2024)



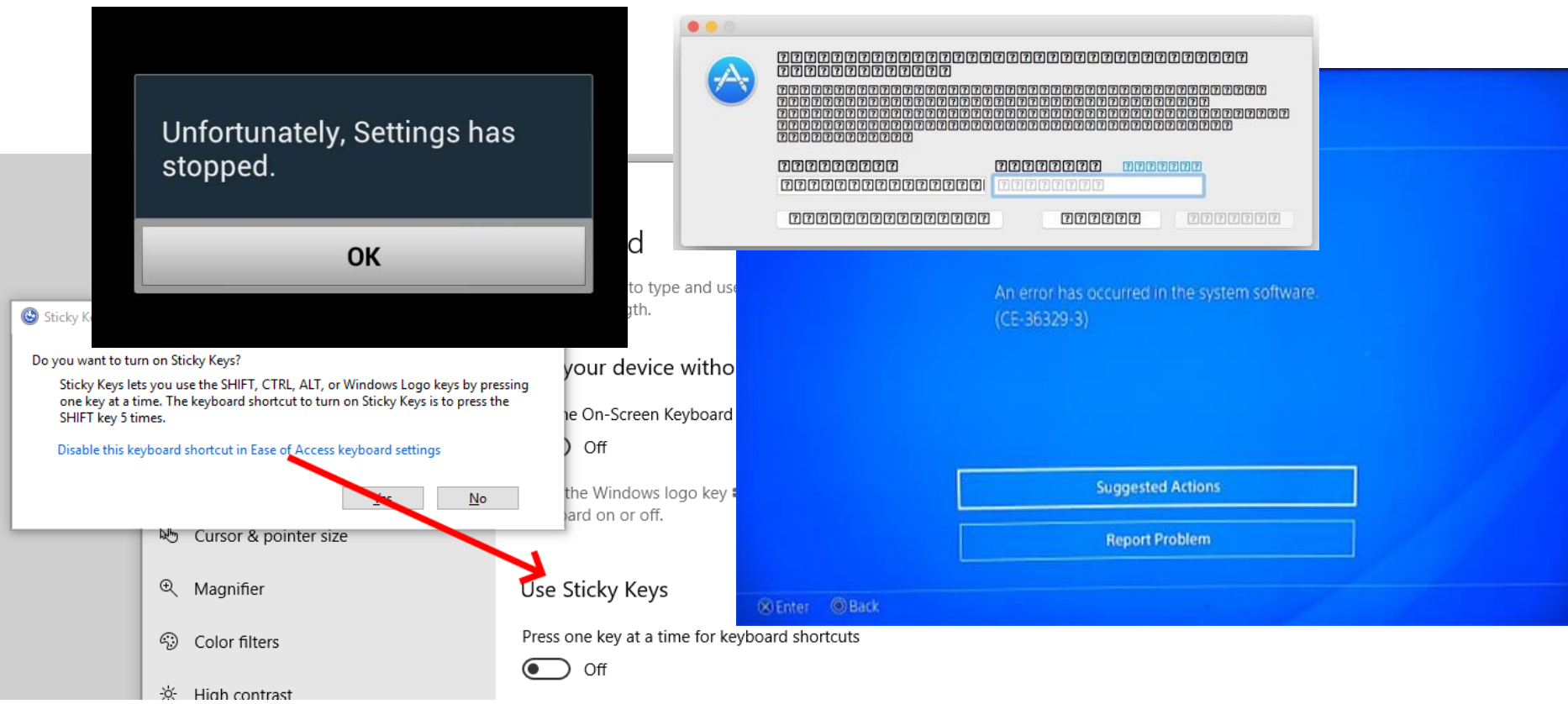
Images:

Blue screens of death at LGA airport, Smishra1, CC-BY-SA 4.0

Payment service disruption in India, MohitSingh, CC-BY-SA 4.0

Notice in a Woolworths supermarket in New Zealand, CCO

Not all flaws are fatal, but still annoying



Discussion

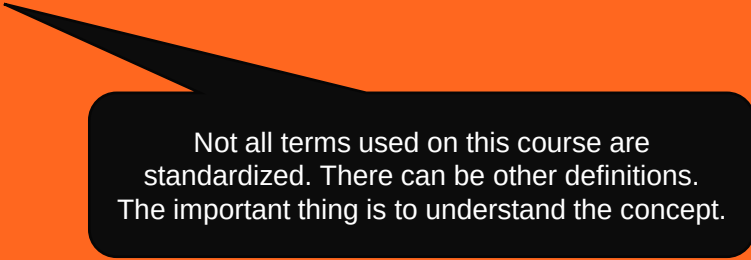
Is defect-free software possible? How?
(Or: Why not?)

(Here, we mean non-trivial, real-world software systems)

Software Testing and Quality Assurance

- Is defect-free software possible? How? (Or: Why not?)
 - Eliminating all software defects is usually not feasible
 - The effort (cost and time) required is too high
 - If the system is complex enough, the conditions under which failures occur are not possible to anticipate fully
 - Changed requirements: the software must change before all defects can be found
 - The defects themselves are complex and can occur in many development stages (requirements, specifications, code, etc.)
- Focus on **quality** and eliminating important defects
- *This course aims to increase your knowledge of software quality and the role of testing in assuring quality*

Preliminaries: Important concepts



Not all terms used on this course are standardized. There can be other definitions. The important thing is to understand the concept.

Important concepts: *Quality*

What is quality?

What would a useful definition look like?



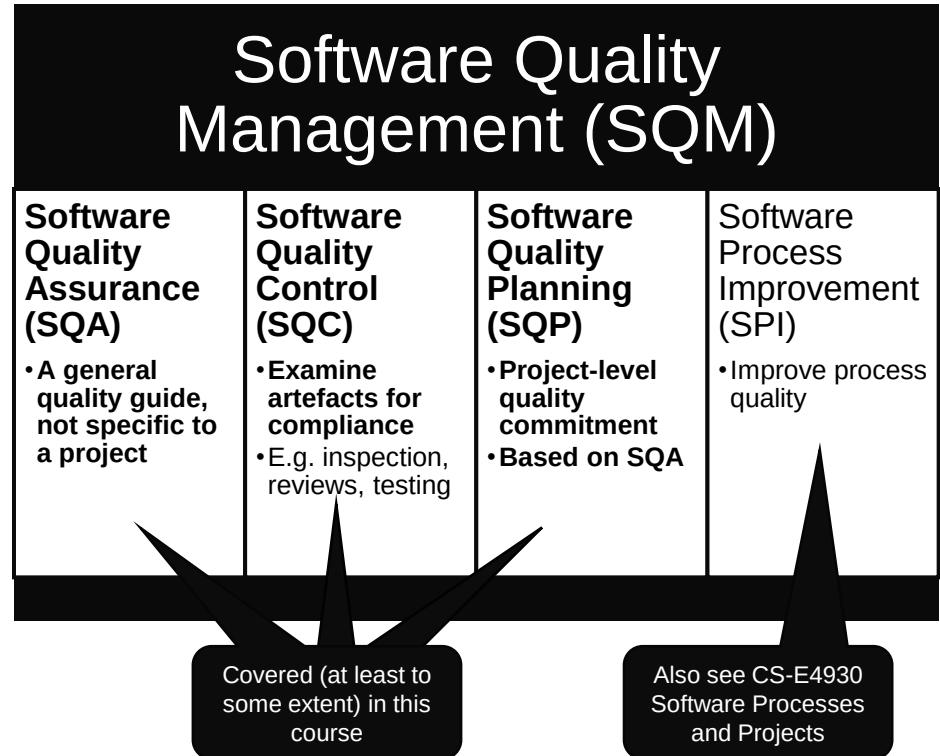
Important concepts: *Quality*

- “Conformance to the requirements” (Crosby)
 - Ignores intrinsic quality differences between products
 - Does not consider whether requirements are appropriate for the product
- “Fitness for use” (Juran)
 - No mechanism to judge better quality when two products are equally fit for use
- ISO/IEC/IEEE 24765:2017 (emphasis added):
 1. degree to which the system *satisfies the stated and implied needs of its various stakeholders*, and thus *provides value*
 2. ability of a product, service, system, component, or process to *meet customer or user needs, expectations, or requirements*
 3. the degree to which a set of inherent characteristics *fulfils requirements*



Important concepts: *Quality Assurance*

- Providing confidence that quality requirements will be fulfilled
- A quality guide with
 - *standards, regulations, best practices* and *software tools* to
 - *produce, verify, evaluate and confirm work products* during the software development life cycle.
- Internal purposes: provide confidence for the management
- External purposes: provide confidence to the customers and other external stakeholders.



Important concepts: Errors to Incidents

Error: Typing = instead of ==

Fault: An infinite loop in the program

Failure: System freezes on Tuesday at 08:00

Incident: The user notices that the system does not respond on Tuesday at 08:15



Error (mistake)

A human error while making a software artefact (requirement, specification code, ...).



Fault (defect)

An incorrectness in a software artefact resulting from an error (also: a bug).



Failure

Manifested inability of system to perform according to specification.



Incident

The symptom associated with a failure. Alerts the user to the occurrence of a failure.

Important concepts: Quality control concepts

Test

The act of exercising the system with test cases

Test case

A check of assumptions related to system behaviour

Test suite

A collection of test scripts or test cases

Test script

Instructions for executing a test case

Test log

A record of test execution

Test report

A document describing the conduct and results of testing

Test plan

A document defining: scope of testing, types of testing to perform, test environment, required hardware and software, estimation of effort and resources, risk management, deliverables, key test milestones, and schedule

Discussion

What is your favourite software incident?

What kind of failure is it an example of?

(i.e. in what way did it fail?)

What kind of error led to it?

Could it have been prevented? How?

 **Error** (mistake)

 **Fault** (defect)

 **Failure**

 **Incident**

Practicalities

Learning on this course

Lectures

- Once a week on Tuesdays at 10-12
- Attendance points
- In case of illness, get in touch

Individual assignments

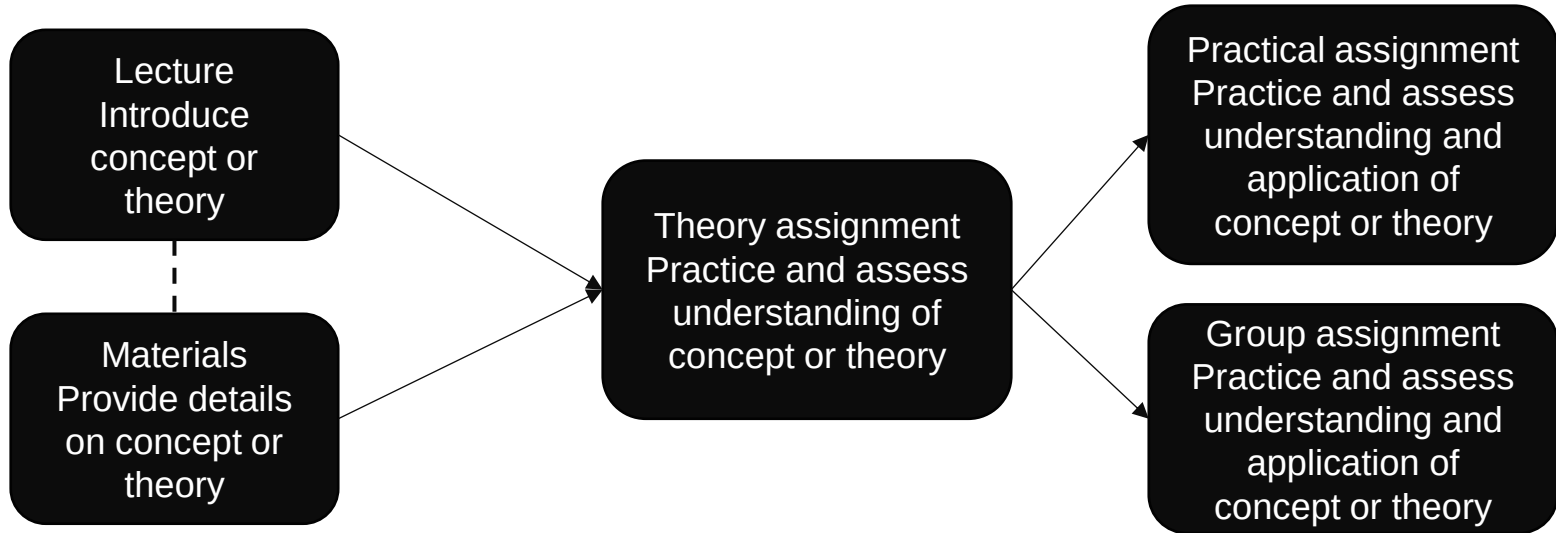
- Individual learning tasks (no cooperation with other students)
- Become available the same day as the lecture
- In MyCourses
- Deadline just before the lecture on the following Tuesday (some tasks have more time)
- Point deduction for late assignments: 25% / day (rounded to nearest 0,5 p)

Group assignments

- Cooperation encouraged!
- Support your learning
- Try testing techniques that cannot be done individually
- Some deliverables build up to final assignment
- Details in Lecture 2



Learning on this course



How is your learning evaluated?

Component		Max points
Individual	Lecture participation* (10 sessions, 1 point each)	10
	Assignments	70
Group	Assignments	15
	Peer review	5
Total		100
Extra points, voluntary	Course feedback	1

See next slide

- To pass: min 50% of individual points and min 25% of group points.
- Grade: 50p → 1, 61p → 2, 71p → 3, 81p → 4, 91p → 5.
- * Alternative assignment: written task based on lecture slides + materials. Inform course staff by week 2 if you will not attend lectures.



Extra points

- Course feedback, 1p
 - Provide feedback at the end of the course
 - Feedback is anonymous: the system records who has answered separately from the answers
 - Very important – you benefit from past years' feedback!
- Extra points count towards your total points but are not counted as individual or group points

Week	Lecture	Topic	Assignment deadlines (Tuesdays 10:00 unless otherwise specified)
36	3.9.2024	Introduction and practicalities	
37	10.9.2024	Software quality	Individual assignments 1
38	17.9.2024	Software testing: levels, test case analysis and design	Individual assignments 2, Group registration (DL: 20.9.)
39	24.9.2024	Testing techniques: Black-box testing	Individual assignments 3
40	1.10.2024	Testing techniques: Black-box testing	Individual assignments 4, Group assignment 1
41	8.10.2024	Testing techniques: Manual testing	Individual assignments 4, Group assignment 2
42	15.10.2024	(No lecture)	
43	22.10.2024	Testing techniques: White-box testing	
44	29.10.2024	Testing techniques: White-box testing	Individual assignments 5
45	5.11.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 6, Group assignment 3
46	12.11.2024	Continuous Integration and Continuous Delivery/Deployment	Individual assignments 7, Group assignment 4
47	19.11.2024	Guest lecture	Individual assignments 8
48	26.11.2024	Test management	Individual assignments 9
49	3.12.2024	(No lecture)	Individual assignments 10, Group assignment 5

New assignments

- Optional (no points) reading and discussion
 - Read: Garman 1981, ESA 1996, Crowdstrike 2024
 - Use the course chat to discuss (#famous incidents)
 - What happened in these cases and why?
 - What can we learn about software quality and testing from past incidents?
- **Deadline: before next lecture (Tuesday by 10:00)**
- Getting help
 - Ask in the course chat, see Communication section in MyCourses
 - Personal questions by email



Study smarter, not harder!

- Plan: when and where
- Go deep at your own pace
- Get some rest
- Practice makes perfect
- Enjoy it ☺

Thank you!
Questions?



Aalto University
School of Science

Software Testing and Quality Assurance

Lecture 2

Fabian Fagerholm

fabian.fagerholm@aalto.fi



Aalto University
School of Science



Week	Lecture	Topic	Assignment deadlines (Tuesdays 10:00 unless otherwise specified)
36	3.9.2024	Introduction and practicalities	
37	10.9.2024	Software quality	Individual assignments 1
38	17.9.2024	Software testing: levels, test case analysis and design	Group registration (DL: 20.9.)
39	24.9.2024	Testing techniques: Black-box testing	Individual assignments 2, Group assignment 1
40	1.10.2024	Testing techniques: Black-box testing	Individual assignments 3
41	8.10.2024	Testing techniques: Manual testing	Individual assignments 4
42	15.10.2024	(No lecture)	
43	22.10.2024	Testing techniques: White-box testing	Individual assignments 5
44	29.10.2024	Testing techniques: White-box testing	Individual assignments 6, Group assignment 2
45	5.11.2024	Guest lecture	Individual assignments 7
46	12.11.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 8
47	19.11.2024	Continuous Integration and Continuous Delivery/Deployment	Individual assignments 9
48	26.11.2024	Test management	Individual assignments 10, Group assignment 3
49	3.12.2024	(No lecture)	Individual assignments 11, Group assignment 5

Summary of previous lecture

- Software failures can lead to
 - catastrophic incidents with major losses
 - annoying incidents that contribute to a bad user experience
- Many failures could be avoided by proper attention to testing and quality assurance

Software Quality Management (SQM)

Software Quality Assurance (SQA)	Software Quality Control (SQC)	Software Quality Planning (SQP)	Software Process Improvement (SPI)
• A general quality guide, not specific to a project	• Examine artefacts for compliance • E.g. inspection, reviews, testing	• Project-level quality commitment • Based on SQA	• Improve process quality



Error (mistake)



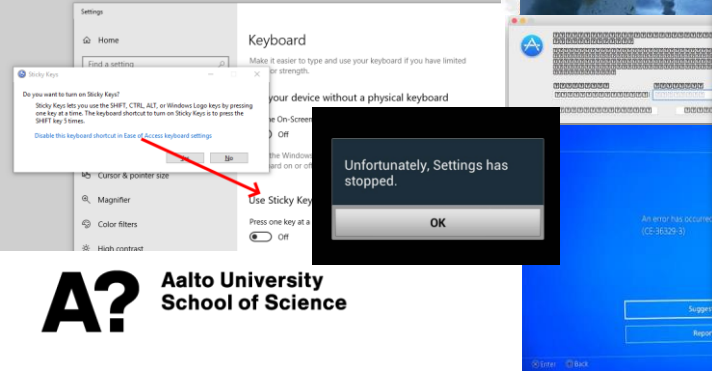
Fault (defect)



Failure



Incident



Important concepts: *Quality*

- “Conformance to the requirements” (Crosby)
 - Ignores intrinsic quality differences between products
 - Does not consider whether requirements are appropriate for the product
- “Fitness for use” (Juran)
 - No mechanism to judge better quality when two products are equally fit for use
- ISO/IEC/IEEE 24765:2017 (emphasis added):
 1. degree to which the system *satisfies the stated and implied needs of its various stakeholders*, and thus *provides value*
 2. ability of a product, service, system, component, or process to *meet customer or user needs, expectations, or requirements*
 3. the degree to which a set of inherent characteristics *fulfils requirements*



For testing and quality assurance to work, we need to know what quality means in our specific case

Software Quality

Quality

- Being superior or not inferior
- Being suitable for an intended purpose while satisfying customer expectations
- Perceptual, somewhat subjective: may be understood differently by different people

Consumer focus: specification quality

- Example: How the product compares to competitors

Producer focus: conformance quality

- Example: The degree to which the product/service was produced correctly

Support focus

- Example: The degree to which a product/service is reliable, maintainable, or sustainable

Software Quality

- Software quality is a multifaceted concept
- It is not only a property of the code
- It includes the quality of several artefacts and processes, for example:

Operating procedures

- A software program is part of a larger system and operating environment – it has users that can be other software and humans

Documentation

- Descriptions of the software for development and use

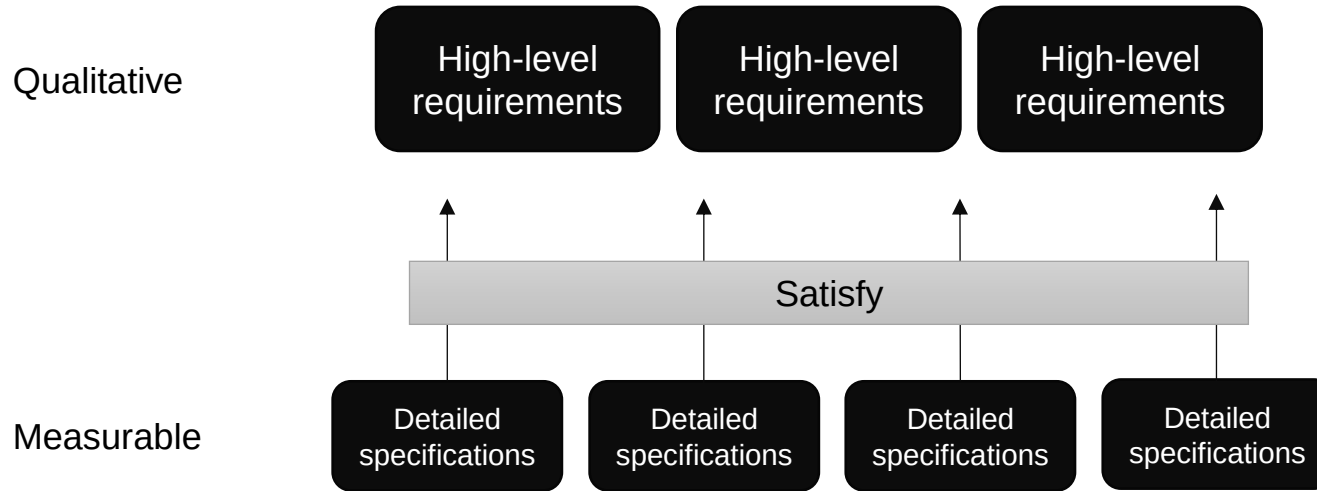
Data

- The data needed in operation

Code

- The software code itself

Decomposing software quality



- High-level requirements are broken down into detailed specifications
- The detailed specifications satisfy the requirements
- The specifications are measurable

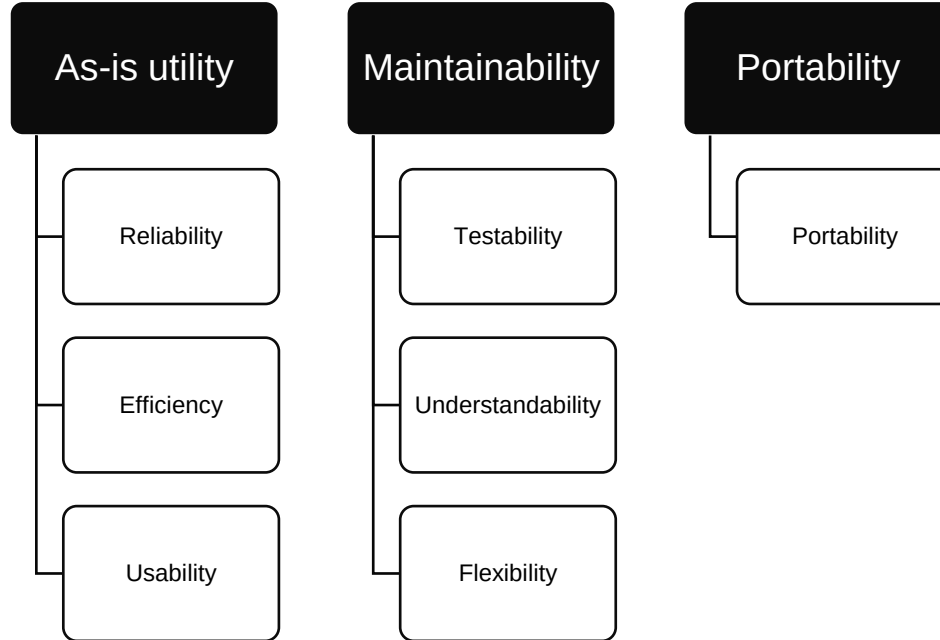
→ A way to test, inspect, review and assure quality

Challenges for software quality:

- What model of quality characteristics (quality requirements) should be used?
- How should the requirements be broken down into measurable, quantitative attributes?

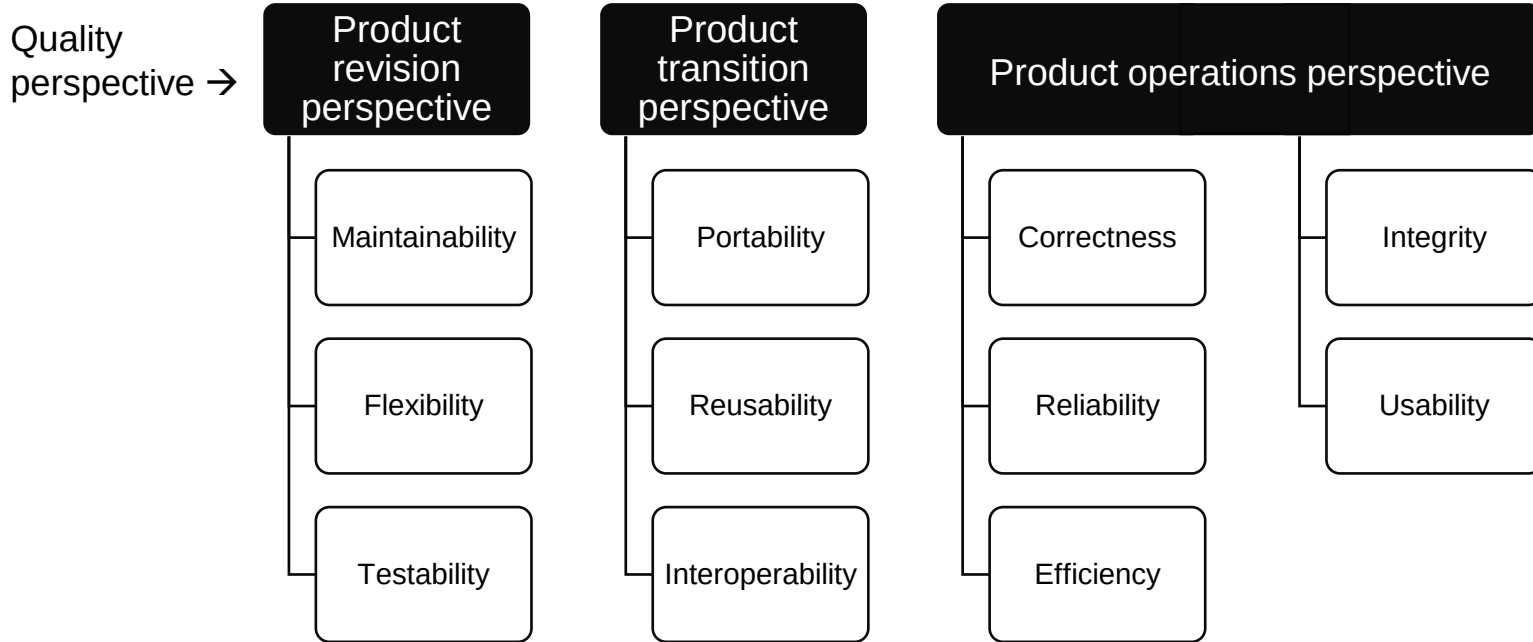
Quality characterisation models

General utility →



Boehm' model

Quality characterisation models



McCall's model

Quality characterisation models

Example:

ISO-9126
High-level
quality factors

Functionality

Reliability

Usability

“The capability of the software product to provide functions, which meet stated and implied needs when the software is used under specified conditions”

Defined in use cases, data flow diagrams, business rules, etc.

Functional requirement: either present or not

Maintainability

Portability

Quality characterisation models

Example:

ISO-9126
High-level
quality factors

Functionality

Reliability

Usability

Efficiency

“The capability of the software product to maintain a specified level of performance when used under specified conditions”

Non-functional requirement: present to some degree

Quality characterisation models

Example:

ISO-9126
High-level
quality factors

Functionality

Reliability

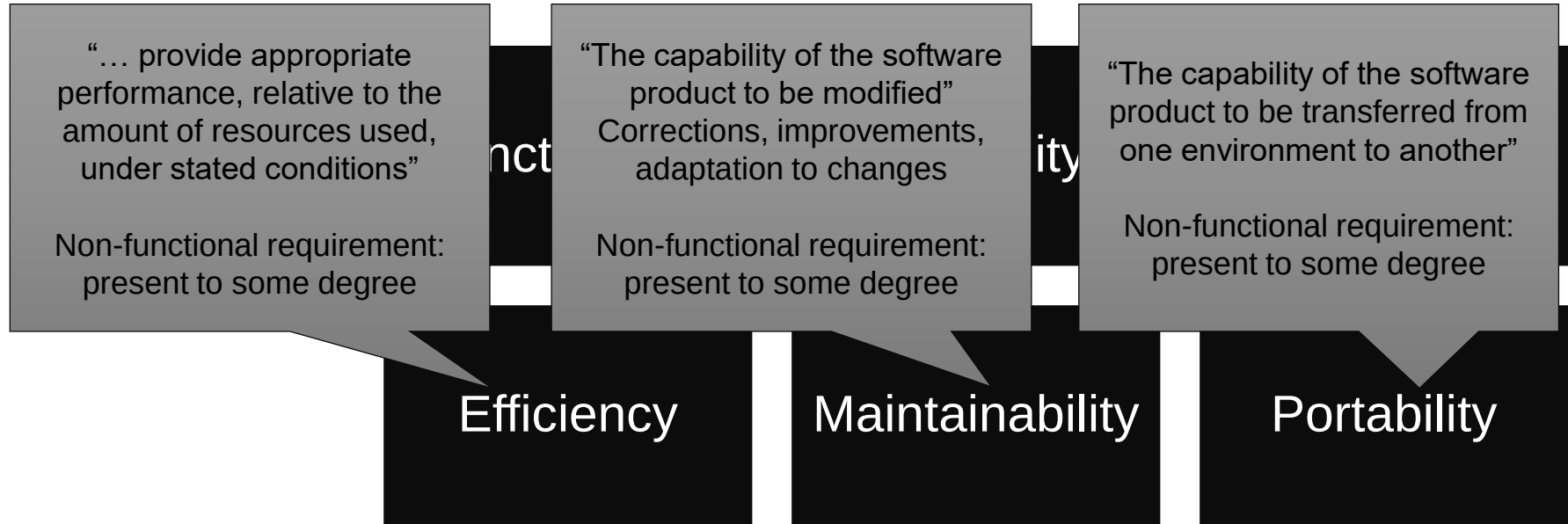
Usability

Efficiency

“The capability of the software product to be understood, learned, used, and attractive to the user, when used under specified conditions”

Non-functional requirement: present to some degree
Usability testing is usually seen as separate from software testing and performed by usability experts. However, some aspects may be taken into software testing.

Quality characterisation models



Decomposing quality criteria: Reliability

- Quality criteria need to be concrete regardless of the quality characterisation model
- Quality criteria are decomposed to a level where one or more *metrics* can be defined to measure each criteria

Reliability			
Error tolerance	Consistency	Accuracy	Simplicity
•Those attributes of the software that provide continuity of operation under nominal conditions	•Those attributes of the software that provide uniform design and implementation techniques and notation	•Those attributes of the software that provide the required precision in calculations and outputs	•Those attributes of the software that provide implementation of functions in the most understandable manner (usually avoidance of practices which increase complexity)

McCall's model

Discussion

Think of examples of quality in real software that you use

What qualities are important?

How would you define them?

How would you assess them?

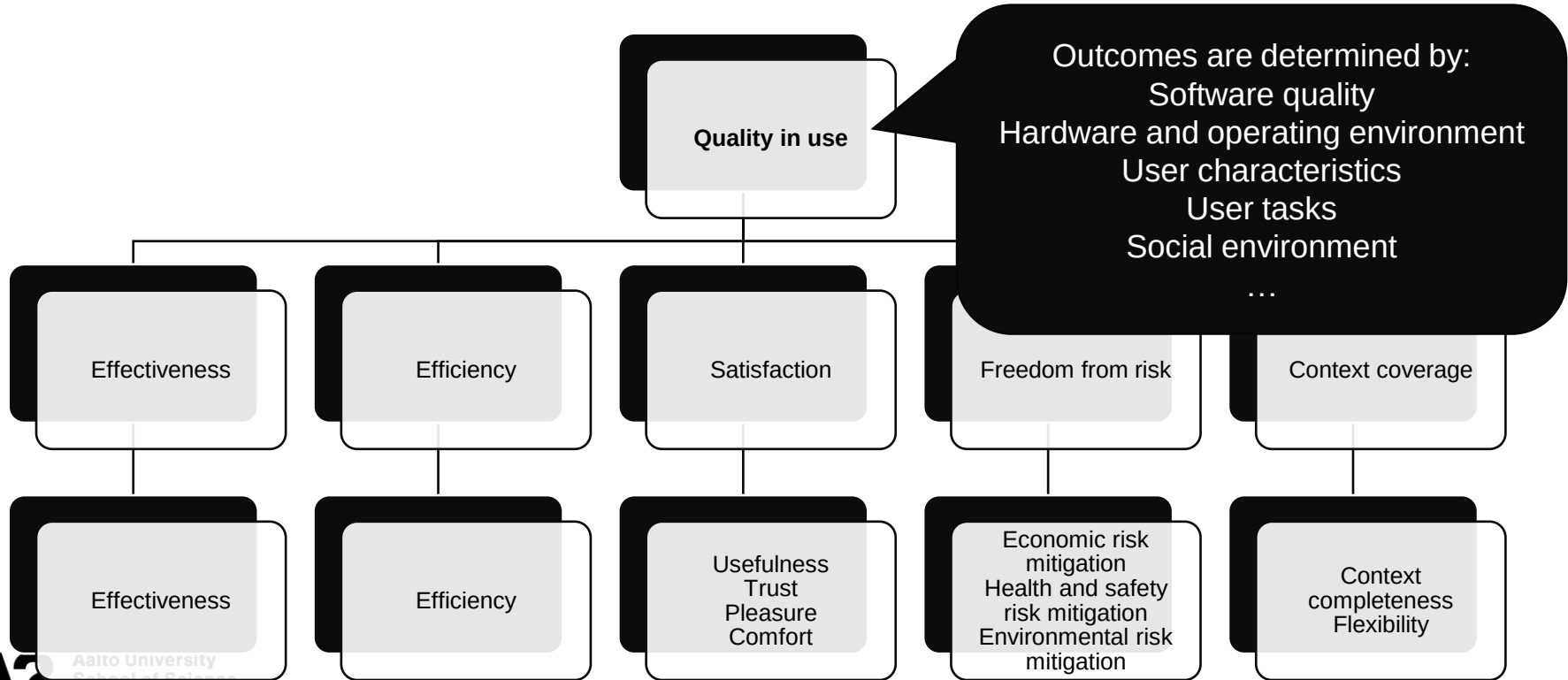
(How do you know if the software meets the quality expectations)

ISO/IEC 25010 (SQuaRE)

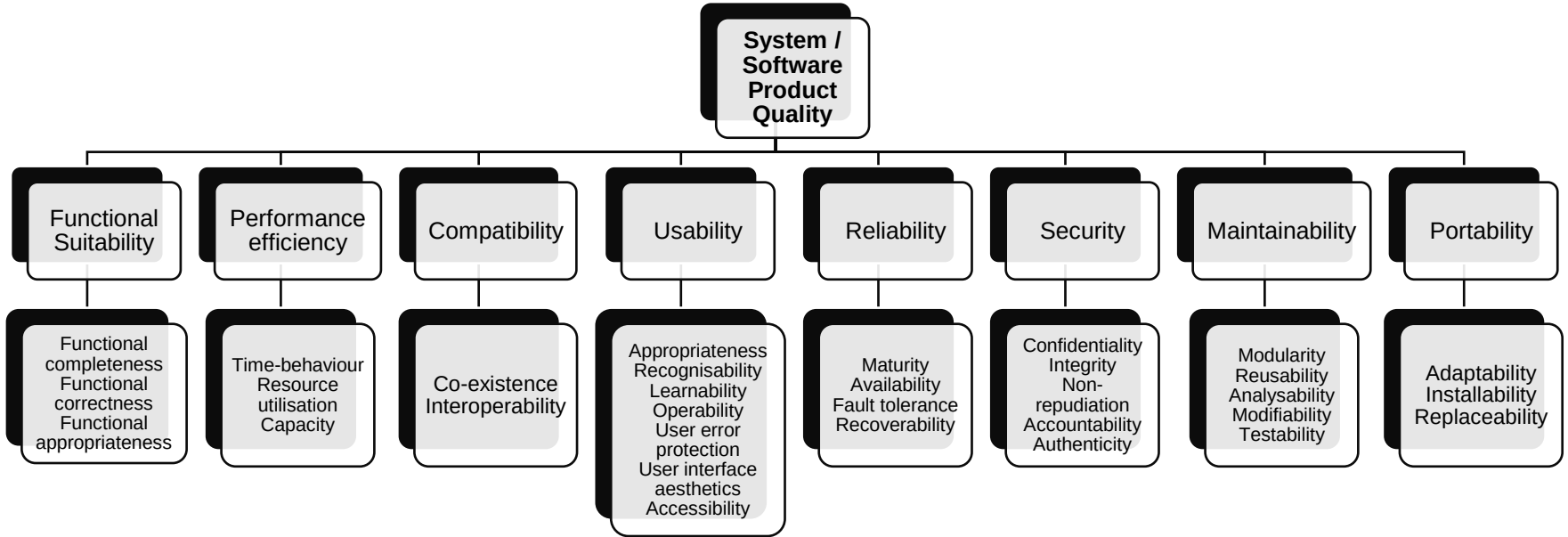
Quality in
use

Product
quality model

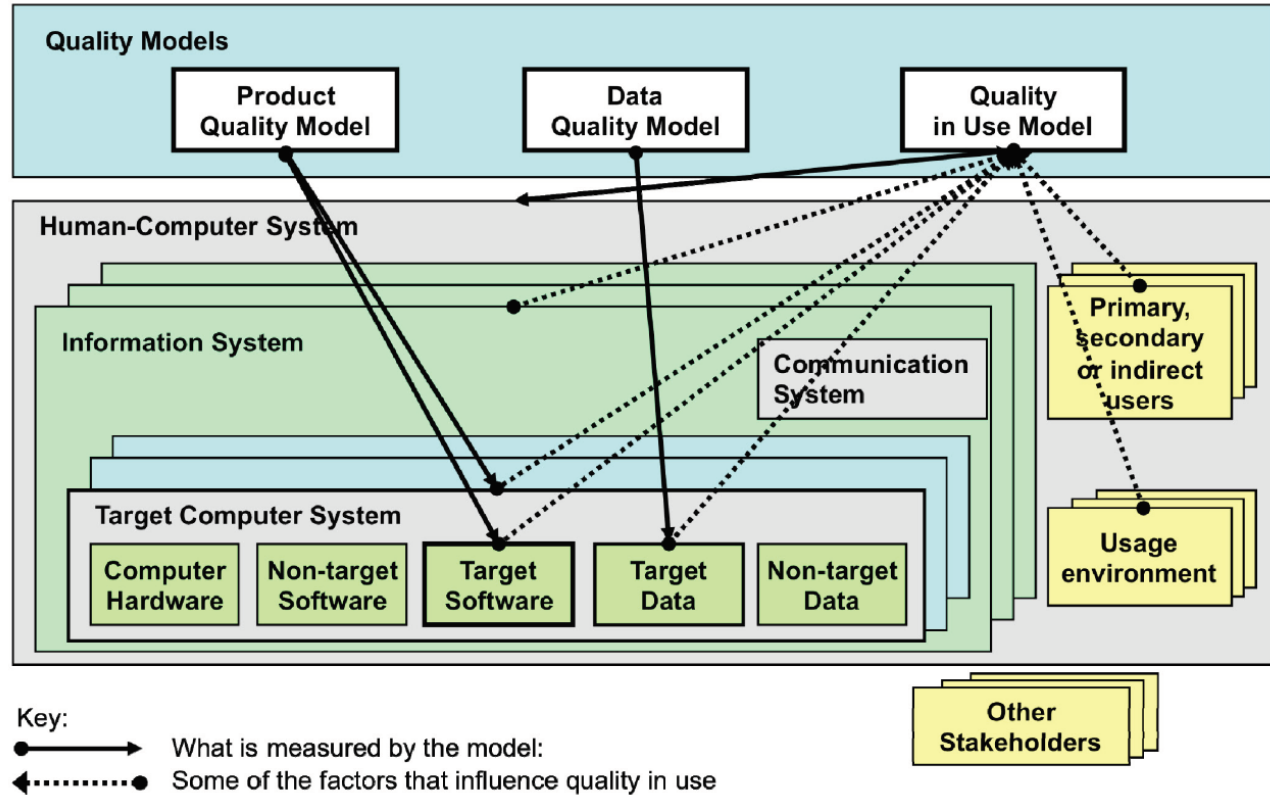
ISO/IEC 25010 (SQuaRE): Quality in use



ISO/IEC 25010 (SQuaRE): Product quality model



ISO/IEC 25010 (SQuaRE): Quality model targets



ISO/IEC 25010 (SQuaRE): Using a quality model

- **Stakeholder perspectives**

- Primary user: person who interacts with the system to achieve the primary goals
- Secondary user: provide support
- Indirect users: person who receives output but does not interact with the system

User needs	Primary user	Secondary users		Indirect user
		Content provider	Maintainer	
	Interacting	Interacting	Maintaining or porting	Using output
Effectiveness	How effective does the user need to be when using the system to perform their task?	How effective does the content provider need to be when updating the system?	How effective does the person maintaining or porting the system need to be?	How effective does the person using output from the system need to be?

Example:
The system must enable the user to complete 10 issues per hour

Software metrics

- *Metric*: A quantitative scale or method, which can be used for measurement
- *Measurement*: The process of assigning a number or category to an entity to describe an attribute of that entity
- *Internal metrics*
 - Assessing or predicting quality during production
 - Measures intermediate deliverables
 - E.g. through reviews and inspection
 - Static testing: the software is not run
- *External metrics*
 - Customer metrics
 - Measures the presence of a quality factor in the final product or component
 - Dynamic testing (or in production): the software is run



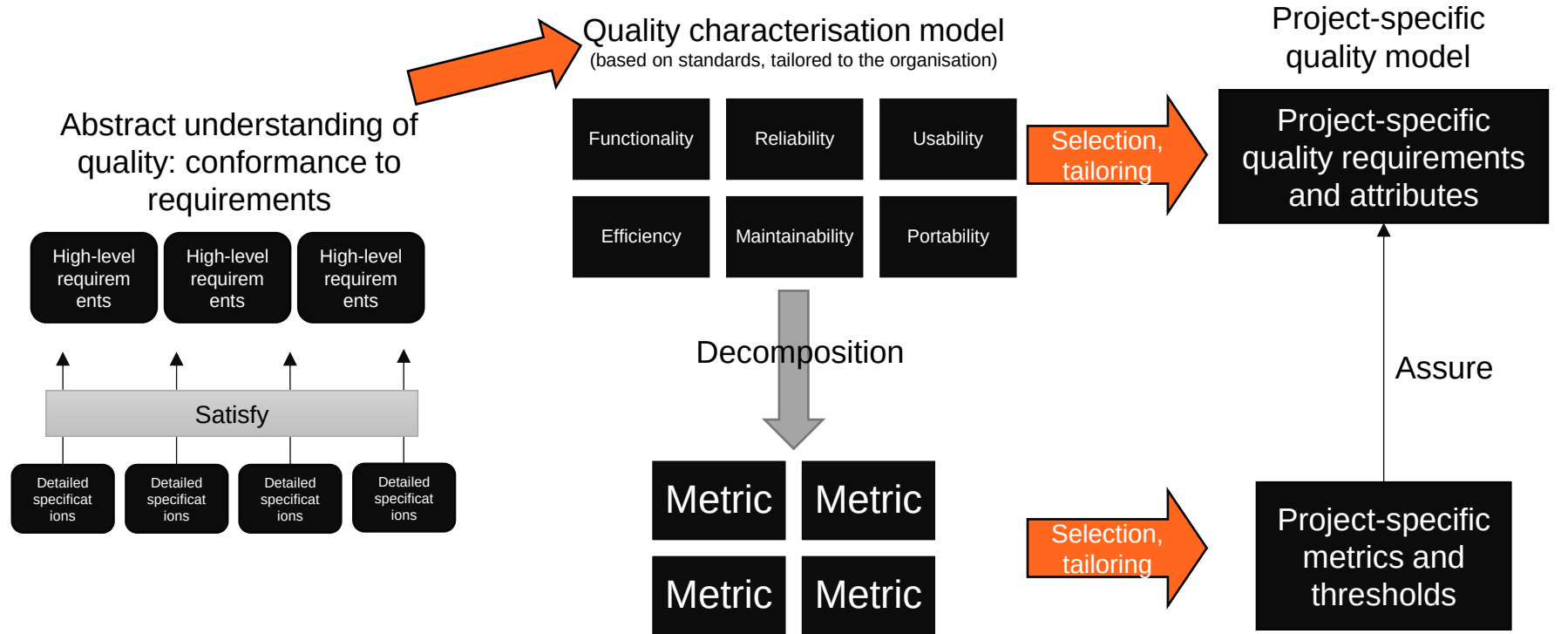
Example:
Simplicity

- Is the design organized top down? (yes/no)
- Are modules independent? (ratio of independent modules)

Example:
Reliability

- Mean time between failure (MTBF) during operation

From abstract to concrete



Challenges for software quality:

- What qualitative model of requirements quality characteristics should be used?
- How should the requirements be broken down into measurable, quantitative attributes?

User stories

- Functional descriptions written from a user perspective
- *As a <role>, I can/want <capability> so that <receive benefit>*
- Can be based on personas (descriptions of persons that represent real-life groups of users)
- User stories do not capture every kind of quality attribute – they most often describe only functionality

The beginnings of acceptance tests

User story

As a content owner, I want to create product content so that I can provide information to my customers

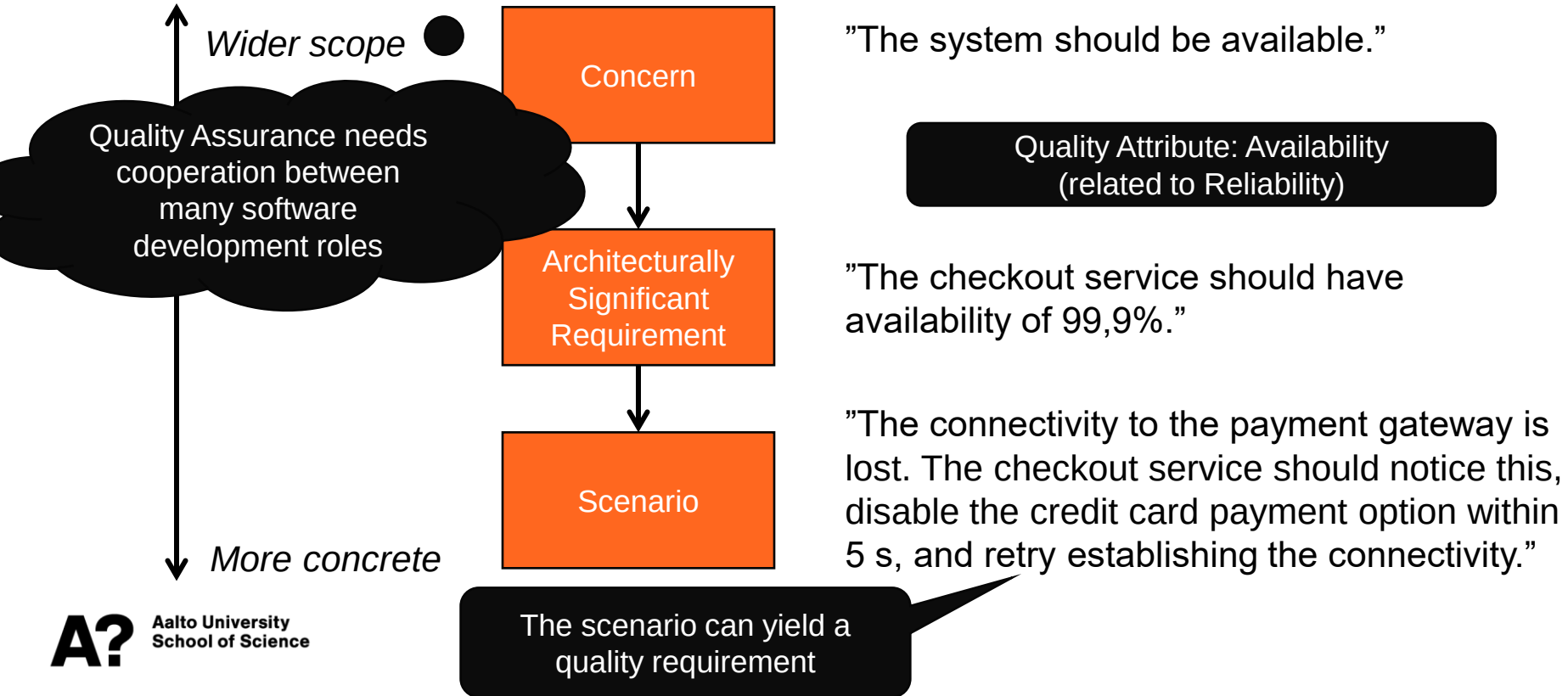
An an editor, I want to review content before it is published so that I can ensure it is correct and has the right style

Acceptance criteria

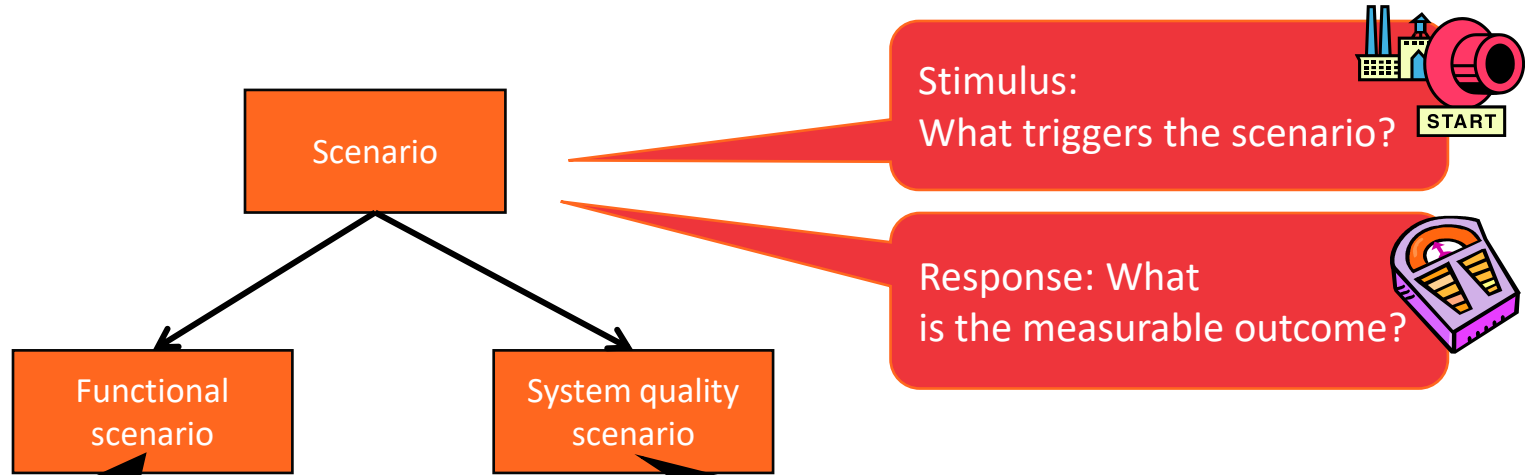
- Log in
- Create content page
- Edit content page
- Save changes
- Assign content page to editor for review

- Log in
- View content page
- Edit content page
- Add comments
- Save changes
- Re-assign content page to content owner

Linking Quality with Requirements and Software Architecture



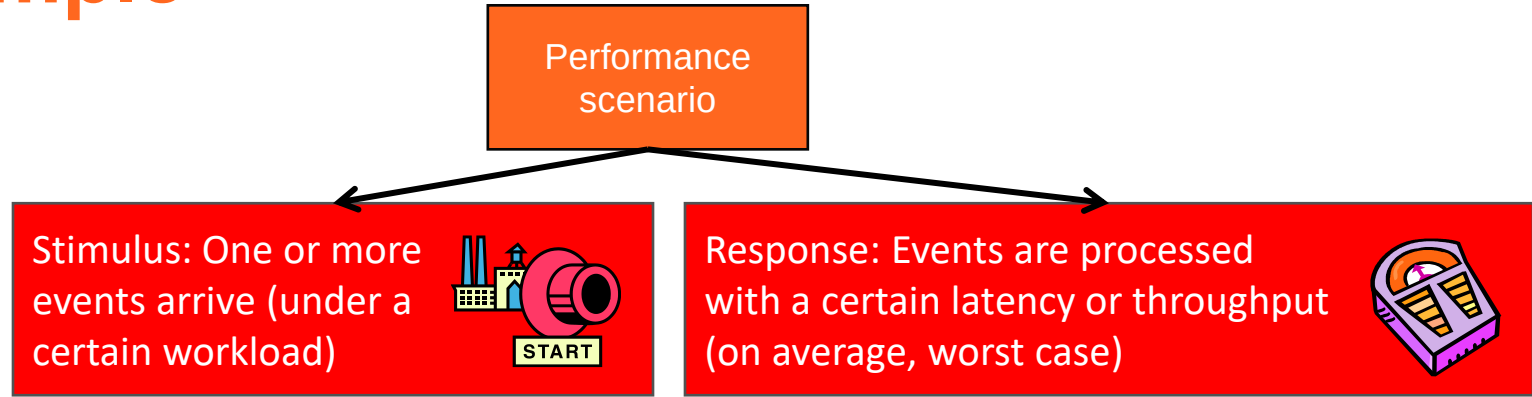
Scenarios concretise requirements



At 03:00 CET, on the first day of the month, the system calculates the order summaries per day and country and sends a report to the offices via e-mail.

The connectivity to the payment gateway is lost. The checkout service should notice this, disable the credit card payment option within 5 s, and retry establishing the connectivity.

Example



Latency: When the researcher presses *Start*, the simulation world with 100 creatures is initialized and animation begins within an average response time of one second.

Throughput: With 100 interacting creatures, the system can process, store and visualize at least 100 simulation steps per minute.

The beginnings of a performance test case

Questions?

Next: Forming groups

Group assignments

- Group assignment 1: Analysing quality using ISO/IEC 25010
- Group assignment 2: Doing testing in a group
- Group assignment 3: Final report
 - Examines a provided software application
 - Creating small quality model
 - Running a few tests
 - Analysing the results of the tests
 - Assessing the quality of the application
- Group assignment 4: Peer review

User needs	Primary user	Secondary users		Indirect user
		Content provider	Maintainer	
	Interacting	Interacting	Maintaining or porting	Using output
Effectiveness	How effective does the user need to be when using the system to perform their task?	How effective does the content provider need to be when updating the system?	How effective does the person maintaining or porting the system need to be?	How effective does the person using output from the system need to be?

How is your learning evaluated?

Component		Max points
Individual	Lecture participation* (10 sessions, 1 point each)	10
	Assignments	70
Group	Assignments	15
	Peer review	5
Total		100
Extra points, voluntary	Course feedback	1

- To pass: min 50% of individual points and min 25% of group points.
- Grade: 50p → 1, 61p → 2, 71p → 3, 81p → 4, 91p → 5.
- * Alternative assignment: written task based on lecture slides + materials. Inform course staff by week 2 if you will not attend lectures.

Forming groups for group work

- **Spread around the lecture hall & corridor**
- Approach people you don't know (that well) from before and find out:
 - *Do you prefer to work online or collocated?*
 - *What are your ambitions in the course?*
- When you find matching people, start gathering more members until you have 4-6 members.
- If you want: Start by picking one friend who you would like to work with in a group
- **If you have formed a group: that's all for today!**
- After the lecture:
 - Register your group and group name (instructions on MyCourses)
 - Agree on a way to communicate (e.g., the course chat)
 - Set up a (weekly?) meeting schedule for your group
 - Get started on the first group assignment

Week	Lecture	Topic	Assignment deadlines (Tuesdays 10:00 unless otherwise specified)
36	3.9.2024	Introduction and practicalities	
37	10.9.2024	Software quality	Individual assignments 1
38	17.9.2024	Software testing: levels, test case analysis and design	Group registration (DL: 20.9.)
39	24.9.2024	Testing techniques: Black-box testing	Individual assignments 2, Group assignment 1
40	1.10.2024	Testing techniques: Black-box testing	Individual assignments 3
41	8.10.2024	Testing techniques: Manual testing	Individual assignments 4
42	15.10.2024	(No lecture)	
43	22.10.2024	Testing techniques: White-box testing	Individual assignments 5
44	29.10.2024	Testing techniques: White-box testing	Individual assignments 6, Group assignment 2
45	5.11.2024	Guest lecture	Individual assignments 7
46	12.11.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 8
47	19.11.2024	Continuous Integration and Continuous Delivery/Deployment	Individual assignments 9
48	26.11.2024	Test management	Individual assignments 10, Group assignment 3
49	3.12.2024	(No lecture)	Individual assignments 11, Group assignment 5

Next steps

- Individual assignments in MyCourses
 - Software Quality Basics
 - Test environment setup
- **Deadline: before next lecture (Tuesday by 10:00)**
- Group assignments in MyCourses
 - Group registration and Group name choice – **Deadline: Friday 20.9. at 16:00**
 - Analysing quality using ISO/IEC 25010 – **Deadline: 24.9. at 10:00**
- Getting help
 - Ask in the course chat, see Communication section in MyCourses
 - Personal questions by email



Study smarter, not harder!

- Plan: when and where
- Go deep at your own pace
- Get some rest
- Practice makes perfect
- Enjoy it ☺

Software Testing and Quality Assurance

Lecture 3

Fabian Fagerholm

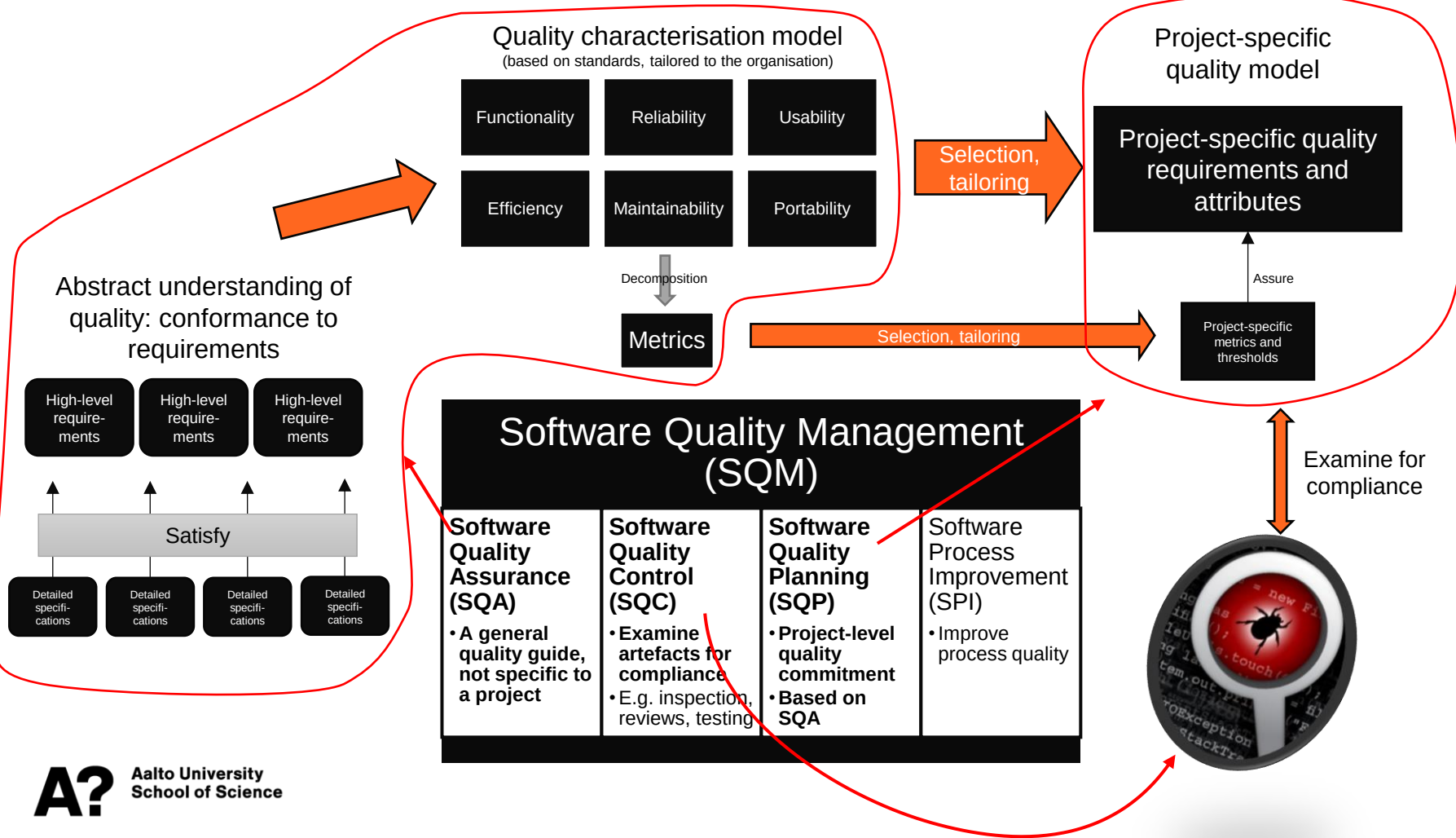
fabian.fagerholm@aalto.fi



Aalto University
School of Science



Week	Lecture	Topic	Assignment deadlines (Tuesdays 10:00 unless otherwise specified)
36	3.9.2024	Introduction and practicalities	
37	10.9.2024	Software quality	Individual assignments 1
38	17.9.2024	Software testing: levels, test case analysis and design	Group registration (DL: 20.9.)
39	24.9.2024	Testing techniques: Black-box testing	Individual assignments 2, Group assignment 1
40	1.10.2024	Testing techniques: Manual testing	Individual assignments 3
41	8.10.2024	Testing techniques: White-box testing	Individual assignments 4
42	15.10.2024	(No lecture)	
43	22.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 5
44	29.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 6, Group assignment 2
45	5.11.2024	Guest lecture	Individual assignments 7
46	12.11.2024	Continuous Integration and Continuous Delivery/Deployment, Test management, and Course summary	Individual assignments 8
47	19.11.2024	(No lecture)	Individual assignments 9
48	26.11.2024	(No lecture)	Individual assignments 10, Group assignment 3



User stories

- Functional descriptions written from a user perspective
- *As a <role>, I can/want <capability> so that <receive benefit>*
- Can be based on personas (descriptions of persons that represent real-life groups of users)
- User stories do not capture every kind of quality attribute – they most often describe only functionality

The beginnings of acceptance tests

User story

As a content owner, I want to create product content so that I can provide information to my customers

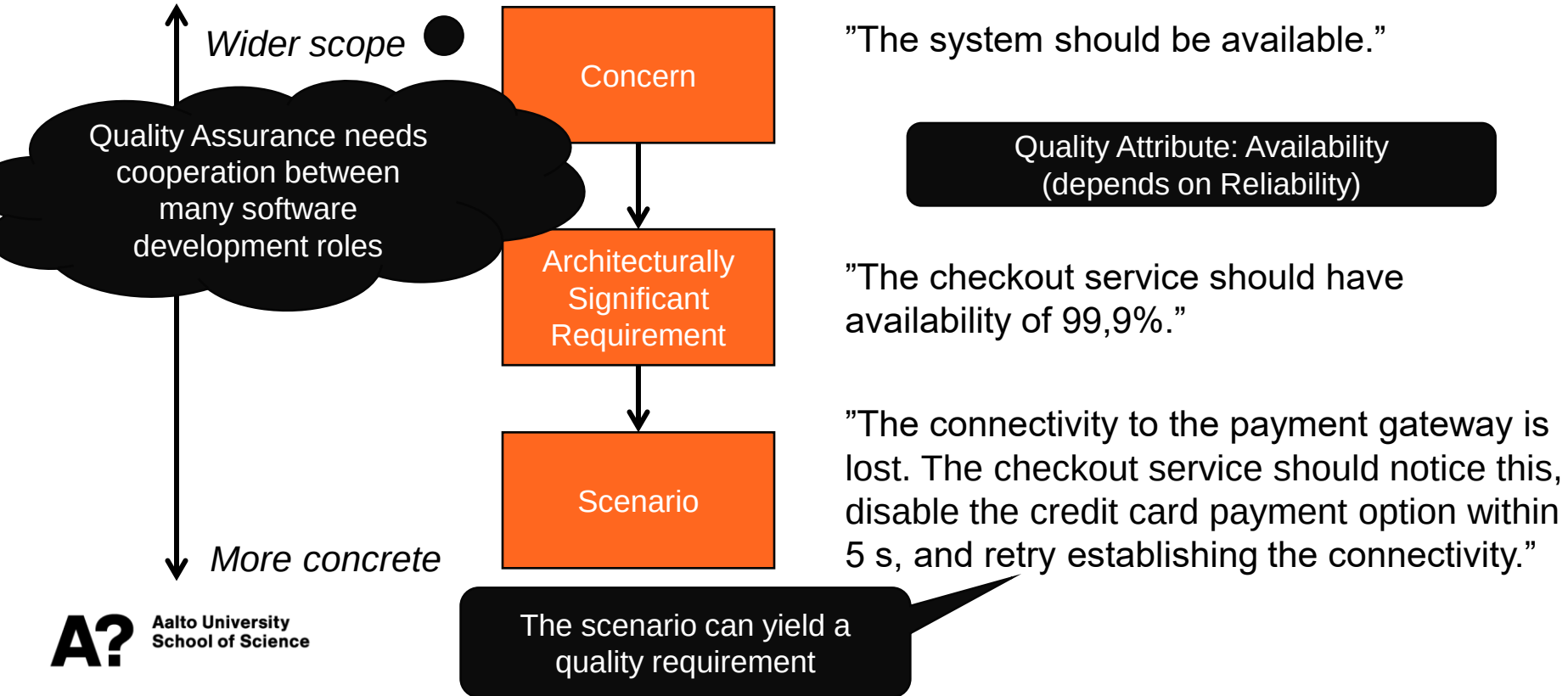
An an editor, I want to review content before it is published so that I can ensure it is correct and has the right style

Acceptance criteria

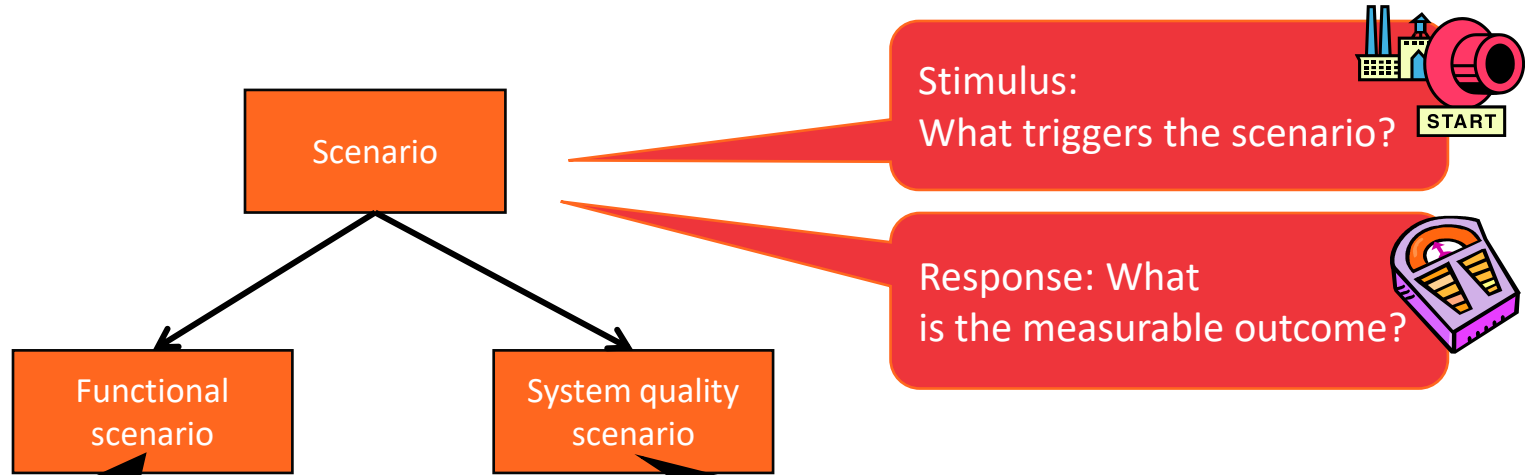
- Log in
- Create content page
- Edit content page
- Save changes
- Assign content page to editor for review

- Log in
- View content page
- Edit content page
- Add comments
- Save changes
- Re-assign content page to content owner

Linking Quality with Requirements and Software Architecture



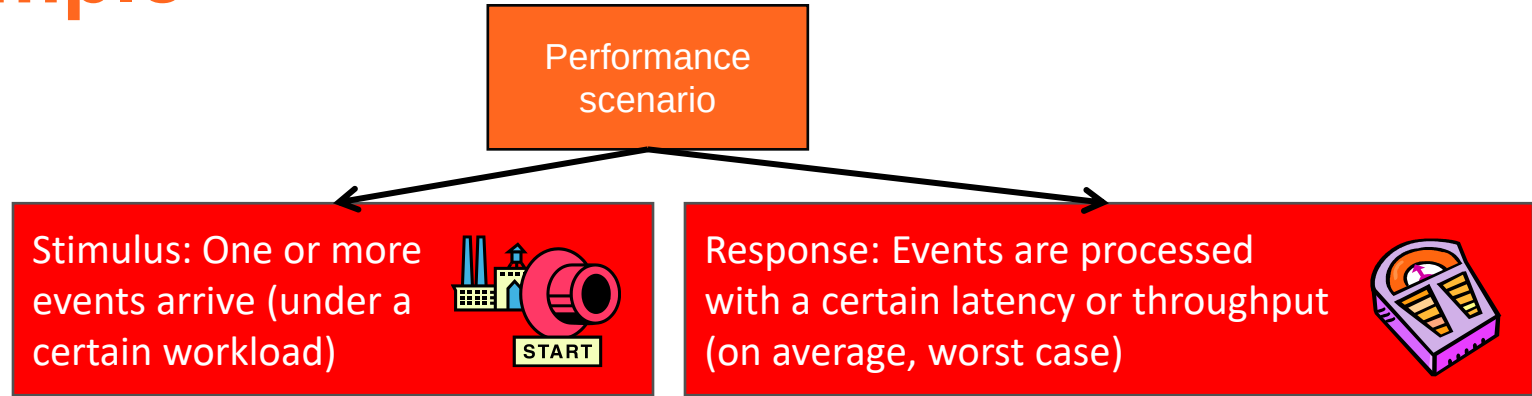
Scenarios concretise requirements



At 03:00 CET, on the first day of the month, the system calculates the order summaries per day and country and sends a report to the offices via e-mail.

The connectivity to the payment gateway is lost. The checkout service should notice this, disable the credit card payment option within 5 s, and retry establishing the connectivity.

Example



Latency: When the researcher presses *Start*, the simulation world with 100 creatures is initialized and animation begins within an average response time of one second.

Throughput: With 100 interacting creatures, the system can process, store and visualize at least 100 simulation steps per minute.

Software test case analysis and design

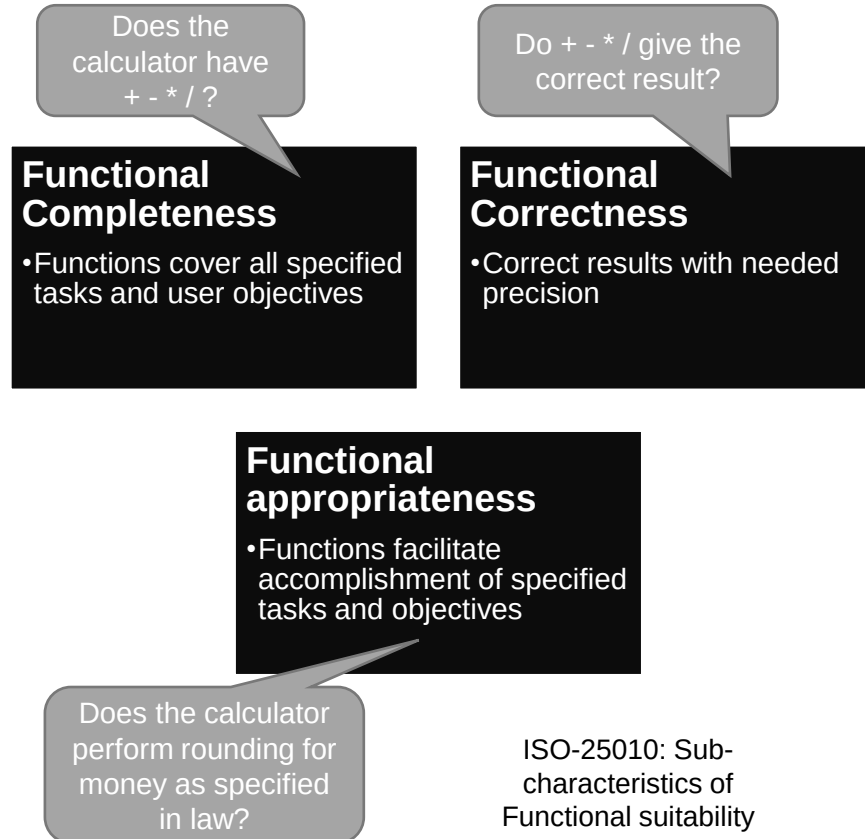


Aalto University
School of Science

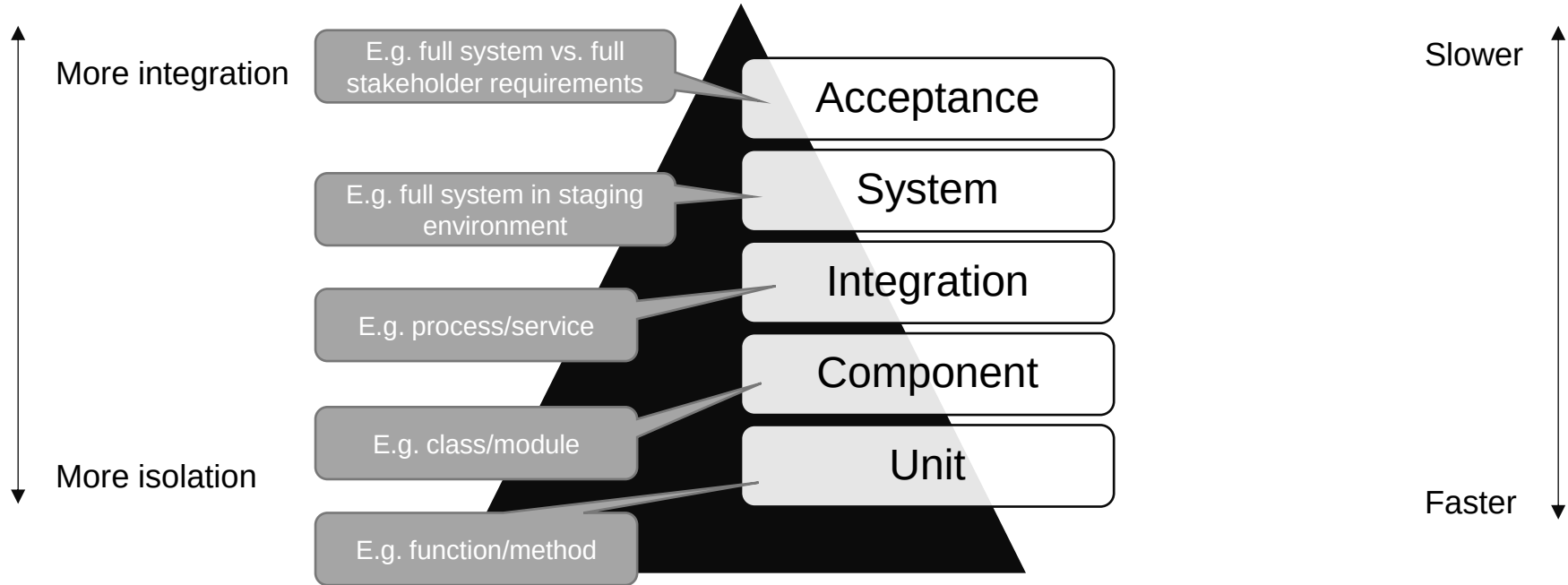
Functional suitability

- The essential purpose of the software system
 - Simple: a basic calculator
 - More complicated: an internet banking system
- “Degree to which [system] provides functions that meet stated and implied needs when used under specified conditions” (ISO 25010)
 - Each individual function can be verified as existing or not existing
 - Specific properties can be specified for a function

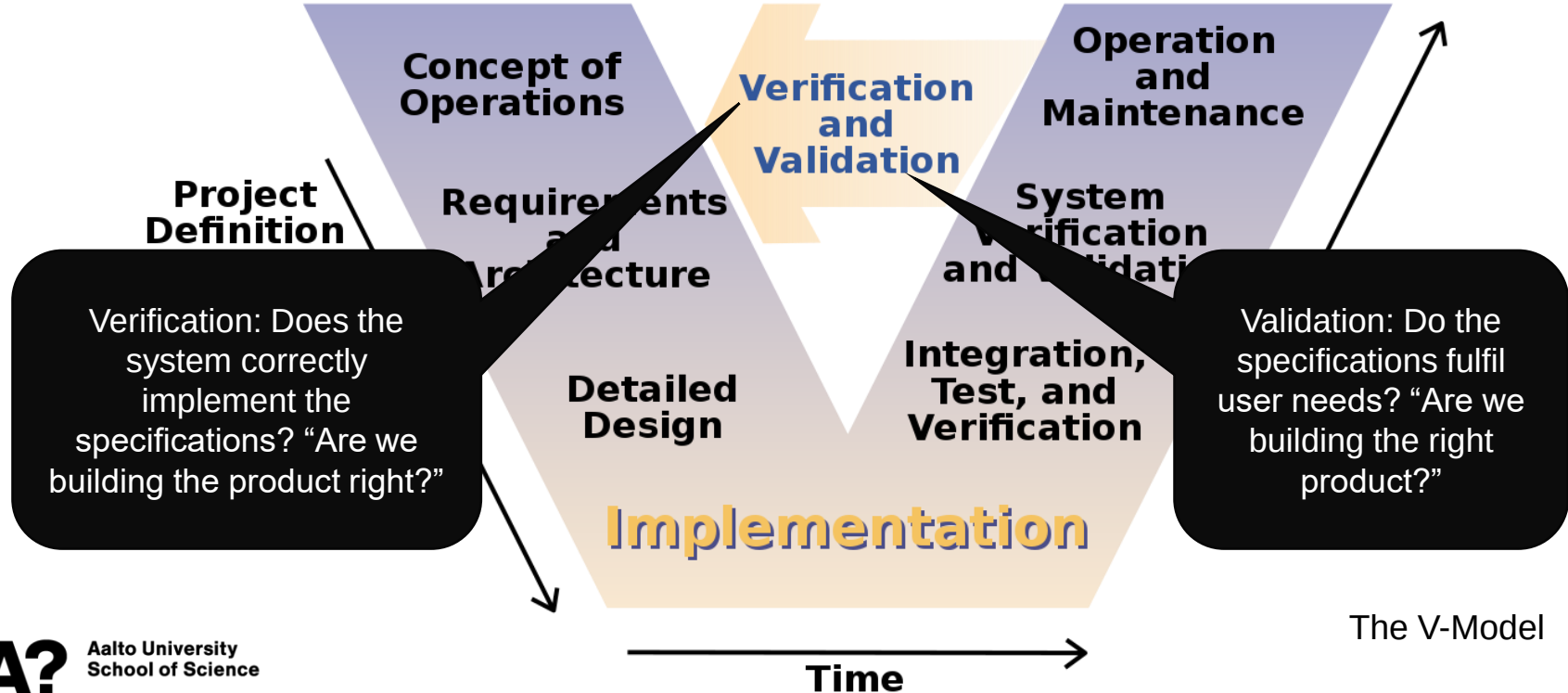
Functionality is “just the beginning” of quality



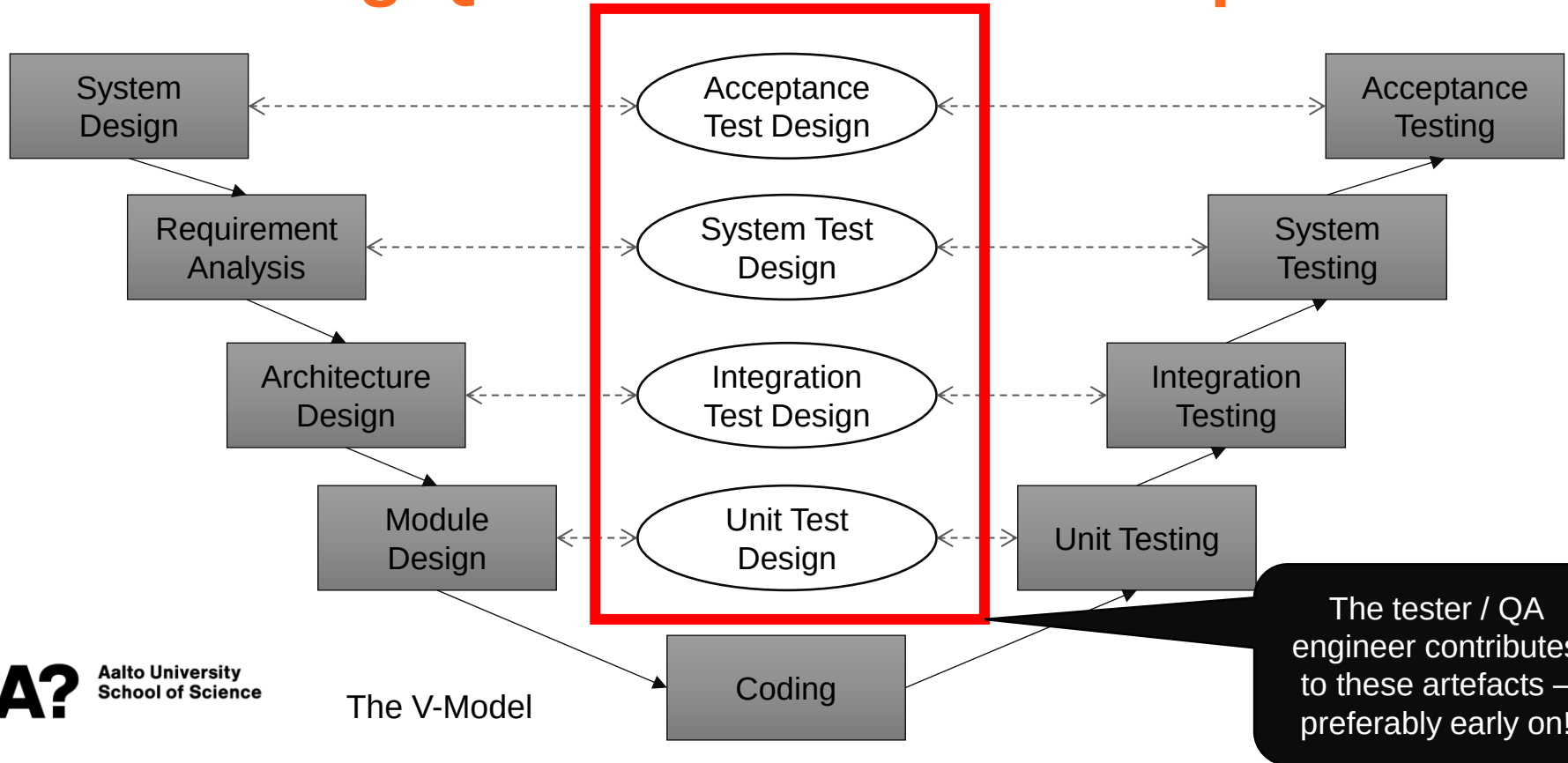
Testing pyramid: levels of testing



Positioning QA in software development



Positioning QA in software development



Different ideas of what testing is

Level 0

- No difference between testing and debugging.
- No purpose of testing
- Defects may be stumbled upon but no formalised effort to find them

Level 1

- “The purpose of testing is to show that software works.”
- Tests are designed to align with what the software does
- Confirmation bias may play a role

Level 2

- “The purpose of testing is to show that software doesn’t work.”
- Challenges testers to find defects.
- Consciously select test cases that evaluate the system under unusual circumstances, using “diabolical” test cases.

Level 3

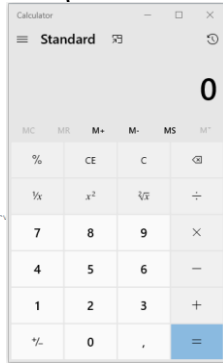
- “The purpose of testing is not to prove anything, but to reduce the perceived risk of not working to an acceptable value.”
- Gain information about the quality of the software in terms of its defects
- Provide management with an evaluation of the impact on our organisation if we ship the system in its present state

Level 4

- “Testing is not an act. It is a mental discipline that results in low-risk software without much testing effort.”
- Focus on making software testable from the start: reviews and inspections of its requirements, design, and code.
- Write code with facilities that allows the tester to easily interrogate it while it is running.
- Write self-diagnosing code that reports errors rather than requiring testers to discover them.

Concepts

System Under Test (SUT): The software system being tested



```
// Copyright (c) Microsoft Corporation. All rights reserved.  
// Licensed under the MIT license.  
  
using CalculatorApp.ViewModel.Common;  
using System;  
  
namespace CalculatorApp  
{  
    namespace Converters  
    {  
        /// <summary>  
        /// Value converter that translates true to false and vice versa.  
        /// </summary>  
        [Windows.Foundation.Metadata.WebHostHidden]  
        public sealed class RadixToStringConverter : Windows.UI.Xaml.Data.IValueConverter  
        {  
            public object Convert(object value, Type targetType, object parameter, string language)  
            {  
                // ...  
            }  
        }  
    }  
}
```



Black-box testing: Observing SUT outputs for specified inputs *without looking at the internals*

Coverage: How much of the source code has been tested?

Grey-box testing: Black-box + White-box



White-box testing: Test SUT behavior *using knowledge of its internal structure*
(Also: clear-box, glass-box, or *structural testing*)

Discussion

Give an example of a useful software test.
What makes it useful?

Test case design

- A single test case can prove the system incorrect, but it is impossible to ever prove it correct
- You would have to test all valid and invalid combinations of input data
- A good test
 - Reveals faults in the current software
 - Reveals faults in the software when it is changed in the future (regression testing)
 - Does so efficiently and effectively – strikes a balance between how much time it takes to run the test and how much coverage is achieved
- To achieve this, test cases must be *designed*, not put together as an afterthought

Test case design

- Three parts of well-designed test cases

Inputs

- Data entered by the user (keyboard, mouse, pointing gestures)
- Data from interfacing systems and devices
- Data from files and databases
- State of the system when data arrives
- Execution environment
- ...

Outputs

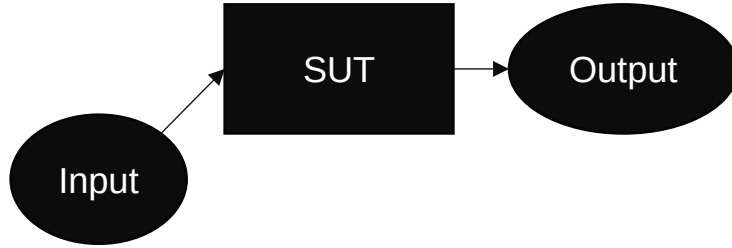
- Data displayed on screen
- Data sent to interfacing systems and devices
- Data written to files and databases
- Modified system state
- Modified execution environment
- ...

Order of execution

- *Cascading* test cases: each test case builds on the other
 - Can be smaller, simpler
 - If one case fails, the others may be invalid
- *Independent* test cases: test cases do not depend on or influence each other
 - Easier to isolate cause of failure
 - Can be larger, more complicated

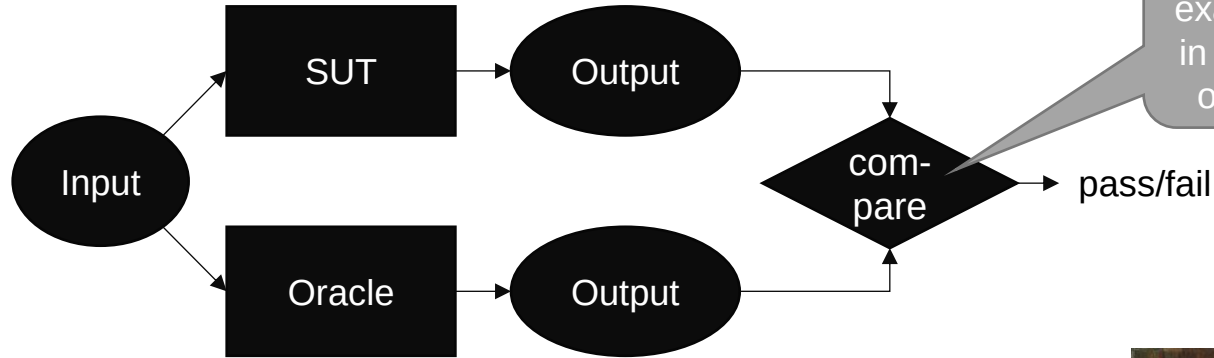
Correctness of Tests

- How do you know whether your test output is correct?



Test Oracle

- Test Oracle: A mechanism for determining the expected outputs of a test

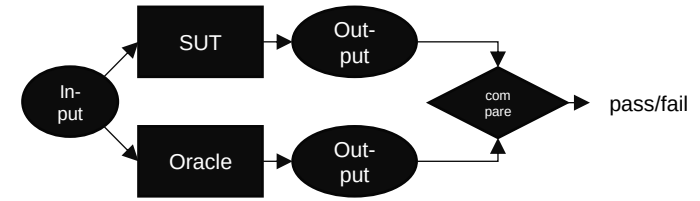


- But: authoritative (always correct) oracles might not exist (for all types of tests)
- Test Oracle (2): A tool that **helps decide** whether the program passed the test
- → Partial oracles (Weyker, 1980: On Testing Nontestable Programs)
- Where could we find partial oracles for different situations?



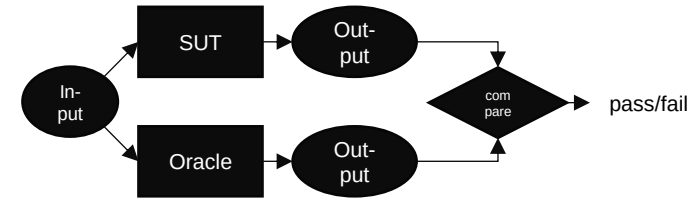
John William Waterhouse: Consulting the Oracle (1884). Public domain.

Test Oracle



- **Specified** oracles
 - Based on formal specifications
 - Model-based design: generate test oracles from formal models
 - Design by contract: assertions
- **Derived** oracles
 - Based on information derived from system artefacts (e.g. documentation, system execution results)
 - Regression testing: assumes that results from a previous system version can be used as an oracle for a future version
 - Performance testing: previous performance measures can be used as an oracle for future versions
 - Pseudo-oracle: a separately written program that takes the same inputs; compare outputs

Test Oracle

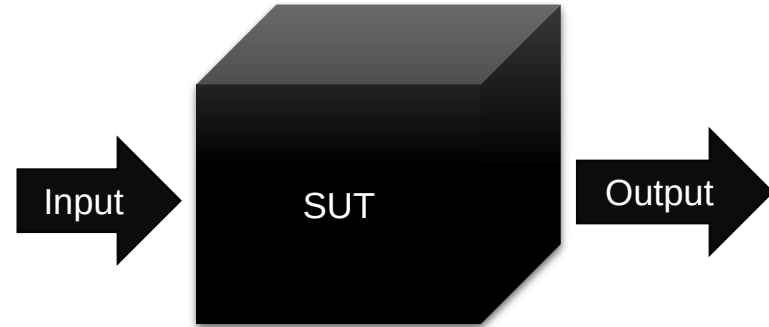


- **Implicit** oracles
 - Based on implied information and assumptions
 - Program crashes may imply a conclusion about unwanted behaviour (but a crash is not always a priority if the system can recover)
 - Negative testing, fuzz testing
- **Human** oracles
 - When other oracles cannot be used
 - Quantitative approach: find enough information about the SUT (e.g. test results) for a stakeholder to make decisions (e.g. release decision)
 - Qualitative approach: analyse SUT and its context to find representative and suitable inputs and outputs (e.g. using realistic input data and making sense of results)
 - Can be guided by heuristic approaches – gut instinct, rule of thumb, checklists, experience

Black-box testing

Black-box testing

- Testing that does not examine the internals of the SUT
 - Inputs
 - Outputs
- Also called *functional testing*, *behavioural testing*, or *specification-based testing*
- Can be applied on all levels of testing (unit, integration, system, acceptance)
- Automated tests can be black-box tests
- Exploratory testing is a kind of black-box testing
- Advantage: can be efficient and effective
- Disadvantage: can never be sure of how much of the system has been exercised
- We focus here on general techniques that can be applied in more rigid tests (both manual and automated)



Some very specific inputs could lead to a rare execution path that cannot be known without seeing the code

```
if (student.number == 1234567) {  
    student.grade = 5;  
}
```

Equivalence partitioning

- Divide test input data into a set of classes
- Select one input value from each class
- The input value represents the class
- Create one test with each chosen input value
- → One test per class
- Significantly reduces number of test cases
- Keeps reasonable test coverage

Example

- Inputs: integers 1-100
- 100 tests?
- With equivalence partitioning:

Invalid low class:
A number below 1

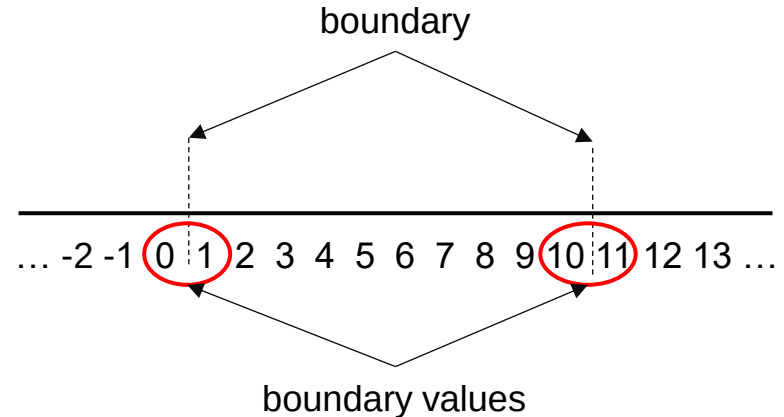
Valid class:
A number between
1 and 100

Invalid high class:
A number above
100

Boundary value analysis (BVA)

- Assumption: defect density is clustered towards value range boundaries
 - Errors in implementing comparisons ($<$ vs. $<=$)
 - Errors in implementing loop termination conditions
 - Errors in interpreting requirements at the boundaries
- Test cases can concentrate on these boundary values
- Identify equivalence classes
- Identify boundaries of each equivalence class
- Create test cases for:
 - One point just above the boundary
 - One point just below the boundary

Example:
Valid inputs: integers 1-10



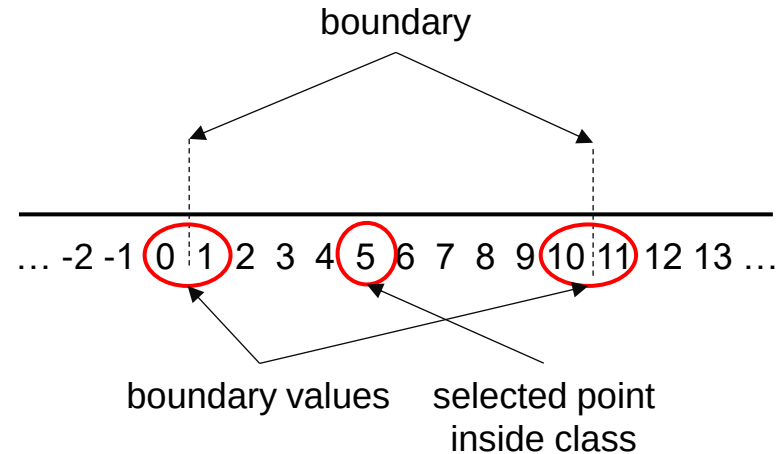
Robustness testing (variant of BVA)...

Some define
BVA as
robustness
testing

- Testing across the valid and invalid partitions
- Create test cases for:
 - One point just above the boundary
 - One point just below the boundary
 - One point inside the class
- BVA and its variants can be used to generate systematic tests and reduce the number of test cases
- Multiple parameters: only change one parameter at a time

Example:

Valid inputs: integers 1-10



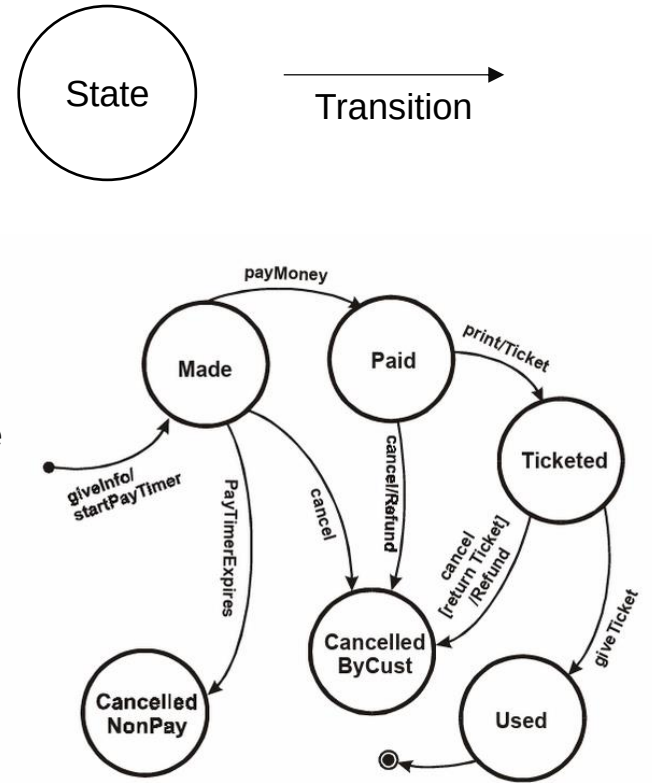
Decision table testing

- Decision table:
 - A systematic way of recording complex business rules
 - Serves as a guide to creating test cases
- List all *input conditions* that can occur and all *actions* that can arise from them
 - Input conditions on the top
 - Resulting actions on the bottom
 - Each column represents a business *rule*
 - Each column becomes a test case
- If the conditions are value ranges, consider testing at both the low and high end of the range (compare: BVA/robustness testing)
- Every possible combination does not need to be entered, only those that represent actual business rules

	Rule 1	Rule 2	Rule 3	Rule 4
<i>Input conditions</i>				
Customer with loyalty card?	Yes	Yes	No	No
Spend > 200?	Yes	No	Yes	No
<i>Actions</i>				
Discount	10%	5%	2%	0%
Accumulate loyalty points?	Yes	Yes	No	No
	↓ Test case 1	↓ Test case 2	↓ Test case 3	↓ Test case 4

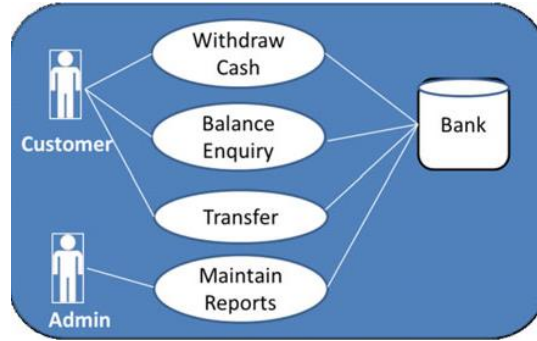
State transition testing

- Sometimes outputs depend on system state: system behaviour is triggered by state transitions
- State: a “stable” system condition – does not change unless a trigger event occurs
- Transition: a description of the trigger that moves the system from one state to another
- Create a state diagram of the system’s states and transitions
- Can also create a state table
- Test cases can be created depending on desired coverage
 - All states visited at least once (weak)
 - All events triggered at least once (weak)
 - All paths executed at least once (stronger)
 - All transitions exercised at least once (strongest)



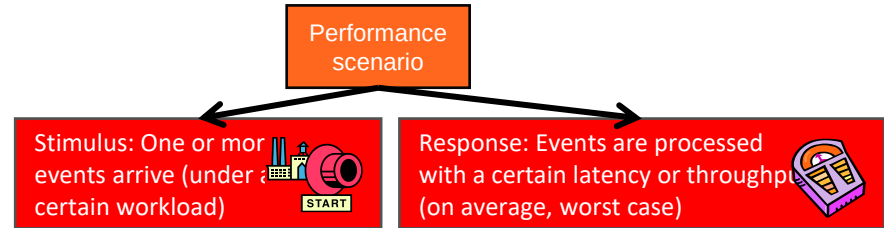
Copeland. (2003). A Practitioner's Guide to Software Test Design.

Use case and scenario-based testing



- Use cases often lack necessary detail for testing
- Add details, e.g.
 - Preconditions
 - Main success scenario
 - Success end conditions
 - Failed end conditions
 - Response time
 - Frequency
 - ...

A detailed specification is an important input to test design



Latency: When the researcher presses *Start*, the simulation world with 100 creatures is initialized and animation begins within an average response time of one second.

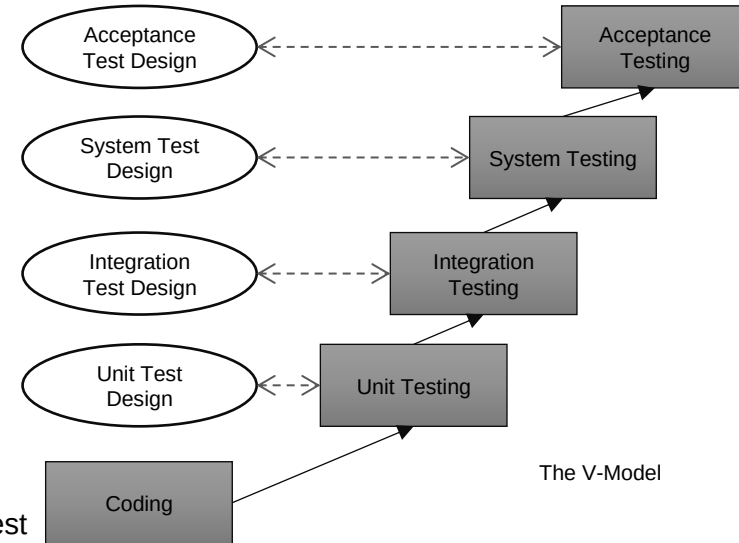
Throughput: With 100 interacting creatures, the system can process, store and visualize at least 100 simulation steps per minute.

The beginnings of a performance test case

Test automation

Automated test generation is something different

- Automated tests use software programs that take another program as input and produce a test result as output
- The test can be performed on the source code (static) or the running program (dynamic)
- Usually a test framework and a test runner are used
- The program to test is called the **System Under Test** (SUT)
- Types of tests (examples)
 - Unit tests
 - Acceptance tests
 - Performance tests
 - Load tests
 - ...
- Automated test *design* is often more time-consuming than manual test design because nothing can be left open to interpretation
- Automated test *execution* can be faster than manual – but running the whole test suite can be slow



Test automation: Unit test example

System Under Test

```
class StudentDirectory:
```

```
    names = []
```

```
    def add_name(self, name):
```

```
        if name not in self.names:
```

```
            self.names.append(name)
```

Test that names are only added once

```
directory = StudentDirectory()
```

```
directory.add_name("Emma")
```

```
directory.add_name("Emma")
```

```
if len(directory.names) != 1:
```

```
    print("Test failed")
```

```
else:
```

```
    print("Test passed")
```

Some important guidelines for unit tests

- **Well named:** The test name explains what it tests
- **Clearly defined**
 - Each test tests one thing
 - Inputs and outputs are unambiguous
- **Repeatable (deterministic):** The test gives the same results no matter how many times it is run
- **Isolated**
 - No interactions or dependencies between tests
 - No dependencies on outside factors (e.g., file system, database)
 - Each test leaves the system in a well-defined state
- **Robust:** The test does not break if the unit internals are changed (related to isolation)
- **Fast:** The total testing time is reasonable even with thousands of tests
- **Documented:** Explain the purpose of the test, the test scenario, and the expected behaviour when the scenario is invoked

Next steps

- Individual assignments in MyCourses
 - Quiz on equivalence partitioning
 - Quiz on boundary value analysis
 - Unit testing 1

Use the related reading
for more details.
Reserve enough time to
set up your environment.

Deadline: before next lecture (Tuesday by 10:00)

- Group assignments in MyCourses
 - Group registration, Group name choice (DL: 20.9 by 16:00)
 - Analysing quality using ISO/IEC 25010 (DL: 24.9)
- Getting help
 - Ask in the course chat, see Communication section in MyCourses
 - Personal questions by email



Study smarter, not harder!

- Plan: when and where
- Go deep at your own pace
- Get some rest
- Practice makes perfect
- Enjoy it ☺

Software Testing and Quality Assurance

Lecture 4

Fabian Fagerholm

fabian.fagerholm@aalto.fi



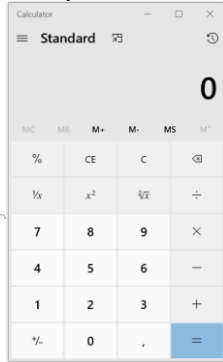
Aalto University
School of Science



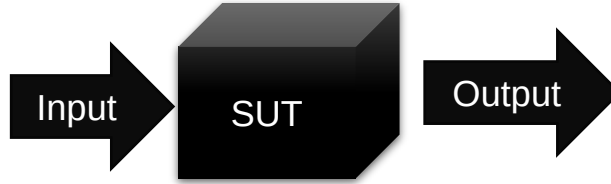
Week	Lecture	Topic	Assignment deadlines (Tuesdays 10:00 unless otherwise specified)
36	3.9.2024	Introduction and practicalities	
37	10.9.2024	Software quality	Individual assignments 1
38	17.9.2024	Software testing: levels, test case analysis and design	Group registration (DL: 20.9.)
39	24.9.2024	Testing techniques: Black-box, white-box testing	Individual assignments 2, Group assignment 1
40	1.10.2024	Testing techniques: Manual testing	Individual assignments 3
41	8.10.2024	Testing techniques: Black-box, white-box, manual testing	Individual assignments 4
42	15.10.2024	(No lecture)	
43	22.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 5
44	29.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 6, Group assignment 2
45	5.11.2024	Guest lecture	Individual assignments 7
46	12.11.2024	Continuous Integration and Continuous Delivery/Deployment, Test management, and Course summary	Individual assignments 8
47	19.11.2024	(No lecture)	Individual assignments 9
48	26.11.2024	(No lecture)	Individual assignments 10, Group assignment 3

Concepts

System Under Test (SUT): The software system being tested



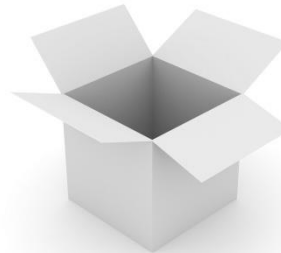
```
// Copyright (c) Microsoft Corporation. All rights reserved.  
// Licensed under the MIT license.  
  
using CalculatorApp.ViewModel.Common;  
using System;  
  
namespace CalculatorApp  
{  
    namespace Converters  
    {  
        /// <summary>  
        /// Value converter that translates true to false and vice versa.  
        /// </summary>  
        [Windows.Foundation.Metadata.WebHostHidden]  
        public sealed class RadioToStringConverter : Windows.UI.Xaml.Data.IValueConverter  
        {  
            public object Convert(object value, Type targetType, object parameter, string language)  
            {  
                // ...  
            }  
        }  
    }  
}
```



Black-box testing: Observing SUT outputs for specified inputs *without looking at the internals*

Coverage: How much of the source code has been tested?

Grey-box testing: Black-box + White-box

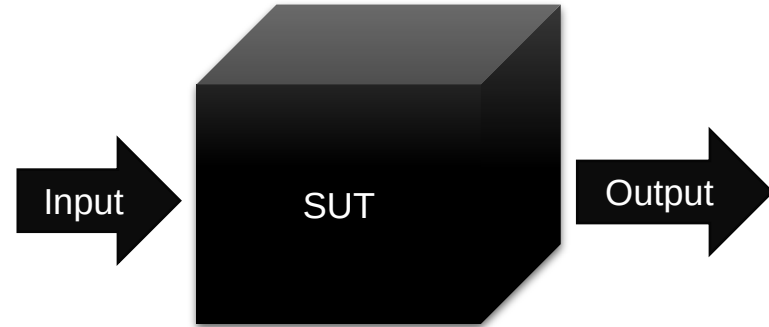


White-box testing: Test SUT behavior *using knowledge of its internal structure*
(Also: clear-box, glass-box, or *structural testing*)

Black-box testing

Black-box testing

- Testing that does not examine the internals of the SUT
 - Inputs
 - Outputs
- Also called *functional testing*, *behavioural testing*, or *specification-based testing*
- Can be applied on all levels of testing (unit, integration, system, acceptance)
- Automated tests can be black-box tests
- Exploratory testing is a kind of black-box testing
- Advantage: can be efficient and effective
- Disadvantage: can never be sure of how much of the system has been exercised
- We focus here on general techniques that can be applied in more rigid tests (both manual and automated)



Some very specific inputs could lead to a rare execution path that cannot be known without seeing the code

```
if (student.number == 1234567) {  
    student.grade = 5;  
}
```

Techniques for designing black-box tests

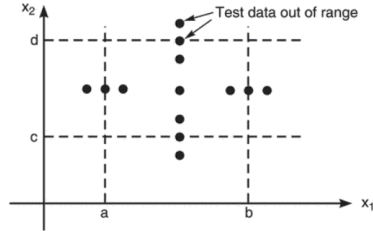
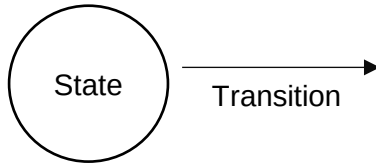


FIGURE 4.2 Robustness Test Cases.
Chopra (2018)

Boundary value analysis
Robustness testing

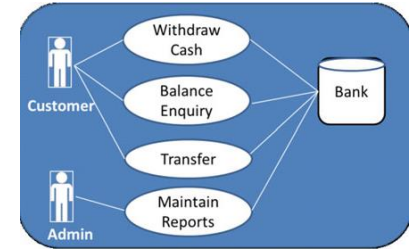


State transition testing

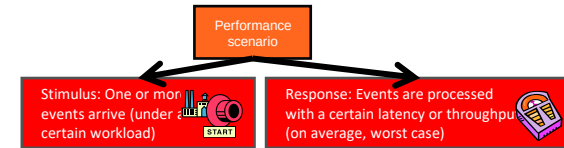
	Rule 1	Rule 2	Rule 3	Rule 4
<i>Input conditions</i>				
Customer with loyalty card?	Yes	Yes	No	No
Spend > 200?	Yes	No	Yes	No
<i>Actions</i>				
Discount	10%	5%	2%	0%
Accumulate loyalty points?	Yes	Yes	No	No

Test case 1 Test case 2 Test case 3 Test case 4

Decision table testing



Use case based testing



Latency: When the researcher presses Start, the simulation world with 100 creatures is initialized and animation begins within an average response time of one second.

Throughput: With 100 interacting creatures, the system can process, store and visualize at least 100 simulation steps per minute.

Scenario based testing

Equivalence partitioning

- Divide test input data into a set of classes
- Select one input value from each class
- The input value represents the class
- Create one test with each chosen input value
- → One test per class
- Significantly reduces number of test cases
- Keeps reasonable test coverage

Example

- Inputs: integers 1-100
- 100 tests?
- With equivalence partitioning:

Invalid low class:
A number below 1

Valid class:
A number between
1 and 100

Invalid high class:
A number above
100

Equivalence class testing: two approaches

- Testing-by-contract
 - Modules are designed to work under certain preconditions
 - Create test cases only for situations where preconditions are met
 - No test cases for conditions that the module does not claim to work under
- Defensive-testing
 - Modules are designed to accept any input
 - Create test cases for both valid and invalid preconditions

Contract for `openFile(filename)`

- Will return a file handle if `filename` exists
- Behaviour if `filename` does not exist: undefined
- Test case: use an existing filename

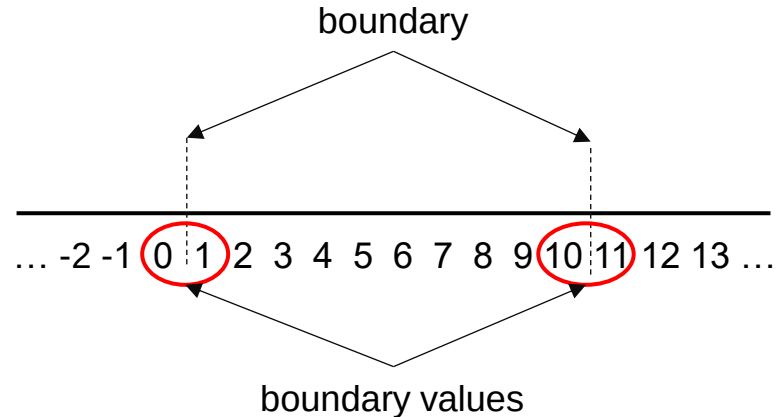
Contract for `openFile(filename)` using defensive design

- Will return a file handle if `filename` exists
- Will return an error if `filename` does not exist
- Test case 1: use an existing filename
- Test case 2: use a non-existing filename

Boundary value analysis (BVA)

- Assumption: defect density is clustered towards value range boundaries
 - Errors in implementing comparisons ($<$ vs. $<=$)
 - Errors in implementing loop termination conditions
 - Errors in interpreting requirements at the boundaries
- Test cases can concentrate on these boundary values
- Identify equivalence classes
- Identify boundaries of each equivalence class
- Create test cases for:
 - One point just above the boundary
 - One point just below the boundary

Example:
Valid inputs: integers 1-10



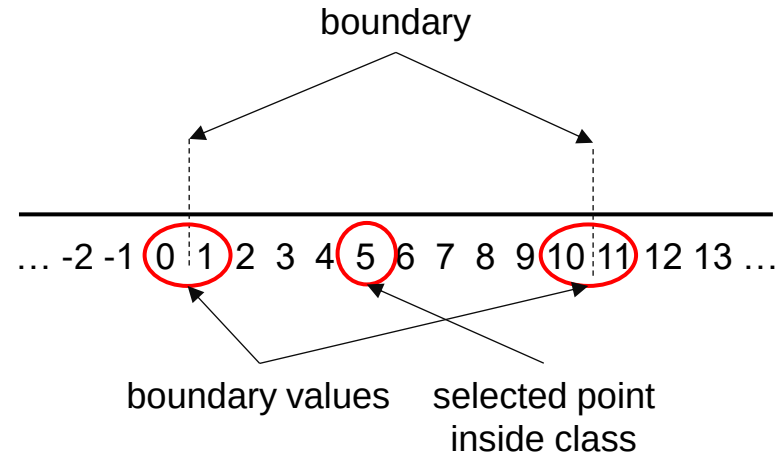
Robustness testing (variant of BVA)...

Some define
BVA as
robustness
testing

- Testing across the valid and invalid partitions
- Create test cases for:
 - One point just above the boundary
 - One point just below the boundary
 - One point inside the class
- BVA and its variants can be used to generate systematic tests and reduce the number of test cases
- Multiple parameters: only change one parameter at a time

Example:

Valid inputs: integers 1-10



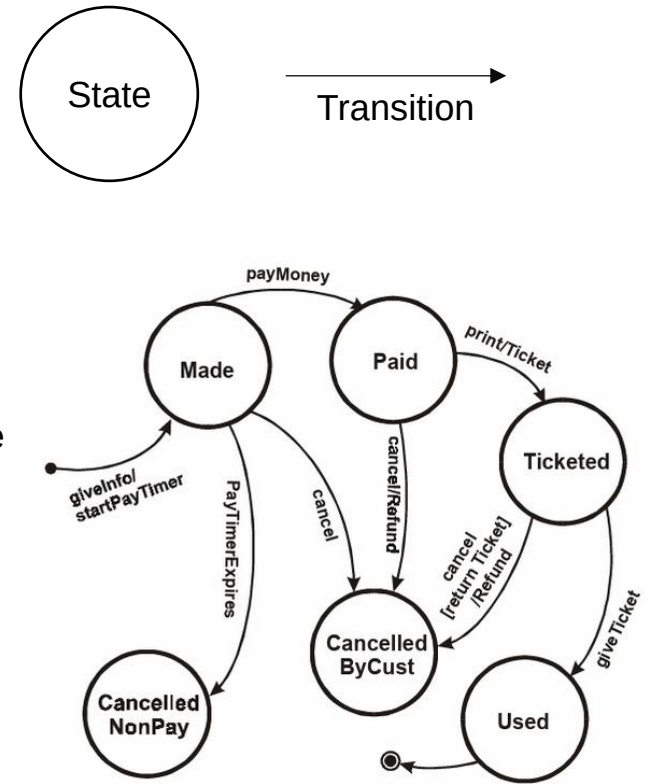
Decision table testing

- Decision table:
 - A systematic way of recording complex business rules
 - Serves as a guide to creating test cases
- List all *input conditions* that can occur and all *actions* that can arise from them
 - Input conditions on the top
 - Resulting actions on the bottom
 - Each column represents a business *rule*
 - Each column becomes a test case
- If the conditions are value ranges, consider testing at both the low and high end of the range (compare: BVA/robustness testing)
- Every possible combination does not need to be entered, only those that represent actual business rules

	Rule 1	Rule 2	Rule 3	Rule 4
<i>Input conditions</i>				
Customer with loyalty card?	Yes	Yes	No	No
Spend > 200?	Yes	No	Yes	No
<i>Actions</i>				
Discount	10%	5%	2%	0%
Accumulate loyalty points?	Yes	Yes	No	No
	↓ Test case 1	↓ Test case 2	↓ Test case 3	↓ Test case 4

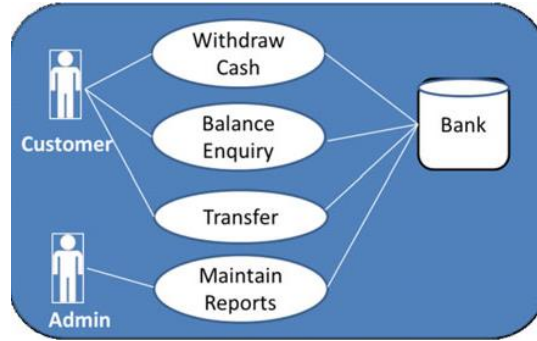
State transition testing

- Sometimes outputs depend on system state: system behaviour is triggered by state transitions
- State: a “stable” system condition – does not change unless a trigger event occurs
- Transition: a description of the trigger that moves the system from one state to another
- Create a state diagram of the system’s states and transitions
- Can also create a state table
- Test cases can be created depending on desired coverage
 - All states visited at least once (weak)
 - All events triggered at least once (weak)
 - All paths executed at least once (stronger)
 - All transitions exercised at least once (strongest)



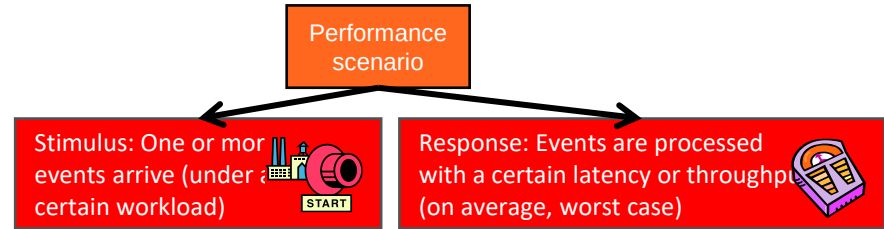
Copeland. (2003). A Practitioner's Guide to Software Test Design.

Use case and scenario-based testing



- Use cases often lack necessary detail for testing
- Add details, e.g.
 - Preconditions
 - Main success scenario
 - Success end conditions
 - Failed end conditions
 - Response time
 - Frequency
 - ...

A detailed specification is an important input to test design



Latency: When the researcher presses *Start*, the simulation world with 100 creatures is initialized and animation begins within an average response time of one second.

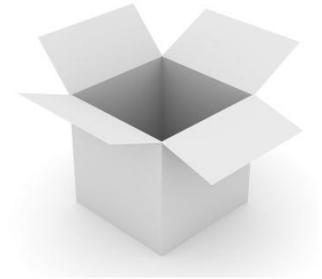
Throughput: With 100 interacting creatures, the system can process, store and visualize at least 100 simulation steps per minute.

The beginnings of a performance test case

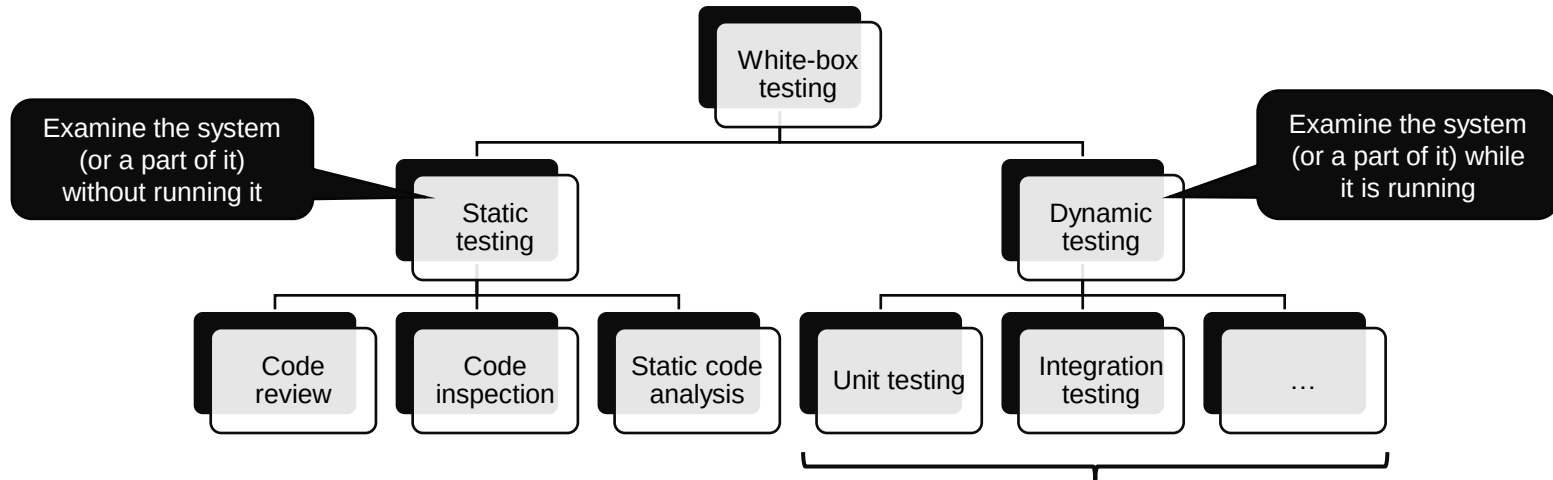
White-box testing

White-box testing

- Also: clear-box, glass-box, or *structural testing*
- Designing tests for the SUT *using knowledge of its internal structure*
- *Today: techniques for designing test cases based on a white-box approach*



[Photo CC BY-SA-NC](#)



Potentially all levels of the testing pyramid – just like in black-box testing but based on knowledge of the internal structure

Dynamic white-box testing

- Creating test cases using knowledge of internal structure:
 - Branches
 - Individual conditions
 - Statements
- General approach
 - Analyse SUT implementation
 - Identify **paths** through SUT
 - Choose input to cause execution of selected paths
 - Determine expected results for selected paths
 - Run tests
 - Compare actual outputs with expected outputs
 - Make conclusions about correctness of SUT implementation

Emphasis is on code **paths**

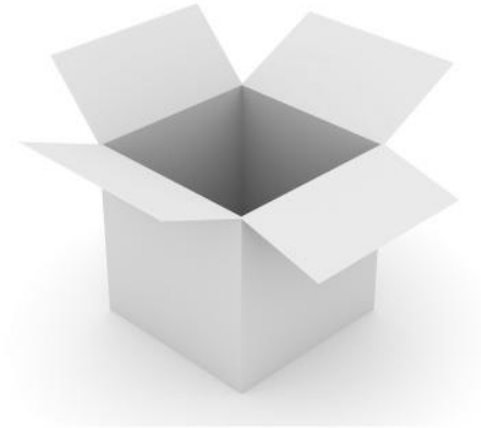
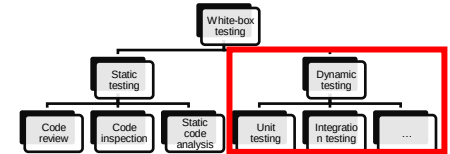


Photo CC BY-SA-NC



Dynamic white-box testing

- Possible applications on different testing levels
 - Unit: Test paths within individual modules before integration
 - Integration: Test paths between modules (also: data flow)
 - System: End-to-end application flow from user perspective
- Challenges
 - Cannot test all possible paths
 - Test cases may not detect data sensitivity errors (e.g., $p=q/r$ may execute correctly except when $r=0$)
 - Assumes control flow is correct (non-existent paths cannot be discovered)
 - The tester must have programming skills to evaluate the SUT

Dynamic white-box testing emphasises paths – test case design is based on path analysis

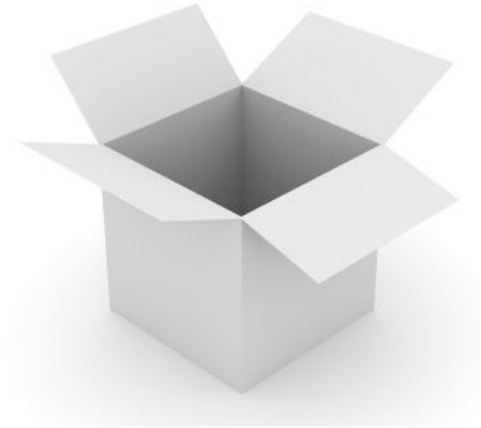
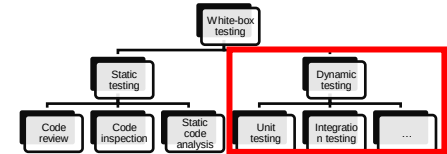


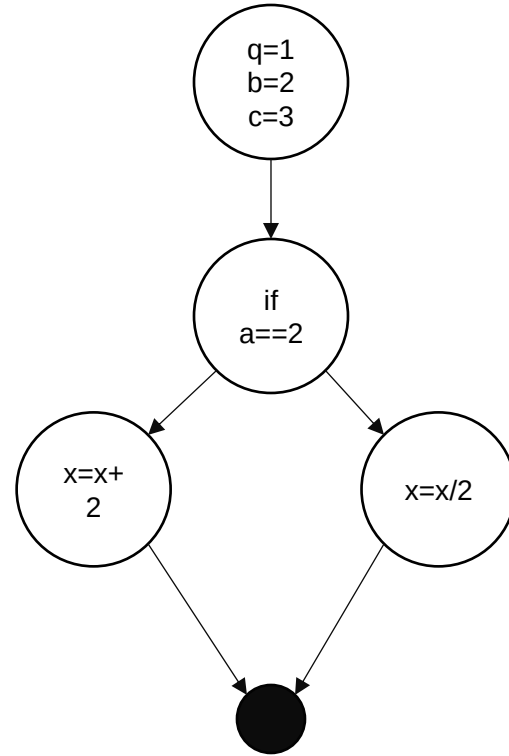
Photo CC BY-SA-NC



Control flow testing

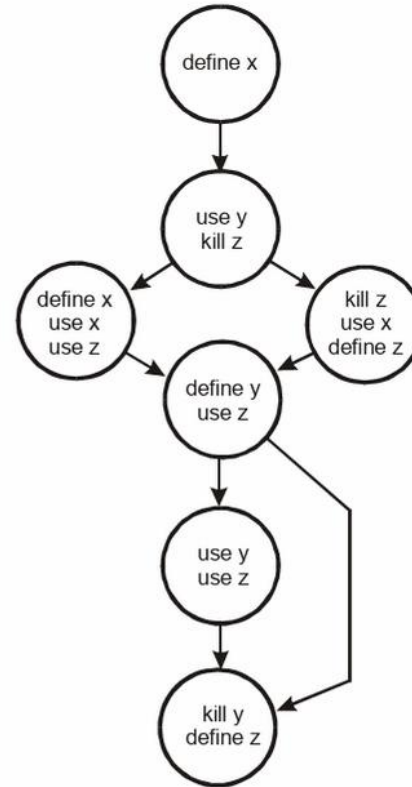
- Convert the code into a graph
- Analyse the paths through the graph
- Create test cases based on the analysis
- There are different possible levels of path coverage

```
q=1
b=2
c=3
if (a==2):
    x=x+2
else:
    x=x/2
```



Data flow testing

- Follow the creation, use, and destruction of variables, e.g.
 - Ensure that variables are not used before they are defined
 - Ensure that variables are destroyed
 - Ensure that variables are not used after they are destroyed
- Data flow testing tools can help
- Many compilers do data flow testing
- (Not covered in depth in this course)



(Copeland 2003)

Statement coverage

- What portion of statements are executed by our tests?
- Expression:
 - A syntactic unit in a programming language
 - May be *evaluated* to determine its value
 - Examples: $3 + 5$ (evaluates to 8), $3 > 5$ (evaluates to false)
- Statement:
 - A syntactic unit in a programming language (at least in imperative languages)
 - Performs some *action*
 - Example (simple): $y = 8$
 - Example: (compound):
if (<condition>):
 <sequence>

(Statements in <sequence> counted separately)

Statement coverage =

$$\frac{\text{Total statements executed}}{\text{Total statements in program}}$$

Q: How many statements does this code snippet have?

```
if (x > 5):  
    y = 8
```

A: Two statements.

Q: How many tests do we need to get 100% statement coverage?

A: One (any value for x that is larger than 5). To get 100% statement coverage, the condition has to be true.

Statement coverage

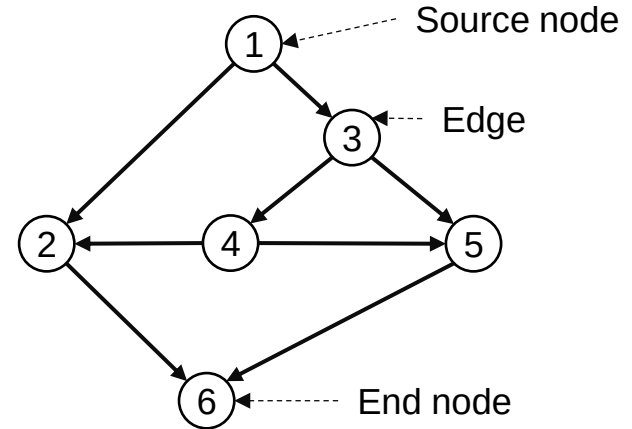
- In principle, we want 100% statement coverage, but ...
- For a set of sequential statements (usually easy, but):
 - Exceptions (e.g. divide by zero) will exit the code early
 - The code could be entered from multiple points
- For code with conditional branches (if-then-else two-way-choices):
 - One test case per branch is needed
- For code with loops, failures are often in boundary conditions. We need test cases that:
 - Skip the loop (termination condition is true before the loop)
 - Check the loop for one iteration ($n = 1$)
 - Check the loop for values close to n (boundary value analysis!)

Statement coverage =

$$\frac{\text{Total statements executed}}{\text{Total statements in program}}$$

Path coverage

- Create a graph representation of the program
 - Start from the beginning
 - Take any path to completion
- Stronger coverage condition than statement coverage
 - 100% statement coverage does not necessarily exercise all code paths
- The more paths are covered, the better
- But what about the nodes?



Path coverage =

$$\frac{\text{Total paths executed}}{\text{Total paths in program}}$$

Some possible paths:

P1: 1 → 2 → 6

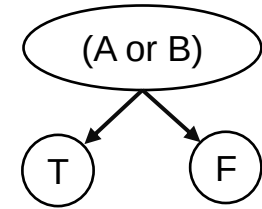
P2: 1 → 3 → 5 → 6

P3: 1 → 3 → 4 → 5 → 6

P4: 1 → 3 → 4 → 2 → 6

Decision coverage

if (A or B):
...



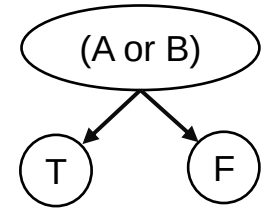
- Requirements for test:
 - There must be one test for each decision outcome (= edge in the graph)

A	B	(A or B)
T	F	T
F	F	F

- Problems
 - The effect of B is not tested: cannot distinguish between decision (A or B) and decision A
- Minimum of two tests per decision

Condition coverage

if (A or B):
...



- Requirements for test:
 - Each condition takes on all possible outcomes at least once
 - (Does not require decision to take on each outcome)

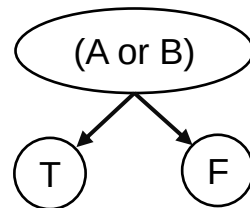
A	B	(A or B)
T	F	T
F	T	T

- Problems
 - The decision does not take on all possible outcomes
- Minimum of two tests per decision

Condition/decision coverage

if (A or B):

...



- Combination of condition and decision coverage
- Requirements for test:
 - There must be one test for each decision outcome
 - Each condition takes on all possible outcomes at least once

A	B	(A or B)
T	T	T
F	F	F

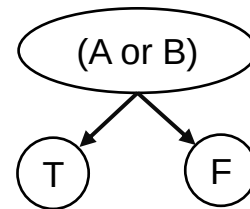
- Problems
 - Cannot distinguish the correct expression (A or B) from A or B or (A and B)

Modified condition/decision coverage

- Stronger coverage condition than condition/decision coverage
- Requirements for test:
 - Each entry and exit point is covered
 - Each decision takes every possible outcome
 - Each condition takes every possible outcome
 - Each condition must be shown to independently affect the outcome
- Minimum of $n+1$ test cases for a decision with n inputs

Trivial in this example

if (A or B):
...



A	B	(A or B)
T	F	T
F	T	T
F	F	F

Example for more complicated code

- Designing tests with the desired coverage can be hard!

```
def do_it(c1, c2, c3):  
    x = 0  
    if (c1):  
        x += 1  
    if (c2):  
        x -= 1  
    if (c3):  
        x += 3  
    return x
```

How many test cases are required?

What should the inputs be for each case?

Statement coverage

Check that all statements in the code are executed
For 100% coverage: 1 test case (true, true, true)

Condition coverage

Check that all possible results of conditions occur
For 100% coverage: 2 test cases (true, true, true), (false, false, false) – or any other combination where each conditional is tested once with true and once with false

Path coverage

Every possible path through the code is executed
For 100% coverage: 8 test cases – two alternatives for each conditional $\rightarrow 2 * 2 * 2 = 8$

Acceptance testing

- “[Testing], which when completed successfully, will result in the customer accepting the software and [paying for it]” (Copeland 2003)
- A set of test cases that are agreed between supplier and buyer to balance completeness and cost
- Conducted to determine if the requirements of a specification or contract are met
- User acceptance testing (UAT): verifying that the solution works for the user
- Operational acceptance testing (OAT): determine operational readiness (pre-release) of the software
- In Extreme programming and agile terminology:
 - Scenarios to test when a user story has been correctly implemented
 - Specified by the customer
- Acceptance tests are usually black-box tests

Acceptance testing with Robot Framework

- Test suite: one .robot file
- Contains one or more test cases
- That utilise keywords
- That can be defined in .resource files
- Or in built-in or custom libraries

```
*** Settings ***
Documentation    A test suite for valid login.
...
...             Keywords are imported from the resource file
Resource        keywords.resource
Default Tags    positive

*** Test Cases ***

Login User with Password
    Connect to Server
    Login User      ironman    1234567890
    Verify Valid Login    Tony Stark
    [Teardown]    Close Server Connection

Denied Login with Wrong Password
    [Tags]    negative
    Connect to Server
    Run Keyword And Expect Error    *Invalid Password    Login User    ironman    123
    Verify Unauthorised Access
    [Teardown]    Close Server Connection
```

Next steps

- Individual assignments in MyCourses
 - Quiz on robustness testing
 - Theory assignment for statement coverage and MC/DC
 - Unit testing 2
 - Acceptance testing 1

Use the related reading for more details.
Reserve enough time to set up your environment.

Deadline: before next lecture (Tuesday by 10:00)

- Getting help
 - Ask in the course chat, see Communication section in MyCourses
 - Personal questions by email



Study smarter, not harder!

- Plan: when and where
- Go deep at your own pace
- Get some rest
- Practice makes perfect
- Enjoy it ☺

Software Testing and Quality Assurance

Lecture 5

Fabian Fagerholm

fabian.fagerholm@aalto.fi



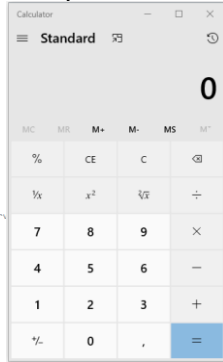
Aalto University
School of Science



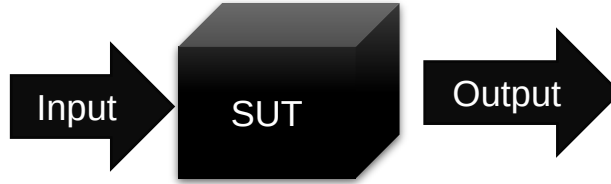
Week	Lecture	Topic	Assignment deadlines (Tuesdays 10:00 unless otherwise specified)
36	3.9.2024	Introduction and practicalities	
37	10.9.2024	Software quality	Individual assignments 1
38	17.9.2024	Software testing: levels, test case analysis and design	Group registration (DL: 20.9.)
39	24.9.2024	Testing techniques: Black-box, white-box testing	Individual assignments 2, Group assignment 1
40	1.10.2024	Testing techniques: Manual testing	Individual assignments 3
41	8.10.2024	Testing techniques: Black-box, white-box, manual testing	Individual assignments 4
42	15.10.2024	(No lecture)	
43	22.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 5
44	29.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 6, Group assignment 2
45	5.11.2024	Guest lecture	Individual assignments 7
46	12.11.2024	Continuous Integration and Continuous Delivery/Deployment, Test management, and Course summary	Individual assignments 8
47	19.11.2024	(No lecture)	Individual assignments 9
48	26.11.2024	(No lecture)	Individual assignments 10, Group assignment 3

Concepts

System Under Test (SUT): The software system being tested



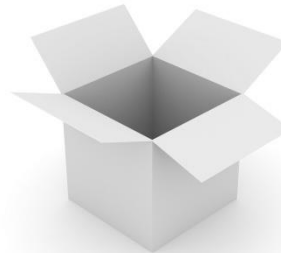
```
// Copyright (c) Microsoft Corporation. All rights reserved.  
// Licensed under the MIT license.  
  
using CalculatorApp.ViewModel.Common;  
using System;  
  
namespace CalculatorApp  
{  
    namespace Converters  
    {  
        /// <summary>  
        /// Value converter that translates true to false and vice versa.  
        /// </summary>  
        [Windows.Foundation.Metadata.WebHostHidden]  
        public sealed class RadixToStringConverter : Windows.UI.Xaml.Data.IValueConverter  
        {  
            public object Convert(object value, Type targetType, object parameter, string language)  
            {  
                // ...  
            }  
        }  
    }  
}
```



Black-box testing: Observing SUT outputs for specified inputs *without looking at the internals*

Coverage: How much of the source code has been tested?

Grey-box testing: Black-box + White-box



White-box testing: Test SUT behavior *using knowledge of its internal structure*
(Also: clear-box, glass-box, or *structural testing*)

Techniques for designing tests

Techniques for designing black-box tests

- Equivalence partitioning
- Boundary value analysis
- Robustness testing
- Decision table testing
- State transition testing
- Use case testing
- Scenario-based testing

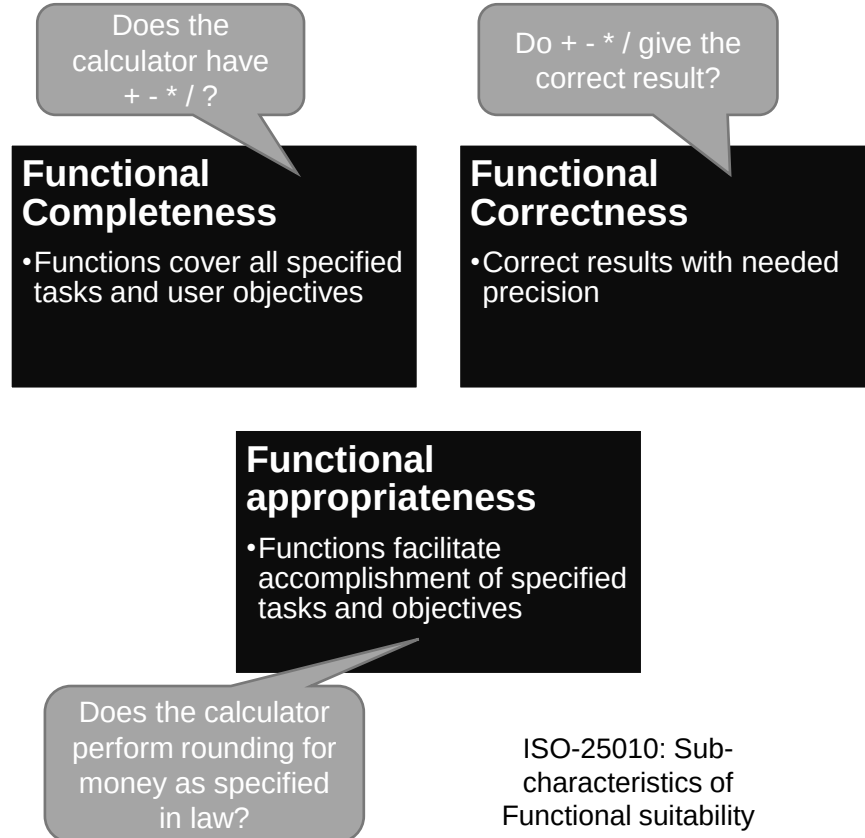
Techniques for designing white-box tests

- Control flow testing
- Statement coverage
- Path coverage
- Decision coverage
- Condition coverage
- Condition/decision coverage
- Modified condition/decision coverage

Functional suitability

- The essential purpose of the software system
- “Degree to which [system] provides functions that meet stated and implied needs when used under specified conditions” (ISO 25010)
 - Each individual function can be verified as existing or not existing
 - Specific properties can be specified for a function

Functionality is “just the beginning” of quality



Manual testing

Manual testing: motivation

- Preventing all faults is practically impossible
- Could all tests be automated?
 - Creating an oracle suitable for automation is often not possible
 - Automation can generally find only the most extreme and most visible faults
 - Automated test code can have faults; then manual intervention is needed
 - Automated tests need maintenance; they too incur costs

→ **Manual testing is needed**

Test automation and manual testing can complement each other.

A manual test may be a good starting point for designing an automated test.



Discussion

Manual testing: a human performs the test (possibly using some tools)

What benefits and challenges
do you identify with manual testing?

How could the challenges be reduced?

Manual testing

- “Human-present testing”
 - A human steers the testing and performs some essential testing actions
 - A human makes the judgement of whether the test passes or not
 - Some tools could be used to help
- Problems can arise with unsystematic manual testing:
 - Ad-hoc testing
 - Manual testing can be too slow
 - Not repeatable
 - Not reproducible
 - Not transferable



Photo: CC BY-SA

Systematic manual
testing

Scripted
testing

Exploratory
testing

Scripted testing

- The tester follows a script to perform tests
- **Rigid scripts**
- Flexible scripts
- Scripted testing can be as rigid or as flexible as necessary

BUT 1: Very rigid scripts could be automated

BUT 2: For very flexible scripts to work, testers need specific instructions on how to handle choices and uncertainty – otherwise it becomes ad-hoc testing!

Exploratory testing can be used instead

Functional area	Test Name	Test Steps	Test Data	Expected Results
Sign-up	Sign-in success	1. Enter a valid username and password 2. Click Login	Username: mary Password: Secret1\$	User is logged in successfully
Sign-up	Sign-in with incorrect password	1. Enter valid username 2. Enter invalid password 3. Click Login	Username: mary Password: wrong0	User is returned to login page and wrong username or password message is shown

Scripted testing

- The tester follows a script to perform tests
- Rigid scripts
- **Flexible scripts**
- Scripted testing can be as rigid or as flexible as necessary

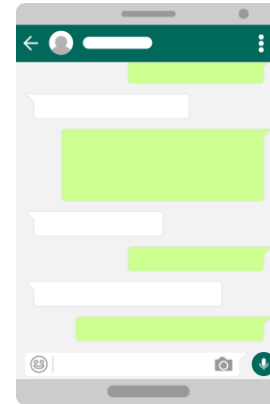
BUT 1: Very rigid scripts could be automated

BUT 2: For very flexible scripts to work, testers need specific instructions on how to handle choices and uncertainty – otherwise it becomes ad-hoc testing!

Exploratory testing can be used instead

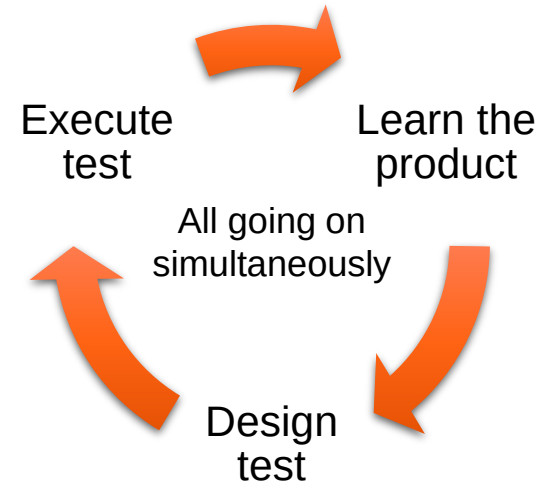
Chat bot test

1. Open the chat bot
2. Interact with the chat bot interface
3. Report findings



Exploratory testing

- No test scripts – but can use guiding plan
- Test documentation (cases, results) recorded as the test proceeds
- Tools can be used (debugger, screen recorder, etc.)
- Focus on *varying things* to explore the SUT
- Systematic guidance increases the chances of finding faults
- Requires skill and experience
- Management challenges
 - Keeping track of testing progress (especially with multiple testers)
 - Summarising the state and results of testing (e.g., for internal and external stakeholders)



Approaches to exploratory testing

- Exploratory testing in the small and in the large
 - Small decisions: the detailed choices the tester makes when exploring
 - Large decisions: choices about managing the overall exploration

Session-based test management (Bach, 2000)

- Strictly time-boxed sessions of free exploration
- Management on the level of sessions
- Sessions are directed and planned

Tour-based exploratory testing (Whittaker, 2009)

- Tourist exploration as a metaphor for testing
- Management on the level of tours
- Tours have a theme or focus to guide tester activities

Examples of "small" choices

User input

State

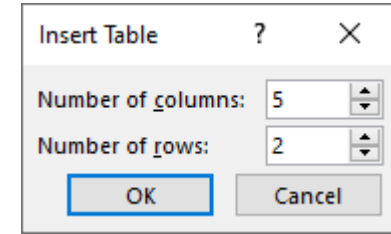
Code paths

User data

Environment

Choices: Input

- Input originates from outside the SUT and causes it to respond
- Legal (permitted) vs. illegal (not permitted) inputs
- Try values from different *equivalence classes*
 - Small positive integers (e.g. 1, 5)
 - Large positive integers (e.g. 40004, 65535)
 - Zero
 - Negative integers
 - Strings
- Filters: dropdowns, integer entry boxes
 - Can they be bypassed?

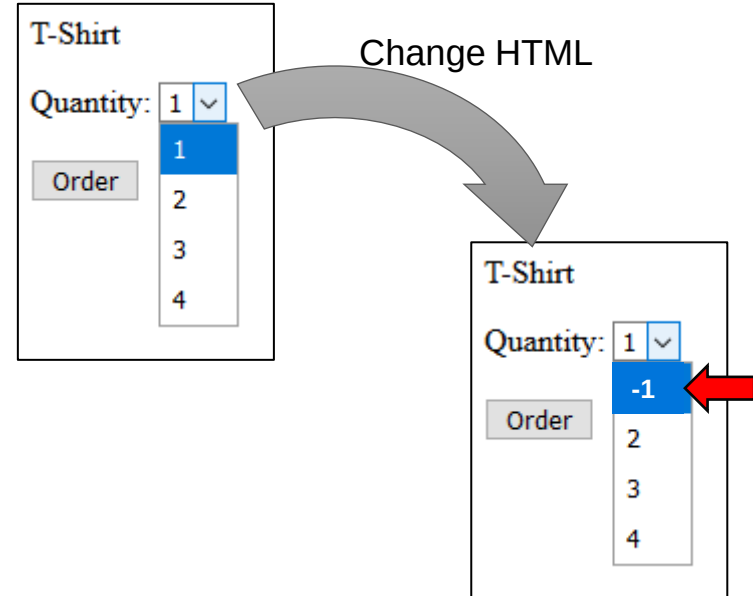


Insert Table ? X

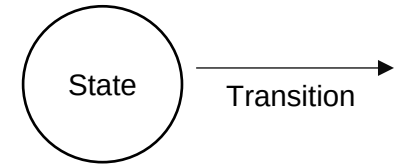
Number of columns: 5

Number of rows: 2

OK Cancel



Choices: State



- The state of the SUT is a coordinate in a state space that contains exactly one value for each internal data structure
- The state space can be extremely large and can only be partially tested
- When possible, start tests from *known states*

Keep track of inputs as they may reappear later, giving indications of internal state

Follow paths through the state space and explore both expected and unexpected paths

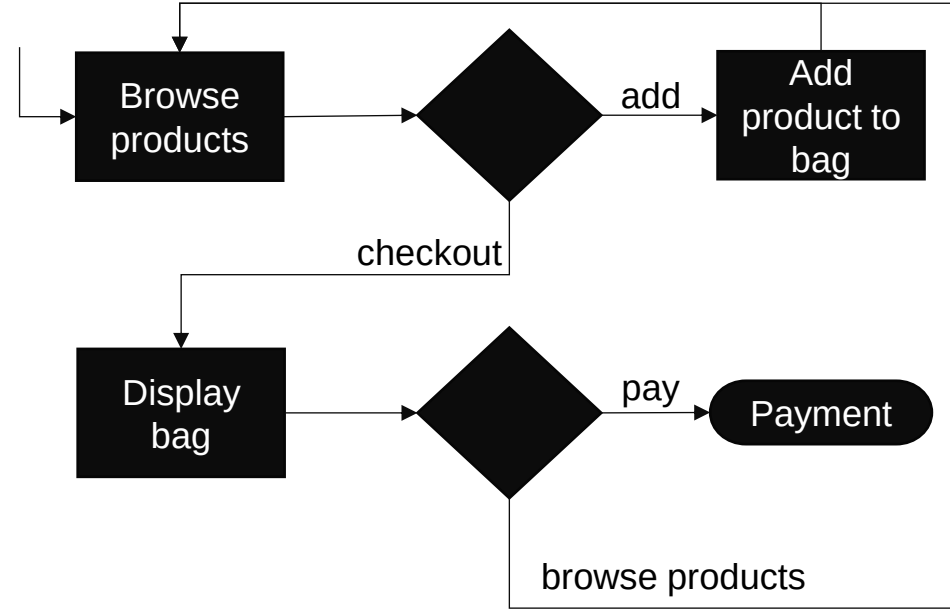
Check for correct data scope: is data *temporary* (persists over one session) or *persistent* (persists between sessions)

Test related inputs together (e.g. if testing a shopping voucher code, also have items in the cart)

Repeat state-changing actions to see if state is accumulative (e.g. a growing data structure)

Choices: Code paths

- Branching structures in the code create different paths that can be explored
- Code paths and state interact
- Without access to source code (or time to analyse it), it is difficult to know which paths exist and have been covered
- To some extent, tools can assist (e.g. web browser inspector)
- The tester must make a judgement based on available information



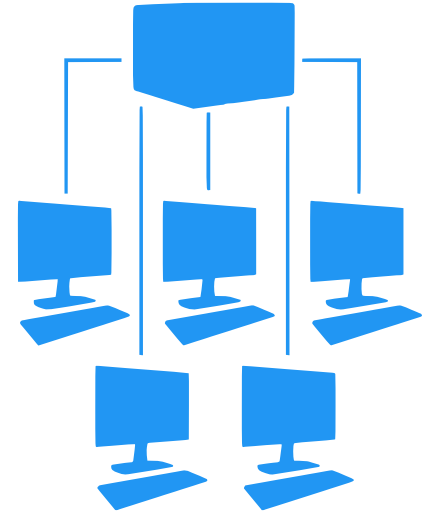
Loops together with branching structures can lead to very complicated code paths

Choices: User data

- Often test data must be obtained or created to make certain tests possible
 - Test data should be as similar as possible to data that users have, but:
 - Real user data evolves over time
 - Real user data often has unexpected relationships and structure
 - Real user data is often large while it would be preferable to test with small, precise data
 - Real user data is often confidential or has other restrictions that prevent its use in tests
- Exploratory testing involves reasoned compromises regarding user data

Choices: Environment

- Differences in operating systems and OS configurations
- Other applications that might interact with the SUT
- Drivers, programs, files, and settings that might interact directly or indirectly with the SUT
- The network connection: its presence or absence, speed, and other properties
- The number of variations is extremely large and can often not be recreated



"Large" choices in exploratory testing

- Large choices concern the management of exploratory testing
- With many tests
- With many testers
- Considering the resources available (e.g. deadlines)
- Considering the quality requirements
- Considering efficiency and effectiveness of the testing process
- Considering needs to monitor and improve the testing process
- Considering the need to report on test results



Discussion

Manual testing: a human performs the test (possibly using some tools)

Analyse mycourses.aalto.fi and report on correctness and consistency issues as well as other observations

Perspective: a person who would like to apply to study at Aalto and is interested in software testing

Session-based test management

Session-based test management (Bach, 2000)

- Strictly time-boxed sessions of free exploration
- Management on the level of sessions
- Sessions are directed and planned

- Sessions are
 - Chartered – they are associated with a mission regarding what to test or what problems are in focus
 - Uninterrupted – no significant interruptions
 - Typically between 45 minutes and 2 hours (this may vary and there is no strict rule)
- What happens during a session depends on the tester and the session charter

Session-based test management

Session-based test management (Bach, 2000)

- Strictly time-boxed sessions of free exploration
- Management on the level of sessions
- Sessions are directed and planned

- Each session is debriefed
 - The session report is walked through
 - There can be a formal acceptance of the session report
 - Provide feedback and coaching to the tester
 - A debriefing can cover more than one session – they can be short
- The debriefing allows the test lead to
 - Understand how much can be done in one session
 - Understand how many sessions can be done per time
 - Provide a basis for effort predictions even though the testing is not planned in detail

Session-based test management

Session-based test management (Bach, 2000)

- Strictly time-boxed sessions of free exploration
- Management on the level of sessions
- Sessions are directed and planned

- Session sheets
 - Function both as direction and reports
 - Reports are written in a standard format that is machine readable (as well as human-readable)
 - All reports are stored in a repository
 - The reports can be checked
 - E.g. if the report lists a data file, has the file been stored in the repository?
 - Metrics can be extracted from the reports
 - Are there bottlenecks preventing testing?
 - How many tests have we performed over the last 4 weeks?
 - How many tests can we expect to do next week?
 - (There is no one “standard” format)

Example session sheet

CHARTER

Analyze MapMaker's View menu functionality and report on areas of potential risk.

#AREAS

OS | Windows 2000

Menu | View

Strategy | Function Testing

Strategy | Functional Analysis

START

5/30/00 03:20 pm

TESTER

Jonathan Bach

Example session sheet

TASK BREAKDOWN

Book-keeping to this level
is not always necessary –
but can be done if needed.

#DURATION

short

#TEST DESIGN AND EXECUTION

65

#BUG INVESTIGATION AND REPORTING

25

#SESSION SETUP

20

#CHARTER VS. OPPORTUNITY

100/0

DATA FILES

#N/A

Keeping track of how
much of the session
followed the charter
and how much was
opportunistic.

Example session sheet

TEST NOTES

I touched each of the menu items, below, but focused mostly on zooming behavior with various combinations of map elements displayed.

View: Welcome Screen

Navigator

Locator Map

Legend

Map Elements

Highway Levels

Street Levels

Airport Diagrams

Zoom In

Zoom Out

Zoom Level

(Levels 1-14)

Previous View

Risks:

- Incorrect display of a map element.
- Incorrect display due to interrupted repaint.
- CD may be unreadable.
- Old version of CD may used.
- Some function of the product may not work at a certain zoom level.

Example session sheet

BUGS

#BUG 1321

Zooming in makes you put in the CD 2 when you get to a certain level of granularity (the street names level) -- even if CD 2 is already in the drive.

#BUG 1331

Zooming in quickly results in street names not being rendered.

#BUG <not_entered>

instability with slow CD speed or low video RAM. Still investigating.

ISSUES

#ISSUE 1

How do I know what details should show up at what zoom levels?

#ISSUE 2

I'm not sure how the locator map is supposed to work. How is the user supposed to interact with it?



Tour-based exploratory testing

Tour-based exploratory testing (Whittaker, 2009)

- Tourist exploration as a metaphor for testing
- Management on the level of tours
- Tours have a theme or focus to guide tester activities

- “Tour” is a metaphor that helps form an overall strategy and set specific goals
 - Aimless exploration: you might stumble upon something interesting
 - Specific goals and purpose: you will visit certain important landmarks
 - How much of each depends on the “tourist” – the character or persona whose *intent* guides the exploration
- Decomposition according to intent rather than the structure of the software
- Software functionality and features are visited more flexibly and in orders that differ from strict functional testing

Tour-based exploratory testing

Tour-based exploratory testing (Whittaker, 2009)

- Tourist exploration as a metaphor for testing
- Management on the level of tours
- Tours have a theme or focus to guide tester activities

- Districts
 - Business: startup/shutdown code, main features
 - Historical: legacy code
 - Tourist: features and functions that attract novice users
 - Entertainment: supportive features; complements the tour
 - Hotel: what the software does “at rest”
 - Seedy: areas of the software that are “off limits” to users

Tour-based exploratory testing

Tour-based exploratory testing (Whittaker, 2009)

- Tourist exploration as a metaphor for testing
- Management on the level of tours
- Tours have a theme or focus to guide tester activities

Examples of business district tours

- The Guidebook Tour
 - Read the manual and follow it to the letter
 - Variation: the Blogger's Tour – follow third-party advice
 - Variation: the Pundit's Tour – create test cases based on negative reviews
- The Intellectual Tour
 - Make the software work as hard as possible – try to break it
 - Look for inputs that cause problems (e.g. too large inputs, inputs that fool error checking)
 - But don't go overboard

Tour-based exploratory testing

Tour-based exploratory testing (Whittaker, 2009)

- Tourist exploration as a metaphor for testing
- Management on the level of tours
- Tours have a theme or focus to guide tester activities

Examples of hotel district tours

- The Rained-Out Tour
 - Start operations, then stop them
 - E.g., start a search, then cancel or start a print job, then cancel
 - Try different ways of cancelling – if there is no cancel button, do something else
 - Start, then restart without waiting for the operation to complete
- The Couch Potato Tour
 - Do the least work possible
 - Accept all defaults, leave fields blank, proceed to the next step without providing input, etc.
 - A strategy to exercise the default code paths

What does exploratory testing test?

- System-level and (in an exploratory fashion) acceptance test -level tests
- Can be inspired by (failed) automated tests – or can discover areas for further automated testing
- Exploratory testing is usually applied through the user interface of an application
 - Exploratory testing can find usability problems, but it is not the same as usability testing
 - Can focus on different quality characteristics and different stakeholders' perspectives
 - It can find important bugs quickly with little preparation
- There are also reports of exploratory testing used through APIs
 - Interactively for interpreted languages (e.g., Python)
 - Through exploratory scripts or small programs – works also for compiled languages
 - The purpose can still be to fulfil users' quality requirements – the “window” into the software is just different

Scientific evidence on exploratory testing

- Similar fault detection effectiveness as scripted testing (total number of defects found)
- Higher efficiency than scripted testing (defects found per time)
- Use of domain knowledge, the SUT, and customers, are important factors explaining the effectiveness
- Ability to provide rapid feedback reported as a benefit
- Limited possibilities to manage test coverage reported as a drawback

Itkonen, J.; Mäntylä, M. V. (2013). Are test cases needed? Replicated comparison between exploratory and test-case-based software testing. *Empirical Software Engineering*. 19 (2): 303–342.

Itkonen, J.; Mäntylä, M. V.; Lassenius, C. (2013). The Role of the Tester's Knowledge in Exploratory Software Testing. *IEEE Transactions on Software Engineering*. 39 (5): 707–724.

Itkonen, J.; Rautiainen, K. (2005). Exploratory testing: a multiple case study. 2005 International Symposium on Empirical Software Engineering, 2005.

Next steps

- Individual assignments in MyCourses
 - Quiz on path testing
 - Quiz on decision table testing
 - Unit testing 3
 - Related reading: Wang et al. (2024). Software Testing with Large Language Models: Survey, Landscape and Vision (see Materials)
 - Acceptance testing 2.1

Use the related reading
for more details.
Reserve enough time to
set up your environment.

Deadline: before next lecture (Tuesday by 10:00)

- Getting help
 - Ask in the course chat, see Communication section in MyCourses
 - Personal questions by email



Study smarter, not harder!

- Plan: when and where
- Go deep at your own pace
- Get some rest
- Practice makes perfect
- Enjoy it 😊

Software Testing and Quality Assurance

Lecture 6

Fabian Fagerholm

fabian.fagerholm@aalto.fi



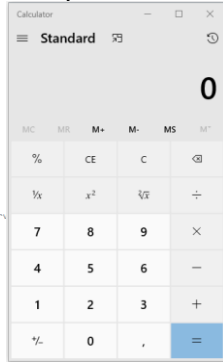
Aalto University
School of Science



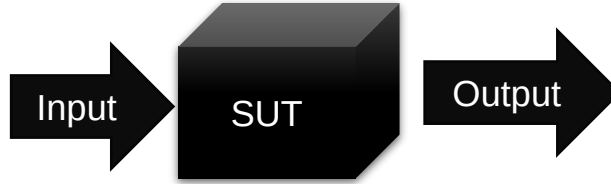
Week	Lecture	Topic	Assignment deadlines (Tuesdays 10:00 unless otherwise specified)
36	3.9.2024	Introduction and practicalities	
37	10.9.2024	Software quality	Individual assignments 1
38	17.9.2024	Software testing: levels, test case analysis and design	Group registration (DL: 20.9.)
39	24.9.2024	Testing techniques: Black-box, white-box testing	Individual assignments 2, Group assignment 1
40	1.10.2024	Testing techniques: Manual testing	Individual assignments 3
41	8.10.2024	Testing techniques: Black-box, white-box, manual testing	Individual assignments 4
42	15.10.2024	(No lecture)	
43	22.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 5
44	29.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 6, Group assignment 2
45	5.11.2024	Guest lecture	Individual assignments 7
46	12.11.2024	Continuous Integration and Continuous Delivery/Deployment, Test management, and Course summary	Individual assignments 8
47	19.11.2024	(No lecture)	Individual assignments 9
48	26.11.2024	(No lecture)	Individual assignments 10, Group assignment 3

Concepts

System Under Test (SUT): The software system being tested



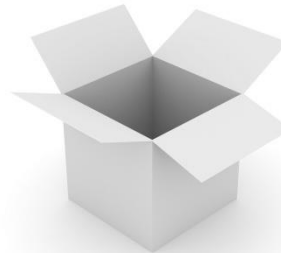
```
// Copyright (c) Microsoft Corporation. All rights reserved.  
// Licensed under the MIT license.  
  
using CalculatorApp.ViewModel.Common;  
using System;  
  
namespace CalculatorApp  
{  
    namespace Converters  
    {  
        /// <summary>  
        /// Value converter that translates true to false and vice versa.  
        /// </summary>  
        [Windows.Foundation.Metadata.WebHostHidden]  
        public sealed class RadioToStringConverter : Windows.UI.Xaml.Data.IValueConverter  
        {  
            public object Convert(object value, Type targetType, object parameter, string language)  
            {  
                // ...  
            }  
        }  
    }  
}
```



Black-box testing: Observing SUT outputs for specified inputs *without looking at the internals*

Coverage: How much of the source code has been tested?

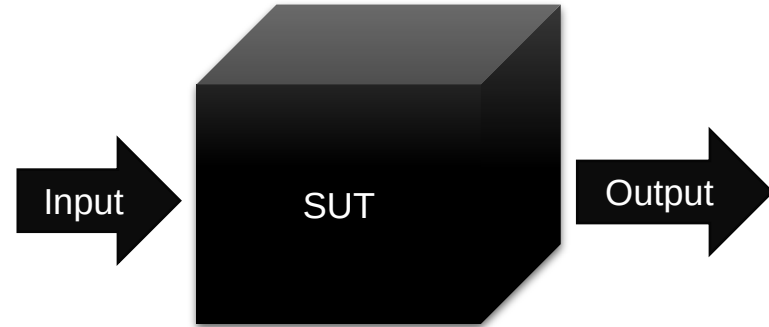
Grey-box testing: Black-box + White-box



White-box testing: Test SUT behavior *using knowledge of its internal structure*
(Also: clear-box, glass-box, or *structural testing*)

Black-box testing

- Testing that does not examine the internals of the SUT
 - Inputs
 - Outputs
- Also called *functional testing*, *behavioural testing*, or *specification-based testing*
- Can be applied on all levels of testing (unit, integration, system, acceptance)
- Automated tests can be black-box tests
- Exploratory testing is a kind of black-box testing
- Advantage: can be efficient and effective
- Disadvantage: can never be sure of how much of the system has been exercised
- **Emphasis on finding a limited set of test cases (inputs) that test the system as thoroughly as possible**



Some very specific inputs could lead to a rare execution path that cannot be known without seeing the code

```
if (student.number == 1234567) {  
    student.grade = 5;  
}
```

Techniques for designing black-box tests

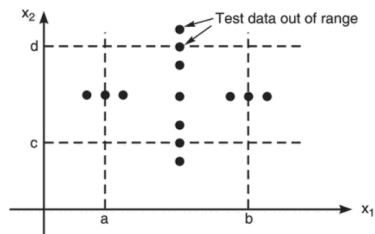
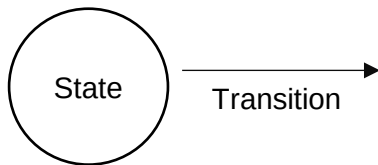


FIGURE 4.2 Robustness Test Cases.
Chopra (2018)

Boundary value analysis
Robustness testing

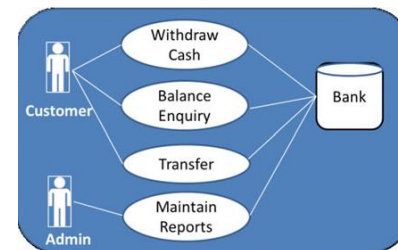


State transition testing

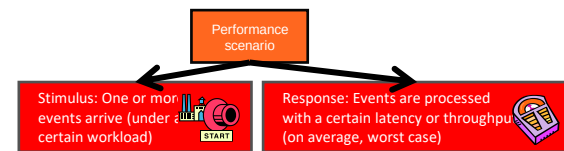
	Rule 1	Rule 2	Rule 3	Rule 4
<i>Input conditions</i>				
Customer with loyalty card?	Yes	Yes	No	No
Spend > 200?	Yes	No	Yes	No
<i>Actions</i>				
Discount	10%	5%	2%	0%
Accumulate loyalty points?	Yes	Yes	No	No

Test case 1 Test case 2 Test case 3 Test case 4

Decision table testing



Use case based testing



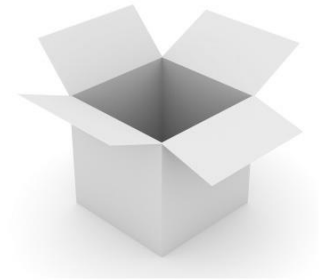
Latency: When the researcher presses Start, the simulation world with 100 creatures is initialized and animation begins within an average response time of one second.

Throughput: With 100 interacting creatures, the system can process, store and visualize at least 100 simulation steps per minute.

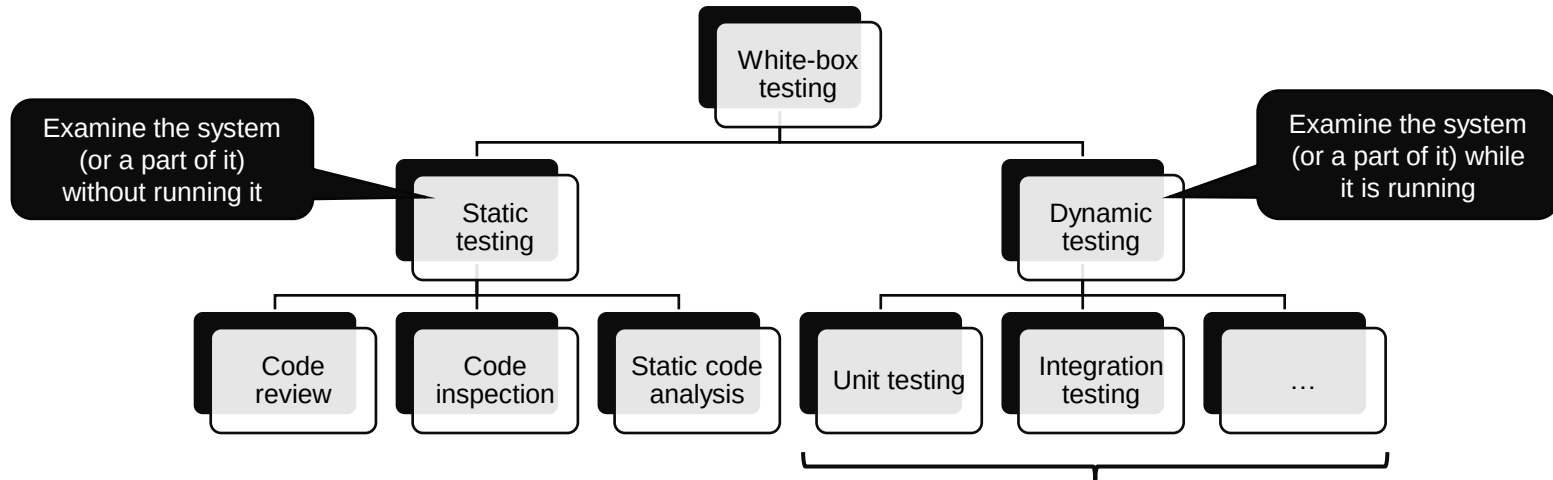
Scenario based testing

White-box testing

- Also: clear-box, glass-box, or *structural testing*
- Designing tests for the SUT *using knowledge of its internal structure*
- *Today: techniques for designing test cases based on a white-box approach*



[Photo CC BY-SA-NC](#)



Potentially all levels of the testing pyramid – just like in black-box testing but based on knowledge of the internal structure

Dynamic white-box testing

- Possible applications on different testing levels
 - Unit: Test paths within individual modules before integration
 - Integration: Test paths between modules (also: data flow)
 - System: End-to-end application flow from user perspective
- Challenges
 - Cannot test all possible paths
 - Test cases may not detect data sensitivity errors (e.g., $p=q/r$ may execute correctly except when $r=0$)
 - Assumes control flow is correct (non-existent paths cannot be discovered)
 - The tester must have programming skills to evaluate the SUT

Dynamic white-box testing emphasises paths – test case design is based on path analysis

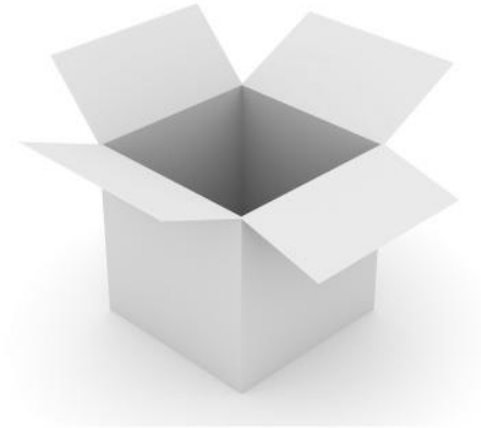
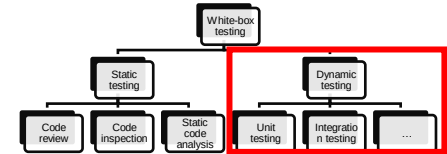


Photo CC BY-SA-NC



Manual testing

- “Human-present testing”
 - A human steers the testing and performs some essential testing actions
 - A human makes the judgement of whether the test passes or not
 - Some tools could be used to help
- Problems can arise with unsystematic manual testing:
 - Ad-hoc testing
 - Manual testing can be too slow
 - Not repeatable
 - Not reproducible
 - Not transferable



Photo: CC BY-SA

Systematic manual
testing

Scripted
testing

Exploratory
testing

Scripted testing

- The tester follows a script to perform tests
- **Rigid scripts**
- Flexible scripts
- Scripted testing can be as rigid or as flexible as necessary

BUT 1: Very rigid scripts could be automated

BUT 2: For very flexible scripts to work, testers need specific instructions on how to handle choices and uncertainty – otherwise it becomes ad-hoc testing!

Exploratory testing can be used instead

Functional area	Test Name	Test Steps	Test Data	Expected Results
Sign-up	Sign-in success	1. Enter a valid username and password 2. Click Login	Username: mary Password: Secret1\$	User is logged in successfully
Sign-up	Sign-in with incorrect password	1. Enter valid username 2. Enter invalid password 3. Click Login	Username: mary Password: wrong0	User is returned to login page and wrong username or password message is shown

Scripted testing

- The tester follows a script to perform tests
- Rigid scripts
- **Flexible scripts**
- Scripted testing can be as rigid or as flexible as necessary

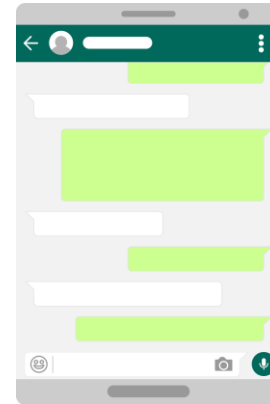
BUT 1: Very rigid scripts could be automated

BUT 2: For very flexible scripts to work, testers need specific instructions on how to handle choices and uncertainty – otherwise it becomes ad-hoc testing!

Exploratory testing can be used instead

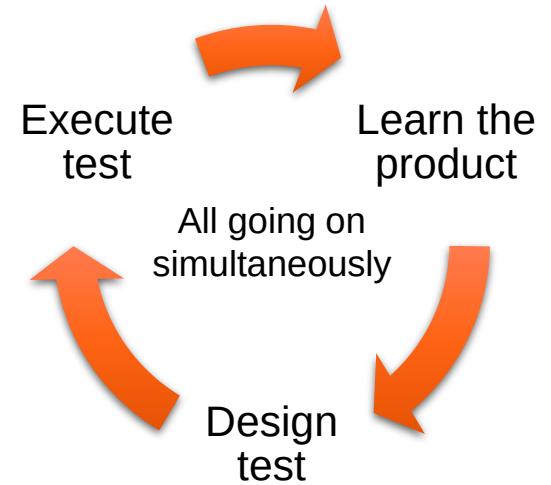
Chat bot test

1. Open the chat bot
2. Interact with the chat bot interface
3. Report findings



Exploratory testing

- No test scripts – but can use guiding plan
- Test documentation (cases, results) recorded as the test proceeds
- Tools can be used (debugger, screen recorder, etc.)
- Focus on *varying things* to explore the SUT
- Systematic guidance increases the chances of finding faults
- Requires skill and experience
- Management challenges
 - Keeping track of testing progress (especially with multiple testers)
 - Summarising the state and results of testing (e.g., for internal and external stakeholders)



Approaches to exploratory testing

- Exploratory testing in the small and in the large
 - Small decisions: the detailed choices the tester makes when exploring
 - Large decisions: choices about managing the overall exploration

Session-based test management (Bach, 2000)

- Strictly time-boxed sessions of free exploration
- Management on the level of sessions
- Sessions are directed and planned

Tour-based exploratory testing (Whittaker, 2009)

- Tourist exploration as a metaphor for testing
- Management on the level of tours
- Tours have a theme or focus to guide tester activities

Examples of "small" choices

User input

State

Code paths

User data

Environment

Discussion

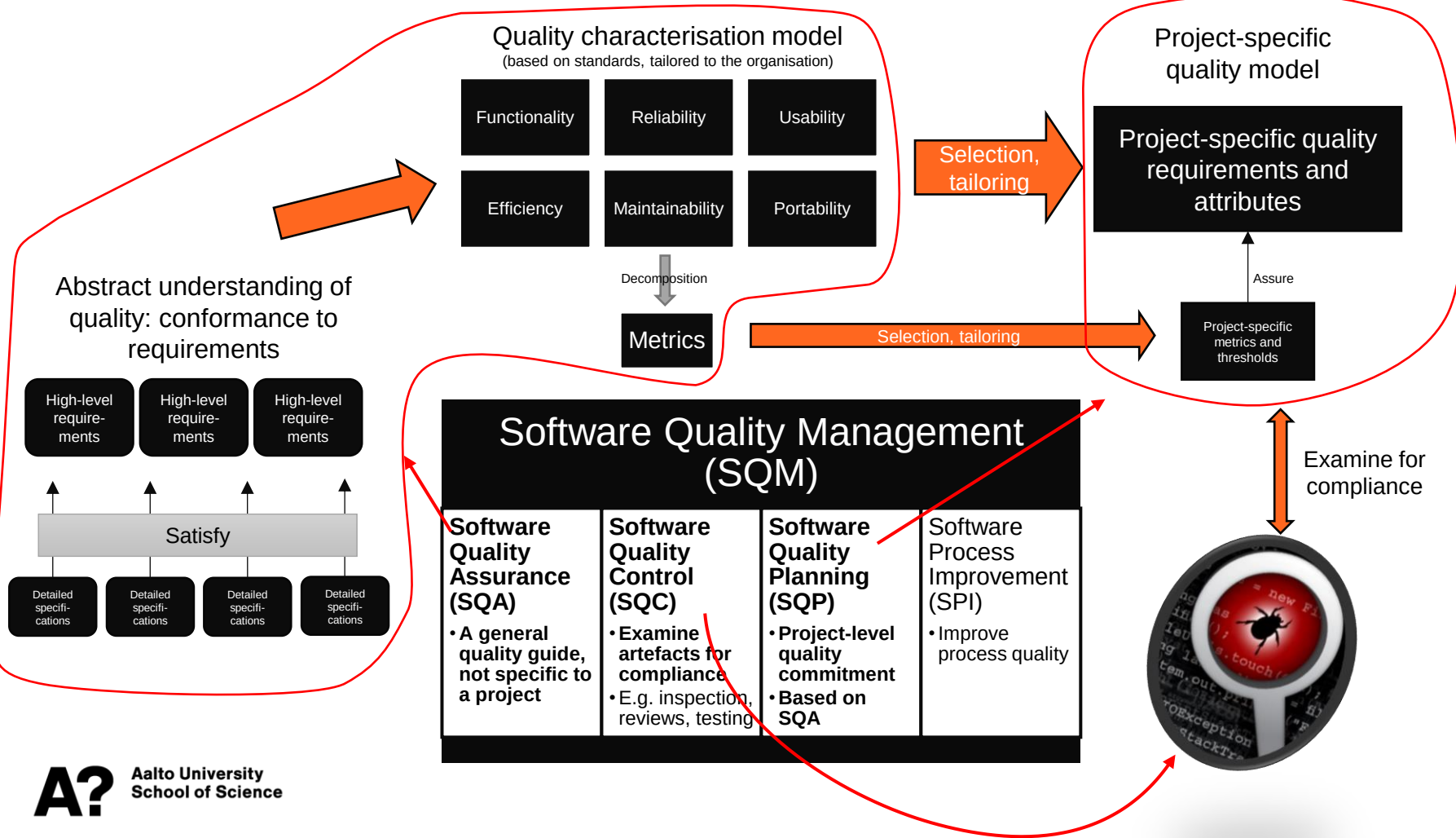
Where should software testing start?

(How do we know where to start looking for defects?)

Where to start testing?

Start from
requirements

Start from a
model of
typical defects



Defect taxonomies

A classification of typical types of defects

- Generate ideas for test design
- Audit test plans to determine coverage
- Understand defects, their types, and severities
- Understand the process producing the defects
- Improve the development process
- Improve the testing process
- Train new testers about areas that need testing
- Explain the complexities of software testing to management

Project-level defect taxonomy example

Class	Element	Attribute
Product engineering	Requirements	Stability Completeness Clarity Validity Feasibility Precedent Scale Functionality
	Design	Functionality Difficulty Interfaces Performance Testability
	Code and Unit Test	Feasibility Testing Coding/implementation
	Integration and Test	Environment Product System
...	...	...

Concern: Incomplete requirements
→
Emphasis: Exploratory testing

Concern: Imprecise requirements
→
Emphasis: Decision tables, state transition diagrams

Concern: System performance
→
Emphasis: Performance testing

Beizer's taxonomy

- 4 levels (top two levels shown →)
- Detailed list of precise defect types
- Points the tester towards different kinds of defects to check for

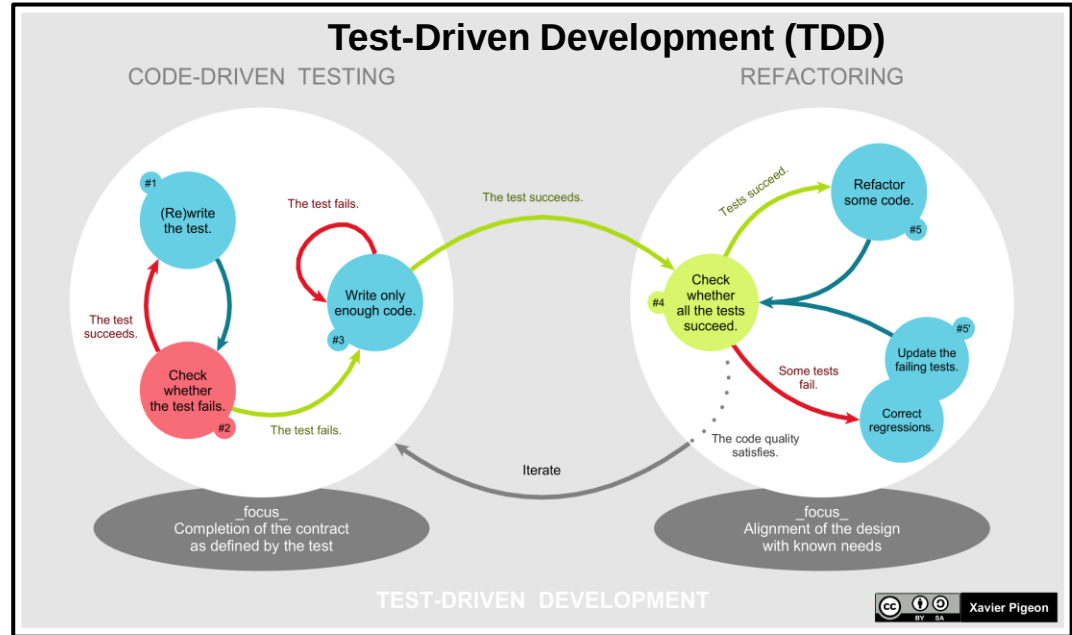
1xxxx	Requirements
11xx	Requirements incorrect
12xx	Requirements logic
13xx	Requirements, completeness
14xx	Verifiability
15xx	Presentation, documentation
16xx	Requirements changes
2xxx	Features and Functionality
21xx	Feature/function correctness
22xx	Feature completeness
...	...

Using a defect taxonomy



Test-driven development (TDD)

- TDD is an approach where tests are written before the code is implemented
- Tries to avoid biasing the test design
 - Testers and developers tend to have *confirmation bias* – looking only for evidence that confirms assumptions (or wishes)
 - Writing the test first can promote a more critical mindset
- Strongly connected to *refactoring* to maintain code quality



Discussion

Pick 2-3 testing techniques from the course

Can you explain them to your neighbour / group members?

Which aspects are clear / unclear to you?

Try to solve the unclear parts together

Next steps

- Individual assignments in MyCourses
 - Exploratory testing

Deadline: before next lecture (Tuesday by 10:00)

- Note: next week is exam week – no lecture!
- Getting help
 - Ask in the course chat, see Communication section in MyCourses
 - Personal questions by email

Use the related reading
for more details.
Reserve enough time to
set up your environment.



Study smarter, not harder!

- Plan: when and where
- Go deep at your own pace
- Get some rest
- Practice makes perfect
- Enjoy it ☺

Static Code Analysis

Lecture 7

Stanislav Chren

stanislav.chren@aalto.fi

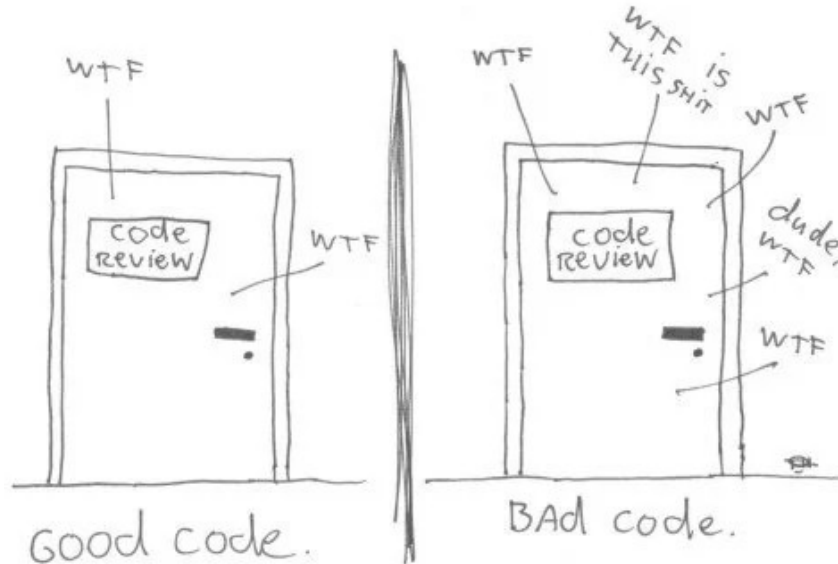


Aalto University
School of Science



Code Quality

The ONLY valid measurement
of code quality: WTFs/minute



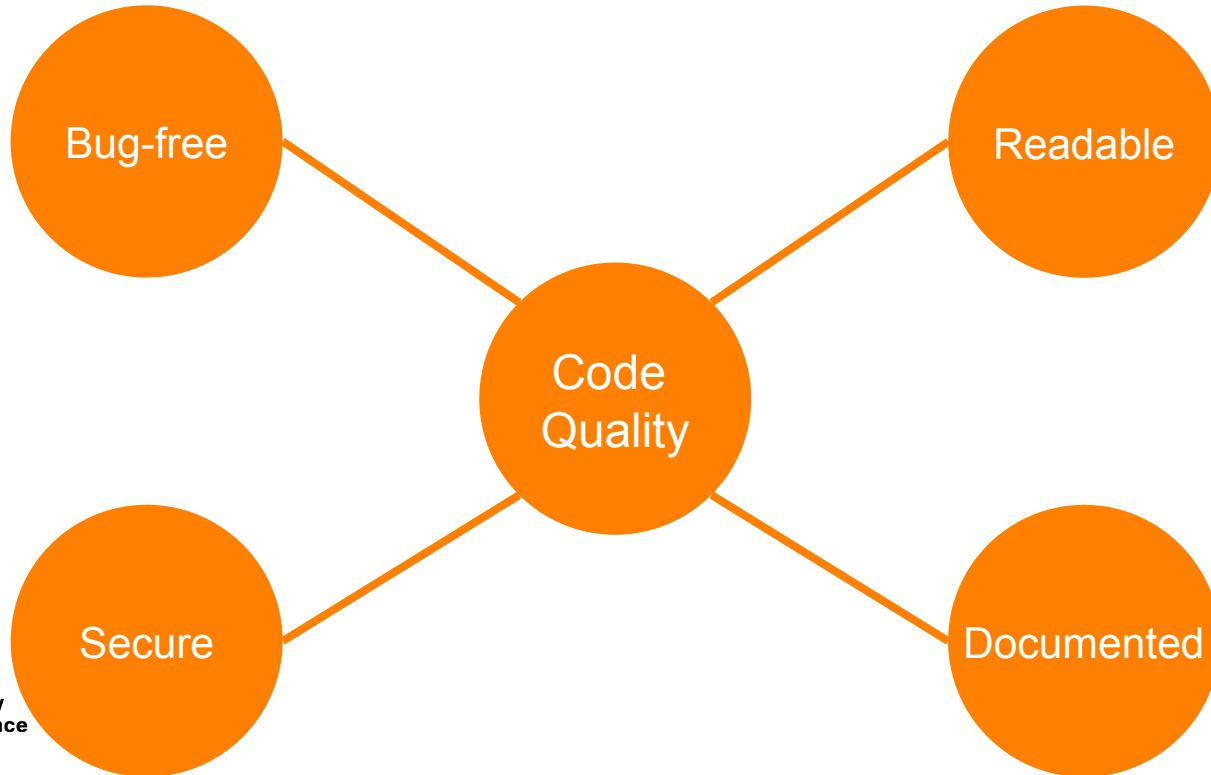
(c) 2008 Focus Shift

Code Quality

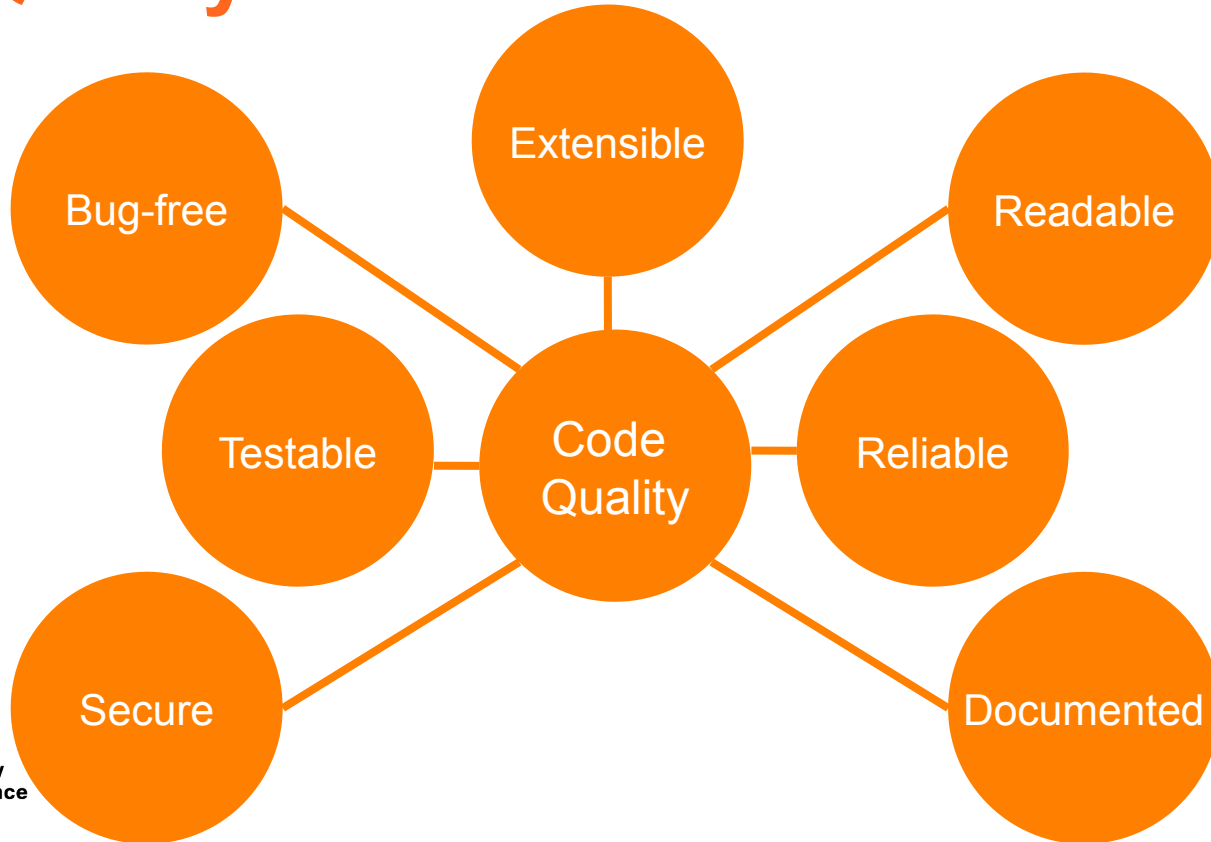


Code
Quality

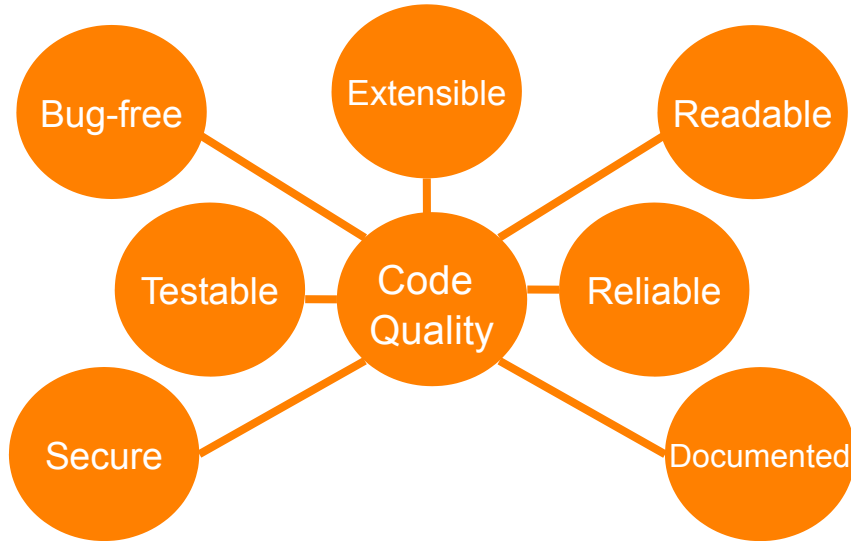
Code Quality



Code Quality



Code Quality



- No established code quality standards
- Guideliness or recommendeations
 - ~ SOLID
 - ~ Clean Code
 - ~ DRY
 - ~ KISS
- Evaluated manually (code reivew) or automatically (SCA, ...)

Code Quality - SOLID

- **S**ingle responsibility principle
- **O**pen-closed principle
- **L**iskov substitution principle
- **I**nterface segregation principle
- **D**ependency inversion principle

Code Quality - SOLID

- Single responsibility principle

```
public class Invoice {  
    ...  
    public Invoice(Book book, int quantity, double discountRate, double taxRate) {  
        ....  
    }  
    public double calculateTotal() {  
        return (book.price - book.price * discountRate) * this.quantity;  
    }  
    public void printInvoice() {  
        System.out.println(quantity + "x " + book.name + " " + book.price + "$");  
        System.out.println("Discount Rate: " + discountRate);  
        ....  
    }  
    public void saveToFile(String filename) {  
        // Creates a file with given name and writes the invoice  
    }  
}
```

Code Quality - SOLID

- Single responsibility principle

```
public class Invoice {  
    ...  
    public Invoice(Book book, int quantity, double discountRate, double taxRate) {  
        ....  
    }  
    public double calculateTotal() {  
        return (book.price - book.price * discountRate) * this.quantity;  
    }  
    public void printInvoice() {  
        System.out.println(quantity + "x " + book.name + " " + book.price + "$");  
        System.out.println("Discount Rate: " + discountRate);  
        ....  
    }  
    public void saveToFile(String filename) {  
        // Creates a file with given name and writes the invoice  
    }  
}
```

Code Quality - SOLID

- Single responsibility principle
- Open-closed principle

```
public class InvoicePersistence {  
    Invoice invoice;  
    public InvoicePersistence(Invoice invoice) {  
        this.invoice = invoice;  
    }  
    public void saveToFile(String filename) {  
        // Creates a file with given name and writes the invoice  
    }  
  
}
```

Code Quality - SOLID

- Single responsibility principle
- Open-closed principle

```
public class InvoicePersistence {  
    Invoice invoice;  
    public InvoicePersistence(Invoice invoice) {  
        this.invoice = invoice;  
    }  
    public void saveToFile(String filename) {  
        // Creates a file with given name and writes the invoice  
    }  
    public void saveToDatabase() {  
        // Saves the invoice to database  
    }  
}
```

Code Quality - SOLID

- Single responsibility principle
- Open-closed principle

```
public class InvoicePersistence {  
    Invoice invoice;  
    public InvoicePersistence(Invoice invoice) {  
        this.invoice = invoice;  
    }  
    public void saveToFile(String filename) {  
        // Creates a file with given name and  
    }  
    public void saveToDatabase() {  
        // Saves the invoice to database  
    }  
}
```



```
interface InvoicePersistence {  
    public void save(Invoice invoice);  
}  
  
public class DatabasePersistence implements InvoicePersistence {  
    @Override  
    public void save(Invoice invoice) {  
        // Save to DB  
    }  
}  
  
public class FilePersistence implements InvoicePersistence {  
    @Override  
    public void save(Invoice invoice) {  
        // Save to file  
    }  
}
```


Code Quality - SOLID

- Single responsibility principle
- Open-closed principle
- Liskov substitution principle

```
public class Bird{
    public void fly();
    public void walk();
    ...
}
public class Parrot extends Bird {
    public void fly(){ ...}
    public void walk(){ ... }
}
public class Penguin extends Bird {
    public void fly(){ ... }
    public void walk(){ ... }
```

A?

Code Quality - SOLID

- Single responsibility principle
- Open-closed principle
- Liskov substitution principle

```
public class Bird{  
    public void fly();  
    public void walk();  
    ...  
}  
  
public class Parrot extends Bird {  
    public void fly(){ ...}  
    public void walk(){ ... }  
}  
  
public class Penguin extends Bird {  
    public void fly(){ ... }  
    public void walk(){ ... }  
}
```



```
public class Bird{  
    public void walk();  
    ...  
}  
  
public class Parrot extends Bird {  
    public void fly(){ ...}  
    public void walk(){ ... }  
}  
  
public class Penguin extends Bird {  
    public void walk(){ ... }  
}
```

A?

Code Quality - SOLID

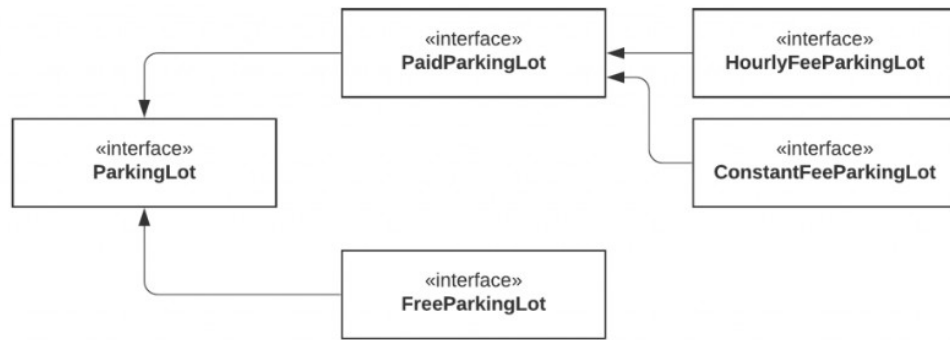
- Single responsibility principle
- Open-closed principle
- Liskov substitution principle
- Interface segregation principle

```
public interface ParkingLot {  
    void parkCar();  
    void unparkCar();  
    void getCapacity();  
    double calculateFee(Car car);  
    void doPayment(Car car);  
}
```

Code Quality - SOLID

- Single responsibility principle
- Open-closed principle
- Liskov substitution principle
- Interface segregation principle

```
public interface ParkingLot {  
    void parkCar();  
    void unparkCar();  
    void getCapacity();  
    double calculateFee(Car car);  
    void doPayment(Car car);  
}
```



Code Quality - SOLID

- Single responsibility principle
- Open-closed principle
- Liskov substitution principle
- Interface segregation principle
- Dependency inversion principle

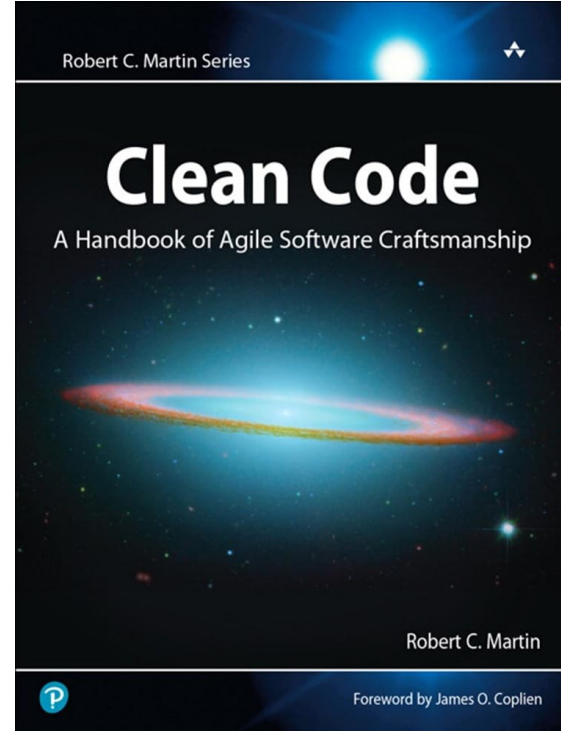
```
class MySQLDatabase {
    public void get() {...}
    public void insert() {...}
    public void update() {...}
    public void delete() {...}
}

class BudgetReport {
    private MySQLDatabase database;
    public BudgetReport(MySQLDatabase database) {
        this.database = database;
    }
    public void open() {
        database.get();
    }
    public void save() {
        database.insert();
    }
}

// Client code
public class Main {
    public static void main(String[] args) {
        MySQLDatabase database = new MySQLDatabase();
        BudgetReport report = new BudgetReport(database);
        report.open();
    }
}
```

Code Quality – Clean Code

- Collection of **recommendations**
 - Naming
 - Code structure
 - Error handling
 - Documentation
 - Unit testing
 -



Code Quality – Clean Code

```
public int[] generatePrimes(int maxValue) {
    if (maxValue >= 2) // the only valid case {
        // declarations
        int s = maxValue + 1; // size of array
        boolean[] f = new boolean[s];
        int i;
        // initialize array to true.
        for (i = 0; i < s; i++)
            f[i] = true;
        // get rid of known non-primes
        f[0] = f[1] = false;
        // sieve
        int j;
        for (i = 2; i < Math.sqrt(s) + 1; i++) {
            if (f[i]) { // if i is uncrossed, cross its multiples.
                for (j = 2 * i; j < s; j += i)
                    f[j] = false; // multiple is not prime
            }
        }
        // how many primes are there?
        int count = 0;
        for (i = 0; i < s; i++) {
            if (f[i])
                count++; // bump count.
        }
        int[] primes = new int[count];
        // move the primes into the result
        for (i = 0, j = 0; i < s; i++) {
            if (f[i]) // if prime
                primes[j++] = i;
        }
        return primes; // return the primes
    }
    else // maxValue < 2
        return new int[0]; // return null array if bad input.
}
```



```
public int[] generatePrimes(int maxValue) {
    if (maxValue < 2)
        return new int[0];
    else {
        generateUncrossedIntegersUpTo(maxValue);
        crossOutMultiples();
        putUncrossedIntegersIntoResult();
        return result;
    }
}

private void generateUncrossedIntegersUpTo(int maxValue) {
    crossedOut = new boolean[maxValue + 1];
    for (int i = 2; i < crossedOut.length; i++)
        crossedOut[i] = false;
}

private void crossOutMultiples() {
    int limit = determineIterationLimit();
    for (int i = 2; i <= limit; i++)
        if (notCrossed(i))
            crossOutMultiplesOf(i);
}

private int determineIterationLimit() {
    double iterationLimit = Math.sqrt(crossedOut.length);
    return (int) iterationLimit;
}

private void crossOutMultiplesOf(int i) {
    for (int multiple = 2 * i;
         multiple < crossedOut.length;
         multiple += i)
        crossedOut[multiple] = true;
}

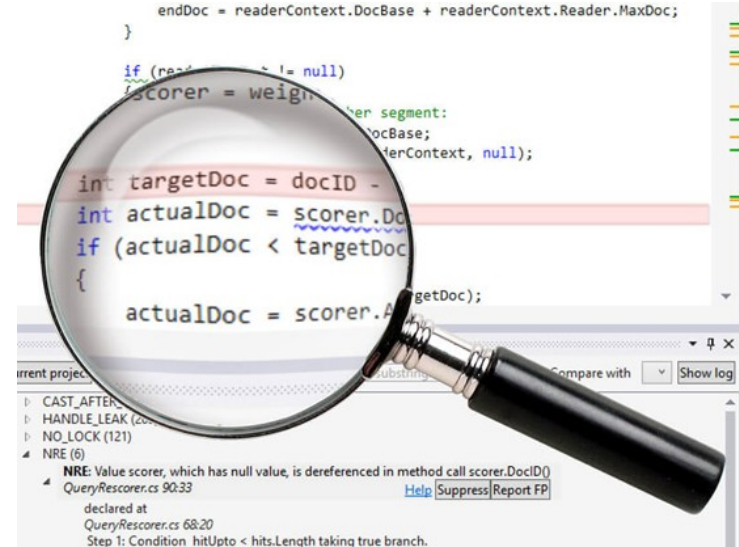
private boolean notCrossed(int i) {
    return crossedOut[i] == false;
}

private void putUncrossedIntegersIntoResult() {
    result = new int[numberOfUncrossedIntegers()];
    for (int j = 0, i = 2; i < crossedOut.length; i++)
        if (notCrossed(i))
            result[j++] = i;
}

private int numberOfUncrossedIntegers() {
    int count = 0;
    for (int i = 2; i < crossedOut.length; i++)
        if (notCrossed(i))
            count++;
    return count;
}
```

What is Static Code Analysis?

- **Analysis of computer software that is performed without actually executing programs.**
 - ~ Performed on a version of the source code, or some form of the object code.
 - ~ The term is usually applied to the analysis performed by an automated tool.



Source: <https://was-hillt.info/references/static-vs-dynamic-source-code-analysis.php>

Discussion

- What are the possible applications of SCA?
- What are the benefits/drawbacks of SCA?



Applications of SCA

Type checking

- Checks correct assignment of object types

```
def headline(text: str, align: bool = True):  
    if align:  
        return f"{text.title()}\n{'-' * len(text)}"  
    else:  
        return f" {text.title()} ".center(50, "o")  
  
print(headline("python type checking"))  
print(headline("use pycharm", "center"))
```

Expected type 'bool', got 'str' instead [more...](#) (Ctrl+F1)

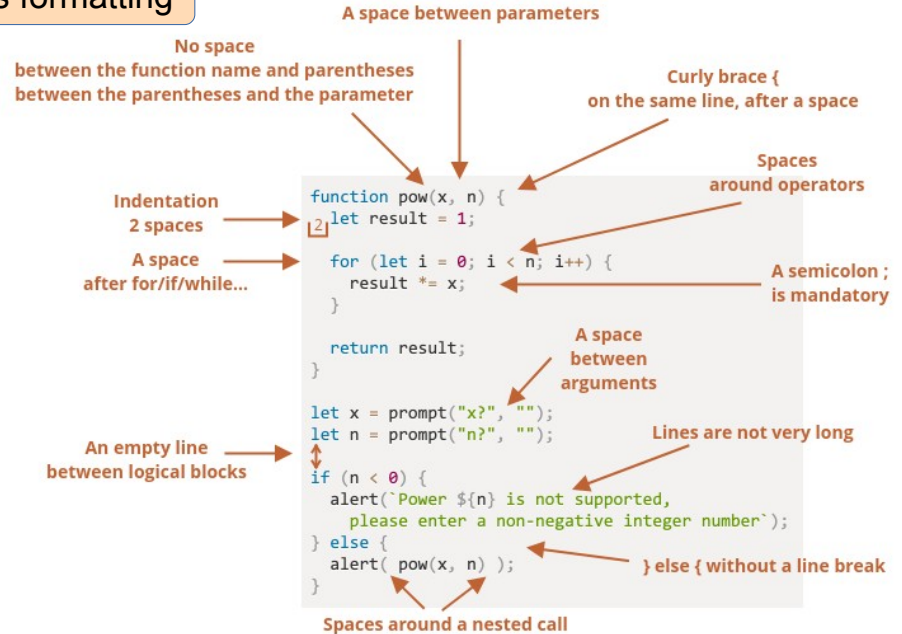
Applications of SCA

Type checking

- Checks correct assignment of object types

Style checking

- Checks style of the code and its formatting



Applications of SCA

Type checking

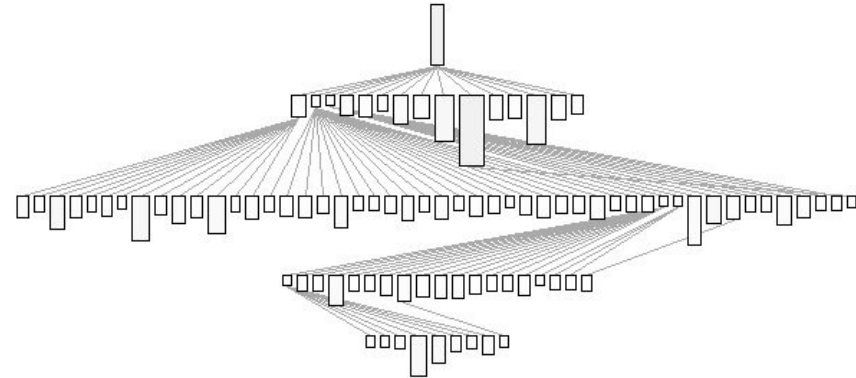
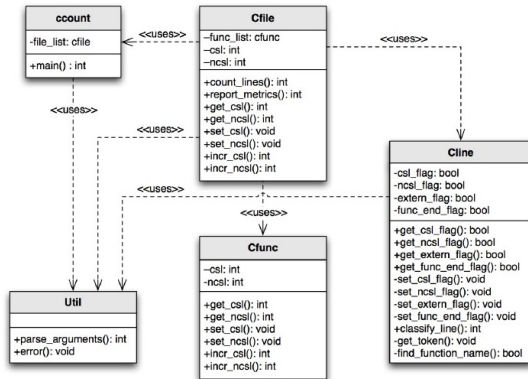
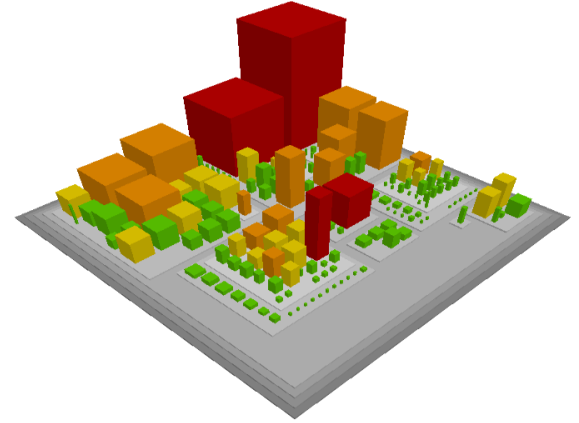
- Checks correct assignment of object types

Style checking

- Checks style of the code and its formatting

Program understanding

- Helps user make sense of large codebase



Applications of SCA

Type checking

- Checks correct assignment of object types

Style checking

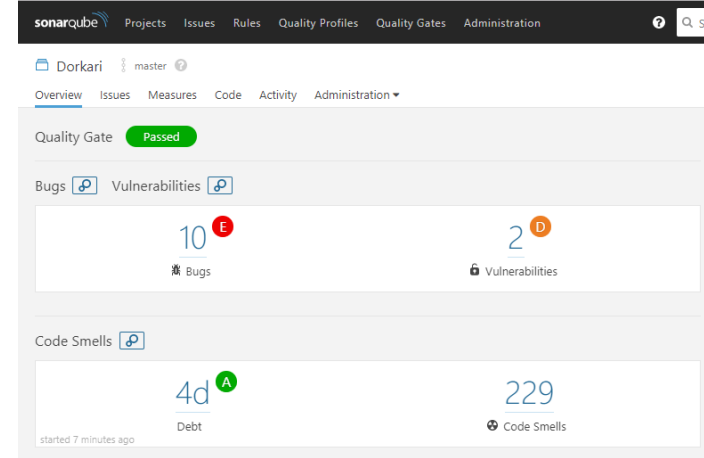
- Checks style of the code and its formatting

Program understanding

- Helps user make sense of large codebase

Issue detection

- Detection of security vulnerabilities
- Detection of bugs
- Detection of code smells



Applications of SCA

Type checking

- Checks correct assignment of object types

Style checking

- Checks style of the code and its formatting

Program understanding

- Helps user make sense of large codebase

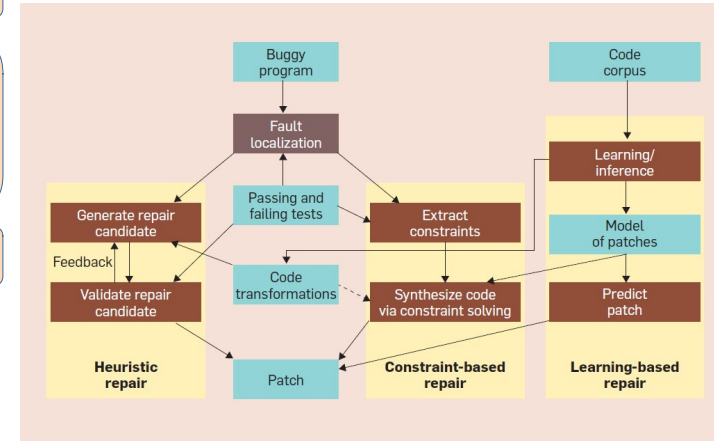
Issue detection

- Detection of security vulnerabilities
- Detection of bugs
- Detection of code smells

Code modification

- Fault injection or code repair

Original Method	With Embedded Mutants
<pre>int Min (int A, int B) { int minVal; minVal = A; if (B < A) { minVal = B; } return (minVal); } // end Min</pre>	<pre>int Min (int A, int B) { int minVal; minVal = A; minVal = B; if (B < A) if (B > A) if (B < minVal) { minVal = B; Bomb(); minVal = A; minVal = failOnZero (B); } return (minVal); } // end Min</pre>



Benefits of SCA

Higher code
quality

Cheaper
bug fixing

No input

Automation

Readability

Coding
guidelines
compliance

Repeatability



Drawbacks of SCA

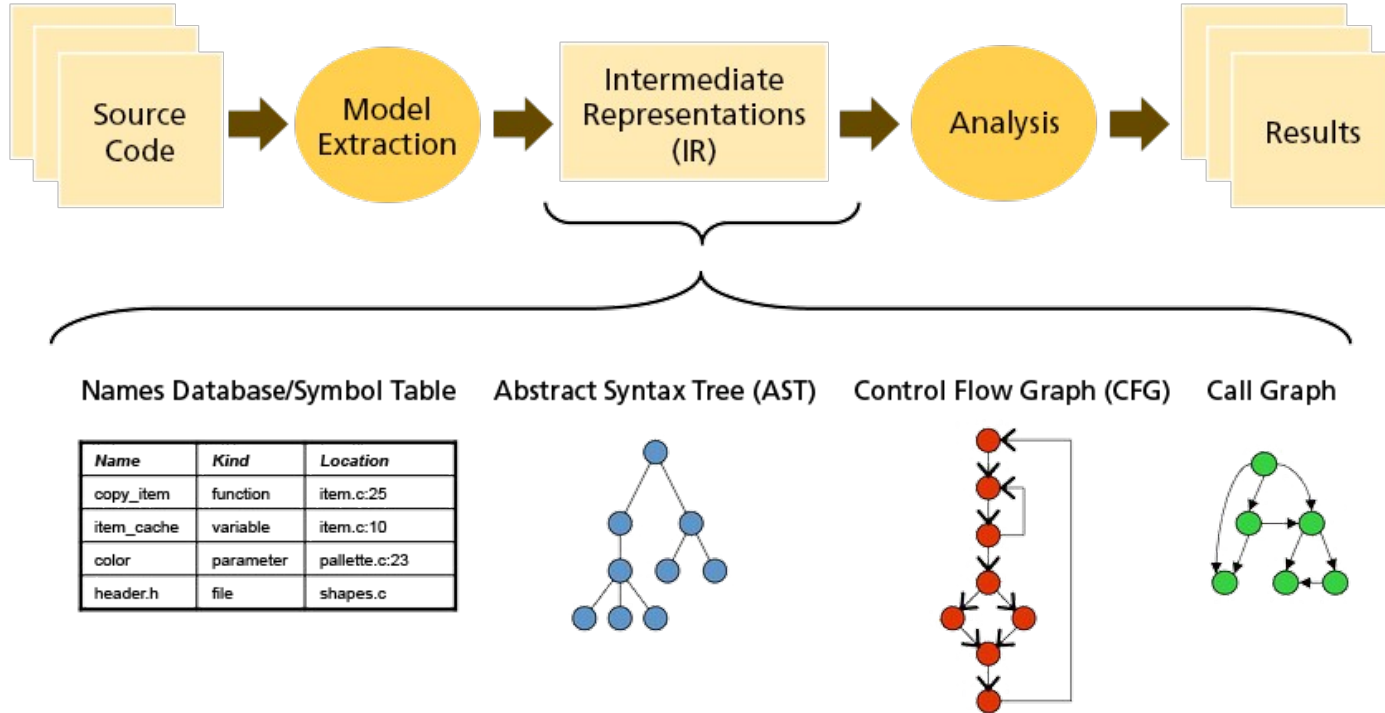
- False positives/negatives
- Only STATIC analysis
- Possible overhead
- Mostly very specialized tools

```
1 public class HelloWorld {  
2  
3     public static void main(String[] args) {  
4         // Prints "Hello, World" to the terminal window.  
5         System.out.println("Hello, World");  
6     }  
7  
8 }
```

Add a private constructor to hide the implicit public one. ... 2 months ago L1 design
Code Smell Major Open Not assigned 30min effort

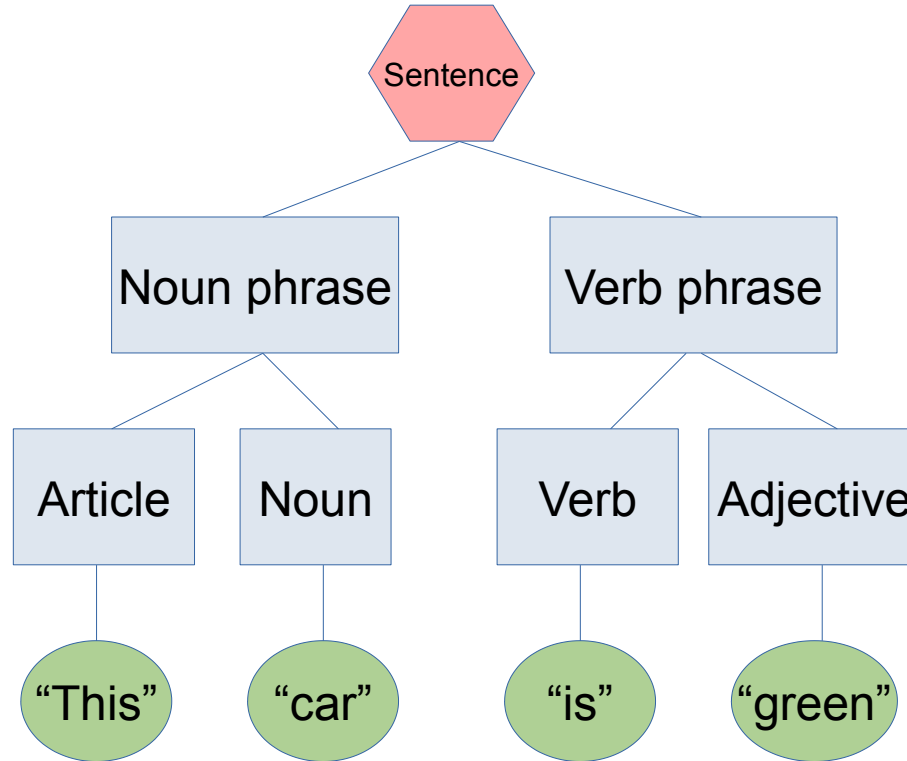
Replace this usage of System.out or System.err by a logger. ... 2 months ago L5 bad-practice, cert
Code Smell Major Open Not assigned 10min effort

How does it work?

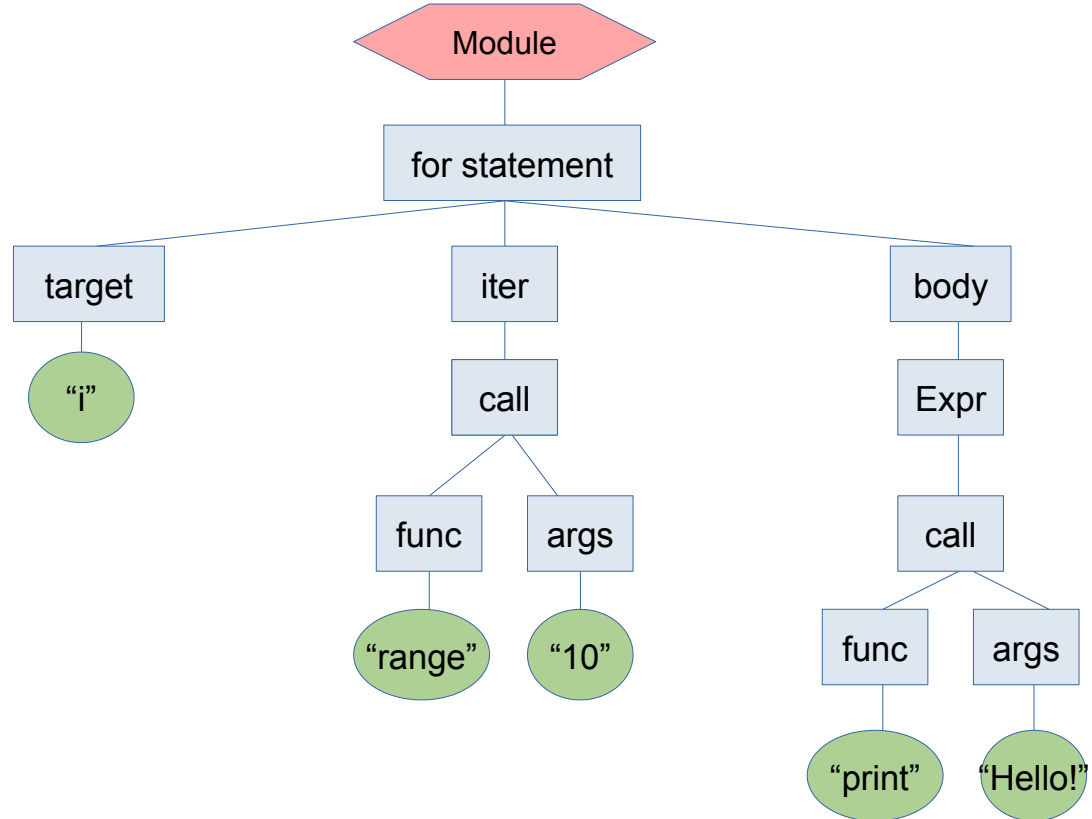


Abstract Syntax Tree (AST)

- This car is green.

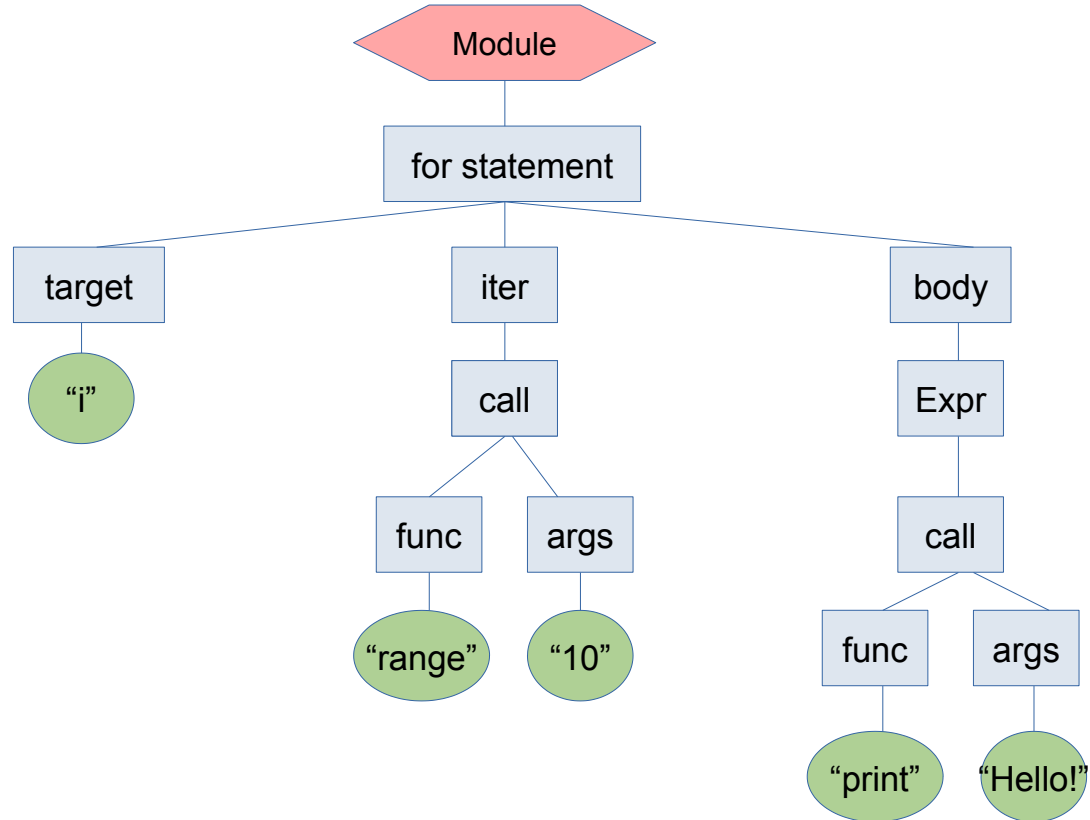


Abstract Syntax Tree (AST)



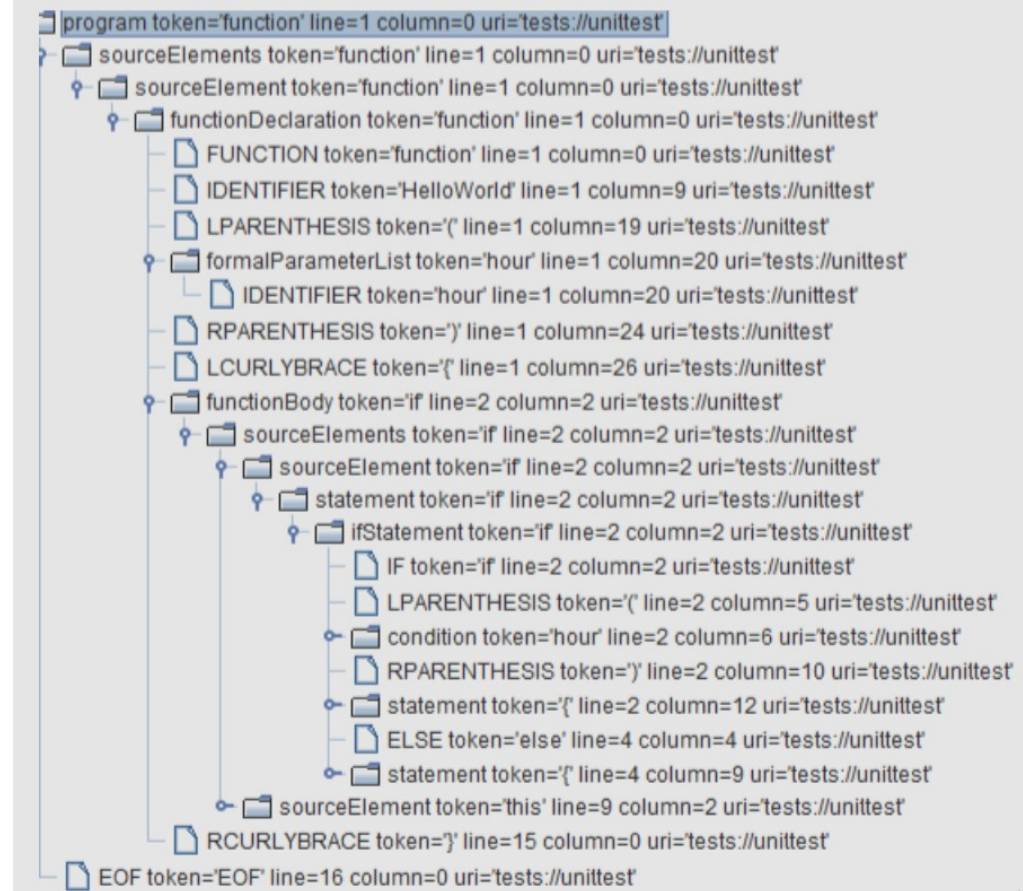
Abstract Syntax Tree (AST)

- for i in range(10):
 print('Hello!')



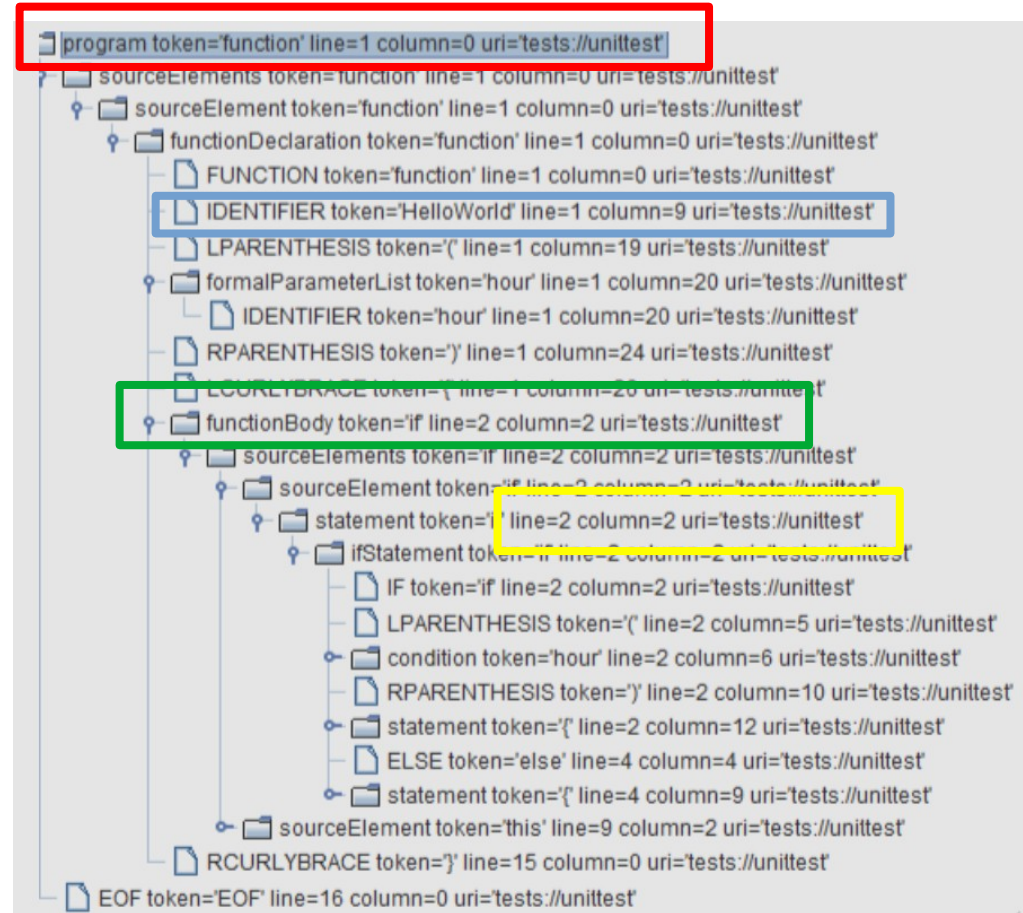
AST – Example

```
function HelloWorld(hour) {  
  if (hour) {  
    this.hour = hour;  
  } else {  
    var date = new Date();  
    this.hour = date.getHours();  
  }  
  this.displayGreeting = function() {  
    if (this.hour >= 22 || this.hour <= 5)  
      document.write("Good night, World!");  
    else  
      document.write("Hello, World!");  
  }  
}
```

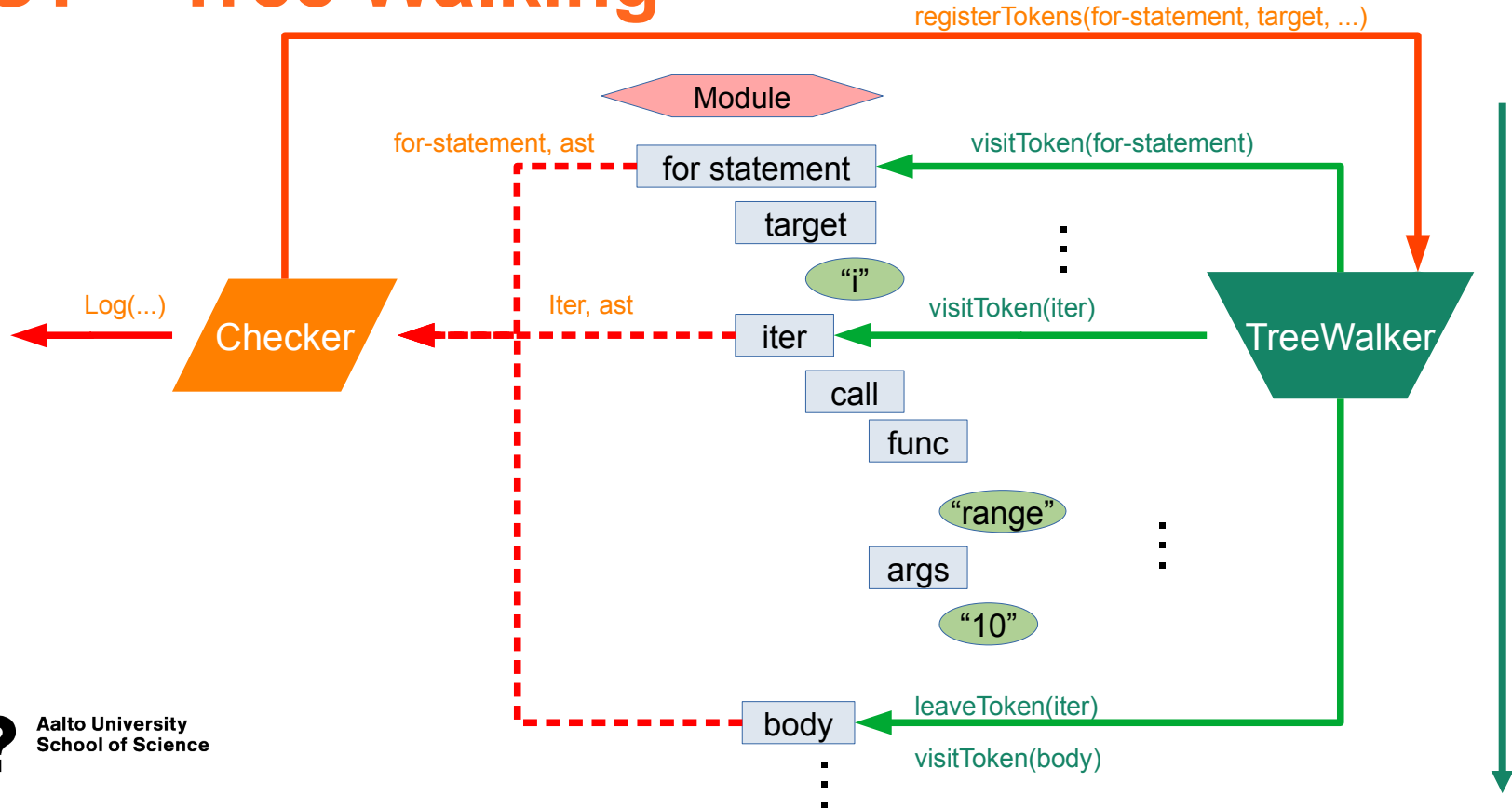


AST – Example

```
function HelloWorld(hour) {  
  if (hour) {  
    this.hour = hour;  
  } else {  
    var date = new Date();  
    this.hour = date.getHours();  
  }  
  this.displayGreeting = function() {  
    if (this.hour >= 22 || this.hour <= 5)  
      document.write("Good night, World!");  
    else  
      document.write("Hello, World!");  
  }  
}
```



AST – Tree Walking



Issue detection - Bug

- Reliability issues
- May crash at runtime
- May cause extremely unpredictable behaviour
- Examples

~ Null pointer dereference

~ Memory leaks

~ Buffer overflow

```
1. class PathManager:
2.     def __init__(self):
3.         self.paths = {}
4.
5.     def add_path(self, name, path):
6.         self.paths[name] = path
7.
8.     def get_normalized_path(self, name):
9.         return self.paths.get(name).lower()
```


Issue detection - Bug

- Reliability issues
- May crash at runtime
- May cause extremely unpredictable behaviour
- Examples

- ~ Null pointer dereference
- ~ Memory leaks
- ~ Buffer overflow

```
1. class PathManager:
2.     def __init__(self):
3.         self.paths = {}
4.
5.     def add_path(self, name, path):
6.         self.paths[name] = path
7.
8.     def get_normalized_path(self, name):
9.         return self.paths.get(name).lower()
```

Issue detection - Vulnerability

- Security issues
- Crash or corrupt the system
- Open space for attack
- Examples
 - ~ Hardcoding credentials
 - ~ Data/SQL injection

```
String accountBalanceQuery =  
    "SELECT accountNumber, balance FROM accounts WHERE account_owner_id = "  
    + request.getParameter("user_id");  
  
try {  
    Statement statement = connection.createStatement();  
    ResultSet rs = statement.executeQuery(accountBalanceQuery);  
    while (rs.next()) {  
        page.addTableRow(rs.getInt("accountNumber"), rs.getFloat("balance"));  
    }  
} catch (SQLException e) { ... }
```

Source: performace.com

```
1.  
2. import pyodbc  
3. info = {  
4.     'user': 'root',  
5.     'password': 'nbusr123'  
6. }  
7. connection_string = "Driver={SQL Server};Server=your_server_name;Database=your_database_name;"  
8. connection = pyodbc.connect(connection_string, attrs_before=info)  
9. connection.close()
```

Issue detection – Code smells

- Maintainability issues
- Decrease readability, architecture quality,...
- Examples

~ Unused private method

~ Feature envy

~ ...

List of code smells:

<https://sourcemaking.com/refactoring/smells>

```
1. class Customer:
2.     def __init__(self, name, address, phone_number, email):
3.         self.name = name
4.         self.address = address
5.         self.phone_number = phone_number
6.         self.email = email
7.
8.     def place_order(self, prod_name, prod_price, quantity, ship_method, pay_method):
9.         ...
10.
11.     def get_invoice(self, order_id, order_date, order_total, pay_status, ship_status):
12.         ...
```

Issue detection – Code smells

- Maintainability issues
- Decrease readability, architecture quality,...
- Examples

~ Unused private method

~ Feature envy

~ ...

List of code smells:

<https://sourcemaking.com/refactoring/smells>

```
1. class Customer:
2.     def __init__(self, name, address, phone_number, email):
3.         self.name = name
4.         self.address = address
5.         self.phone_number = phone_number
6.         self.email = email
7.
8.     def place_order(self, prod_name, prod_price, quantity, ship_method, pay_method):
9.         ...
10.
11.     def get_invoice(self, order_id, order_date, order_total, pay_status, ship_status):
12.         ...
```

Primitive obsession

Issue detection – Code smells

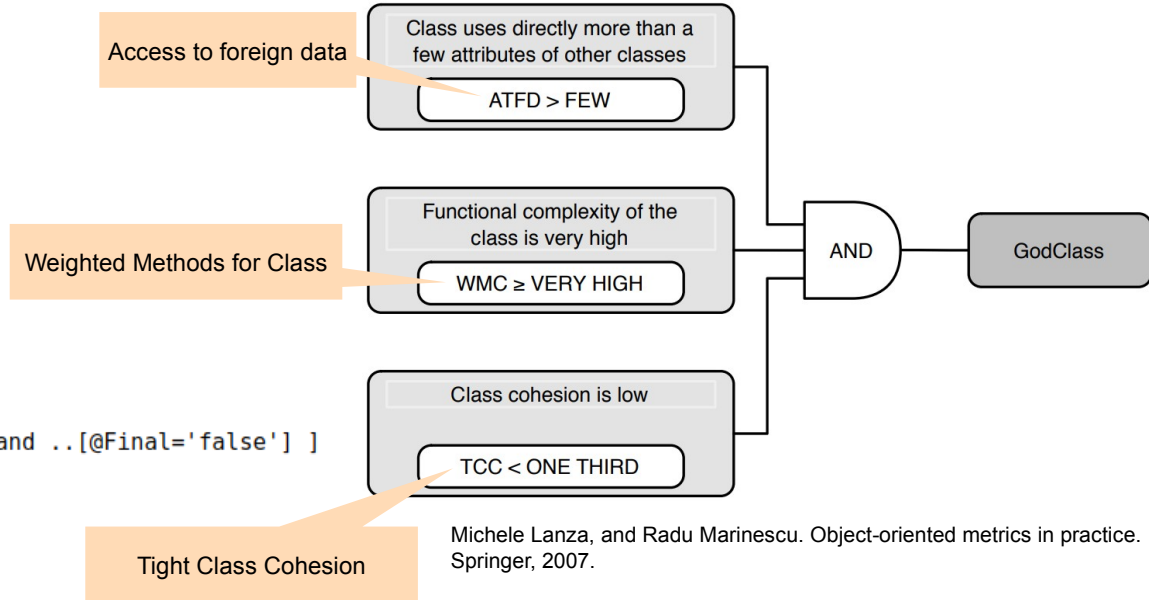
- Pattern-based

```
public class a {  
    Factory f1;  
  
    void myMethod() {  
        Factory f2;  
        int a;  
    }  
}
```

- Factory declaration should be final:

```
//VariableDeclarator[.. /Type/Name[@Image='Factory'] and ..[@Final='false'] ]
```

- Metric-based



Michele Lanza, and Radu Marinescu. Object-oriented metrics in practice. Springer, 2007.

Tools

- Linters
 - PyLint, Prospector, Checkstyle, Resharper, Findbugs, PMD, Sonarlint,...
- Metrics analyzers
 - SourceMeter, Jhawk, Codecity plugins, Pharo, ...
- Code quality analysis platforms
 - SonarQube, Klocwork, Coverity, ...



Sonar Products



- Code quality analysis platform
- Reliability, security, maintainability issues
- Test coverage
- Code duplications
- Technical debt
- Trends
- Multi-language support
- CI integration
- Plugin support
- Self-hosted
- Free community version



- Most features of SonarQube
- Cloud-based
- No plugins
- No free versions



- Real-time issue detection
- IDE integration
- Free

SonarQube - Task



Browse through projects In the SonarCloud demo server:

- <https://sonarcloud.io/explore/projects>
- Focus on projects with higher number of issues
- Explore the issues
 - Try to look for something interesting, e.g.
 - What are the main common types of issues?
 - Where are they located?
 - What kind of functionality is usually affected, etc.?

Week	Lecture	Topic	Assignment deadlines (Tuesdays 10:00 unless otherwise specified)
36	3.9.2024	Introduction and practicalities	
37	10.9.2024	Software quality	Individual assignments 1
38	17.9.2024	Software testing: levels, test case analysis and design	Group registration (DL: 20.9.)
39	24.9.2024	Testing techniques: Black-box, white-box testing	Individual assignments 2, Group assignment 1
40	1.10.2024	Testing techniques: Manual testing	Individual assignments 3
41	8.10.2024	Testing techniques: Black-box, white-box, manual testing	Individual assignments 4
42	15.10.2024	(No lecture)	
43	22.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 5
44	29.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 6, Group assignment 2
45	5.11.2024	Guest lecture	Individual assignments 7
46	12.11.2024	Continuous Integration and Continuous Delivery/Deployment, Test management, and Course summary	Individual assignments 8
47	19.11.2024	(No lecture)	Individual assignments 9
48	26.11.2024	(No lecture)	Individual assignments 10, Group assignment 3



Next steps

- Individual assignments in MyCourses
 - Acceptance testing 2.2
 - Static code analysis 1
- Group assignments in MyCourses
 - Team testing

Deadline: before next lecture (Tuesday by 10:00)

- Individual assignment in MyCourses
 - Static code analysis 2

Deadline: before the next lecture in two weeks (5.11., by 10:00)

- Getting help
 - Ask in the course chat, see Communication section in MyCourses
 - Personal questions by email

Use the related reading
for more details.
Reserve enough time to
set up your environment.



Study smarter, not harder!

- Plan: when and where
- Go deep at your own pace
- Get some rest
- Practice makes perfect
- Enjoy it ◀◀

Stanislav Chren

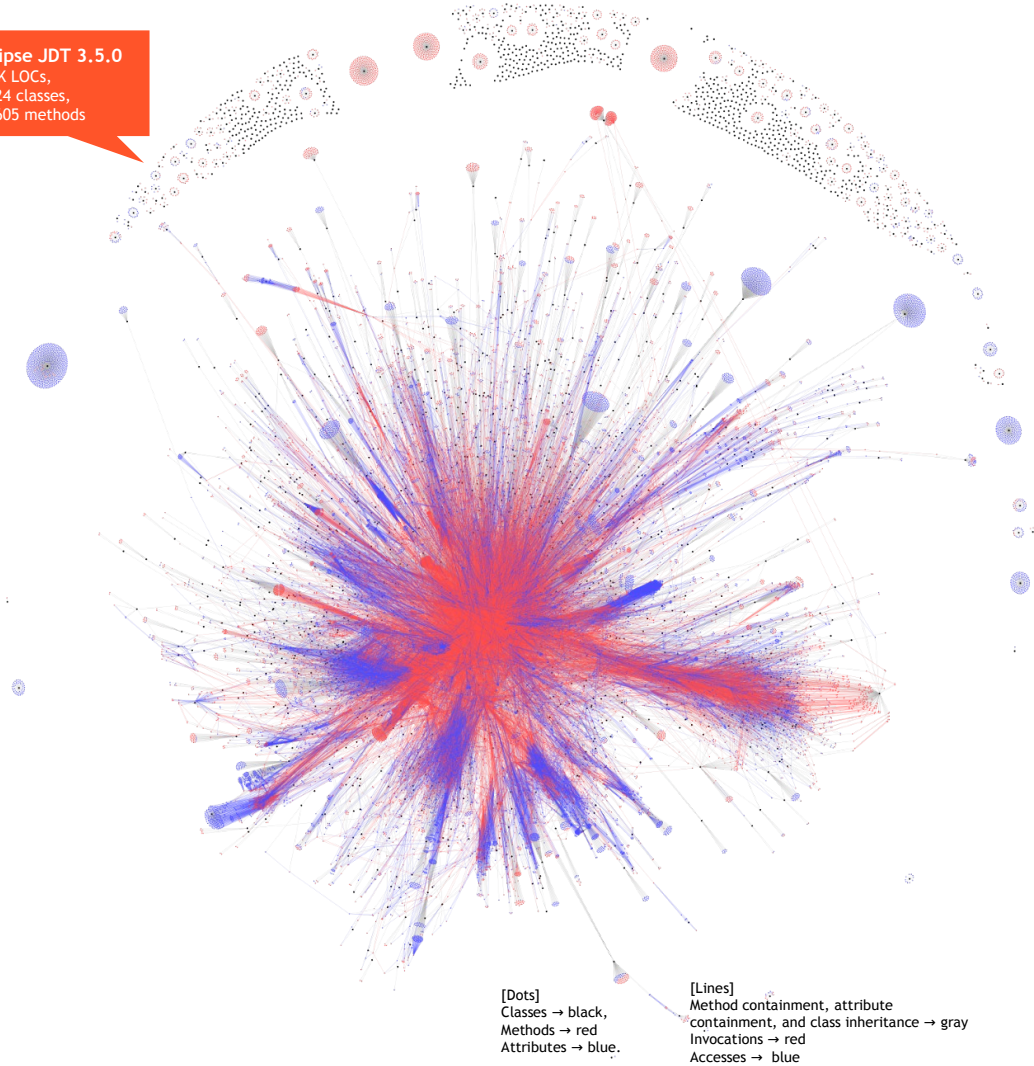
A”



Introduction

Eclipse JDT 3.5.0
350K LOCs,
1.324 classes,
23.605 methods

- Modern systems are very large & complex in terms of structure & runtime behaviour
- We need ways to understand attributes of software, represent it in a concise way and use it to track for software & development process improvement
- Software Measurement and Metrics are one of the aspects we can consider

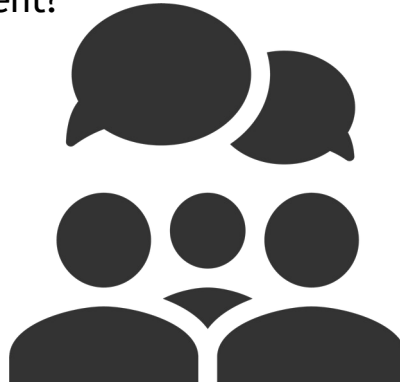


Background on Software Measurement

Measurement

- **Measurement** is the process by which **numbers** or **symbols** are assigned to **attributes of entities in the real world** in such a way as to describe them according to **clearly defined rules**
(Fenton & Pfleeger, 1997)

Why do we need software
measurement?



Measurement

- **Measurement** is the process by which **numbers** or **symbols** are assigned to **attributes of entities** in **the real world** in such a way as to describe them according to **clearly defined rules**
(Fenton & Pfleeger, 1997)

Why do we need software
measurement?

To avoid anecdotal evidence without a clear research

To increase the visibility and the understanding of the process

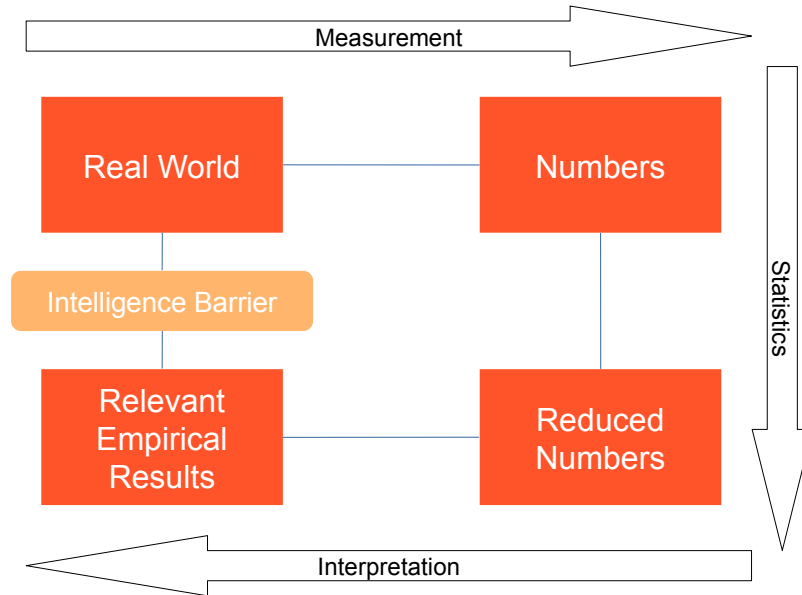
To analyze the software development process

To make predictions through statistical models

To set measurable targets for the software products

Measurement Process

- The measurement process goes from the **real world** to the **numerical representation**
- Interpretation goes from the **numerical representation** to the **relevant empirical results**



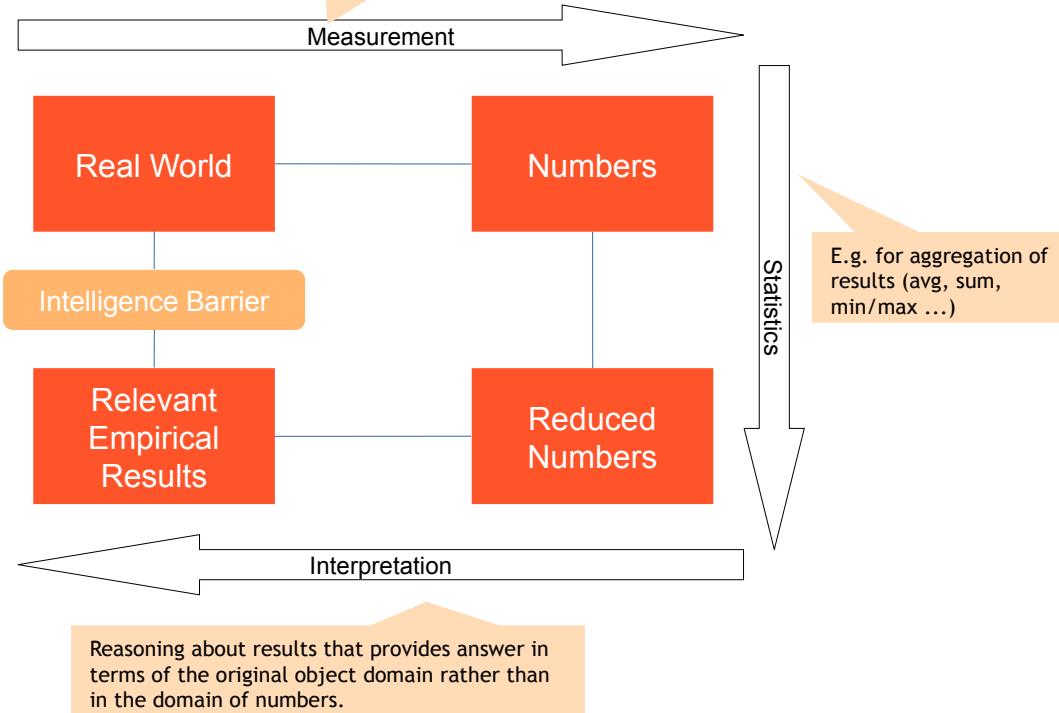
Measurement Process

- The measurement process goes from the **real world** to the **numerical representation**
- Interpretation goes from the **numerical representation** to the **relevant empirical results**

A **measure** is a mapping between the real world and mathematical/formal world

- Different mappings give different views of the world (height, weight, ...)
- The **mapping relates attributes to mathematical objects**; it does not relate entities to mathematical objects
- **Direct vs indirect**
- **Objective vs subjective**

Missing intuitive mapping between real world objects and numerical results, i.e. barrier between questions about objects and the answers to those questions.



Valid Measures

- The validity of a measure depends on definition of the attribute coherent with the specification of the real world

		Measurement	
		Low	High
Real World	Low	TRUE NEGATIVE	FALSE POSITIVE
	High	FALSE NEGATIVE	TRUE POSITIVE

- Q: Is LOC a valid measure of productivity?*

Valid Measures

- The validity of a measure depends on definition of the attribute coherent with the specification of the real world

		Measurement	
		Low	High
Real World	Low	TRUE NEGATIVE	FALSE POSITIVE
	High	FALSE NEGATIVE	TRUE POSITIVE

- Q: Is LOC a valid measure of productivity?
 - ~ For example, 100K print() statements vs 100K of complex loops and algorithms
 - ~ Can we compare LOC between different projects?

Valid Measures - Example

- **Code coverage** is a measure giving an indication of how much of the source code has been run (“covered”) by running the tests

```
[01] * multiples. Repeat until there are no more multiples
[02] * in the array.
[03] */
[04] public class PrimeGenerator
[05] {
[06]     private static boolean[] crossedOut;
[07]     private static int[] result;
[08]     public static int[] generatePrimes(int maxValue){
[09]         if (maxValue < 2){
[10]             return new int[0];
[11]         }else{
[12]             uncrossIntegersUpTo(maxValue);
[13]             crossOutMultiples();
[14]             putUncrossedIntegersIntoResult();
[15]             return result;
[16]         }
[17]     }
[18] }
```

Valid Measures - Example

- **Code coverage** is a measure giving an indication of how much of the source code has been run (“covered”) by running the tests
- *“...A program with high code coverage has been more thoroughly tested and has a lower chance of containing software bugs than a program with low code coverage...”*
(Wikipedia, until 2022)
- **Q: Would you consider code coverage as a valid measure of how much thoroughly one software project has been tested and how good the tests are?**
 - Suppose you have two projects with the code coverage P1: 70% vs P2: 80%
 - Would you generally consider P2 to be “better” (more accurately) tested than P1?

```
[01] * multiples. Repeat until there are no more multiples
[02] * in the array.
[03] */
[04] public class PrimeGenerator
[05] {
[06]     private static boolean[] crossedOut;
[07]     private static int[] result;
[08]     public static int[] generatePrimes(int maxValue){
[09]         if (maxValue < 2){
[10]             return new int[0];
[11]         }else{
[12]             uncrossIntegersUpTo(maxValue);
[13]             crossOutMultiples();
[14]             putUncrossedIntegersIntoResult();
[15]             return result;
[16]         }
[17]     }
[18] }
```

Valid Measures - Example

- **Code coverage** is a measure giving an indication of how much of the source code has been run (“covered”) by running the tests
- *“...A program with high code coverage has been more thoroughly tested and has a lower chance of containing software bugs than a program with low code coverage...”*
(Wikipedia, until 2022)
- **Q: Would you consider code coverage as a valid measure of how much thoroughly one software project has been tested and how good the tests are?**
 - Suppose you have two projects with the code coverage P1: 70% vs P2: 80%
 - Would you generally consider P2 to be “better” (more accurately) tested than P1?

```
[01] * multiples. Repeat until there are no more multiples
[02] * in the array.
[03] */
[04] public class PrimeGenerator
[05] {
[06]     private static boolean[] crossedOut;
[07]     private static int[] result;
[08]     public static int[] generatePrimes(int maxValue){
[09]         if (maxValue < 2){
[10]             return new int[0];
[11]         }else{
[12]             uncrossIntegersUpTo(maxValue);
[13]             crossOutMultiples();
[14]             putUncrossedIntegersIntoResult();
[15]             return result;
[16]         }
[17]     }
[18] }
```

Q: Would you consider every test covering the same lines as equal?

Valid Measures - Example

- **Code coverage** is a measure giving an indication of how much of the source code has been run (“covered”) by running the tests
- *“...A program with high code coverage has been more thoroughly tested and has a lower chance of containing software bugs than a program with low code coverage...”*
(Wikipedia, until 2022)
- **Q: Would you consider code coverage as a valid measure of how much thoroughly one software project has been tested and how good the tests are?**
 - Suppose you have two projects with the code coverage
P1: 70% vs P2: 80%
 - Would you generally consider P2 to be “better” (more accurately) tested than P1?

```
[01] * multiples. Repeat until there are no more multiples
[02] * in the array.
[03] */
[04] public class PrimeGenerator
[05] {
[06]     private static boolean[] crossedOut;
[07]     private static int[] result;
[08]     public static int[] generatePrimes(int maxValue){
[09]         if (maxValue < 2){
[10]             return new int[0];
[11]         }else{
[12]             uncrossIntegersUpTo(maxValue);
[13]             crossOutMultiples();
[14]             putUncrossedIntegersIntoResult();
[15]             return result;
[16]         }
[17]     }
[18] }
```

Q: Would you consider every test covering the same lines as equal?

```
[01] double div (int x, int y){
[02]     return x/y;
[03] }
```

Coverage 100%

```
assertEquals(1.0, div(1,1));
```

Coverage 100%

```
assertEquals(0.66, div(2,3), 0.1);
```

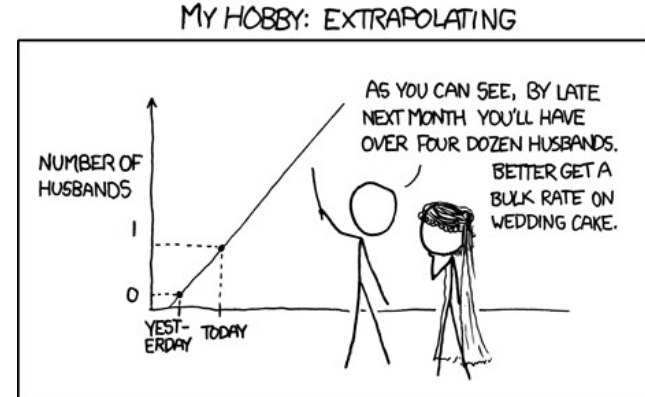
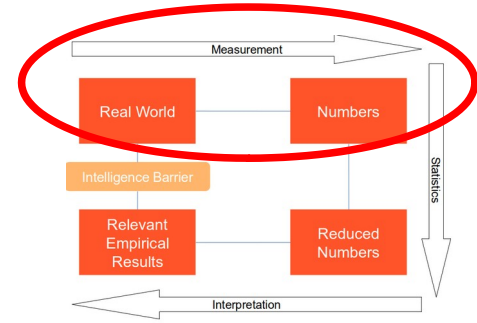
Valid Measures - Example

- According to Martin Fowler:
- → *“Test coverage is a useful tool for finding untested parts of a codebase. Test coverage is of little use as a numeric statement of how good your tests are”*

(<https://martinfowler.com/bliki/TestCoverage.html>)

- In this case, **we do not respect the representation condition**: when we assign symbols to the attributes of entities we need to preserve the meaning of relationships when moving entities from the real world to the numerical world

		Measurement	
		Low	High
Real World	Low	TRUE NEGATIVE	FALSE POSITIVE
	High	FALSE NEGATIVE	TRUE POSITIVE

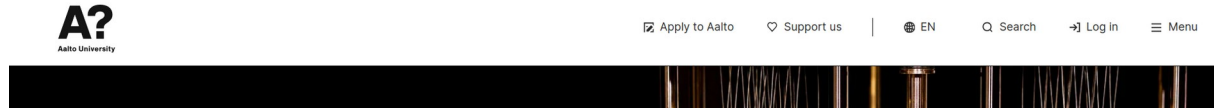


<https://xkcd.com/605/>

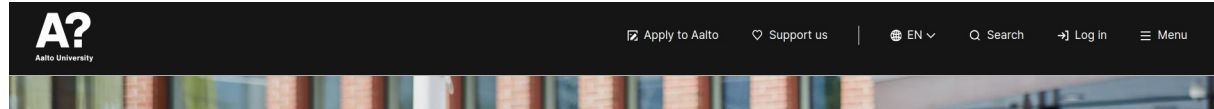
Valid Measures

- A/B Testing is a kind of **randomized experiment** in which you can propose **two variants** of the same application to the users
- Example
 - We set-up an experiment with two browsers and two variations of the same webpage
 - **Conversion Rate:** % of users completing an action

Variant A



Variant B

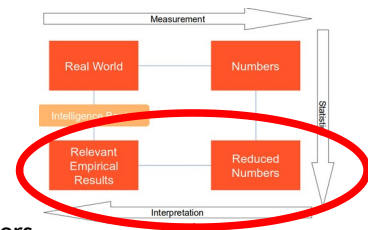


200 users	Conv Rate A	Conv Rate B
Firefox	87.50%	100.00%
Chrome	50.00%	62.50%

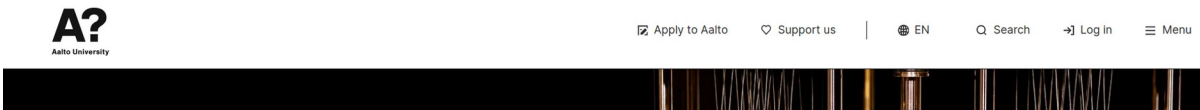
Q: What can you conclude? Which alternative is better?

Valid Measures

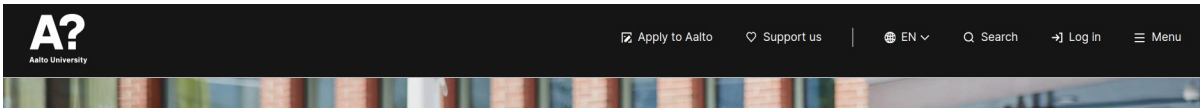
- A/B Testing is a kind of **randomized experiment** in which you can propose **two variants** of the same application to the users
- Example
 - We set-up an experiment with two browsers and two variations of the same webpage
 - **Conversion Rate:** % of users completing an action



Variant A



Variant B



200 users	Conv Rate A	Conv Rate B
Firefox	87.50%	100.00%
Chrome	50.00%	62.50%

	Conv Rate A	Conv Rate B
Firefox	70/80 = 87.5%	20/20 = 100%
Chrome	10/20 = 50%	50/80 = 62.5%
Both	80/100 = 80%	70/100 = 70%

Q: What can you conclude? Which alternative is better?

Measurement Scales

- Every measurement is mapped to a scale (nominal, ordinal, interval, rational)
- Considering the scale is important for the allowed operations

Name of prog. Language
(e.g. Java, C++,...)

Ranking of
failures/severity
(e.g. critical, minor,...)

Start date,
end date of activities

Lines of code
(as a measure of
program size)

	$\neq, =$	$<, >$	min,max	median	avg	prop
Nominal →						
Ordinal →						
Interval →						
Rational →						



Aalto University
School of Science

Measurement Scales - Example

Level	Severity	Description
6	Blocker	Prevents function from being used, no work-around, blocking progress on multiple fronts
5	Critical	Prevents function from being used, no work-around
4	Major	Prevents function from being used, but a work-around is possible
3	Normal	A problem making a function difficult to use but no special work-around is required
2	Minor	A problem not affecting the actual function, but the behavior is not natural
1	Trivial	A problem not affecting the actual function, a typo would be an example

Order	Severity	P1	P2
6	Blocker	2	10
5	Critical	36	19
4	Major	25	22
3	Normal	15	32
2	Minor	2	5
1	Trivial	121	113

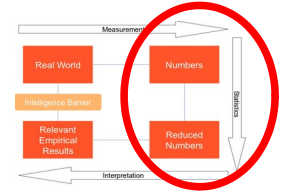
- **Q: Is it meaningful to use the weighted average to compare two projects in terms of severity of the open issues?**

$$Sev(P_n) = avg\left(\sum issues_i * weight_i\right)$$

$$Sev(P_1) = avg(2 * 6 + 36 * 5 + 25 * 4 + 15 * 3 + 2 * 2 + 121 * 1) = 77$$

$$Sev(P_2) = avg(10 * 6 + 19 * 5 + 22 * 4 + 32 * 3 + 5 * 2 + 113 * 1) = 77$$

Measurement Scales - Example



Level	Severity	Description
6	Blocker	Prevents function from being used, no work-around, blocking progress on multiple fronts
5	Critical	Prevents function from being used, no work-around
4	Major	Prevents function from being used, but a work-around is possible
3	Normal	A problem making a function difficult to use but no special work-around is required
2	Minor	A problem not affecting the actual function, but the behavior is not natural
1	Trivial	A problem not affecting the actual function, a typo would be an example

Order	Severity	P1	P2
6	Blocker	2	10
5	Critical	36	19
4	Major	25	22
3	Normal	15	32
2	Minor	2	5
1	Trivial	121	113

- **Q: Is it meaningful to use the weighted average to compare two projects in terms of severity of the open issues?**

$$Sev(P_n) = avg\left(\sum issues_i * weight_i\right)$$

$$Sev(P_1) = avg(2 * 6 + 36 * 5 + 25 * 4 + 15 * 3 + 2 * 2 + 121 * 1) = 77$$

$$Sev(P_2) = avg(10 * 6 + 19 * 5 + 22 * 4 + 32 * 3 + 5 * 2 + 113 * 1) = 77$$

Software Metrics

Measures of Size

Q: What size measures can you come up with in this example?

```
[01] * multiples. Repeat until there are no more multiples
[02] * in the array.
[03] */
[04] public class PrimeGenerator
[05] {
[06]     private static boolean[] crossedOut;
[07]     private static int[] result;
[08]     public static int[] generatePrimes(int maxValue){
[09]         if (maxValue < 2){
[10]             return new int[0];
[11]         }else{
[12]             uncrossIntegersUpTo(maxValue);
[13]             crossOutMultiples();
[14]             putUncrossedIntegersIntoResult();
[15]             return result;
[16]         }
[17]     }
[18] }
```

Measures of Size

LOC = 18
(Lines of Code)

CLOC = 3
(Commented
Lines of Code)

NCLOC = 3
(Commented
Lines of Code)

```
[01] * multiples. Repeat until there are no more multiples
[02] * in the array.
[03] */
[04] public class PrimeGenerator
[05] {
[06]     private static boolean[] crossedOut;
[07]     private static int[] result;
[08]     public static int[] generatePrimes(int maxValue){
[09]         if (maxValue < 2){
[10]             return new int[0];
[11]         }else{
[12]             uncrossIntegersUpTo(maxValue);
[13]             crossOutMultiples();
[14]             putUncrossedIntegersIntoResult();
[15]             return result;
[16]         }
[17]     }
[18] }
```


Measures of Size

LOC = 18
(Lines of Code)

CLOC = 3
(Commented
Lines of Code)

NCLOC = 3
(Commented
Lines of Code)

NOC = 1
(Number of
Classes)

NOM = 1
(Number of
Methods)

```
[01] * multiples. Repeat until there are no more multiples
[02] * in the array.
[03] */
[04] public class PrimeGenerator
[05] {
[06]     private static boolean[] crossedOut;
[07]     private static int[] result;
[08]     public static int[] generatePrimes(int maxValue){
[09]         if (maxValue < 2){
[10]             return new int[0];
[11]         }else{
[12]             uncrossIntegersUpTo(maxValue);
[13]             crossOutMultiples();
[14]             putUncrossedIntegersIntoResult();
[15]             return result;
[16]         }
[17]     }
[18] }
```

Measures of Size

LOC = 18
(Lines of Code)

CLOC = 3
(Commented
Lines of Code)

NCLOC = 3
(Commented
Lines of Code)

NOC = 1
(Number of
Classes)

NOM = 1
(Number of
Methods)

Q: Can you think of something
more useful to report than
CLOC?

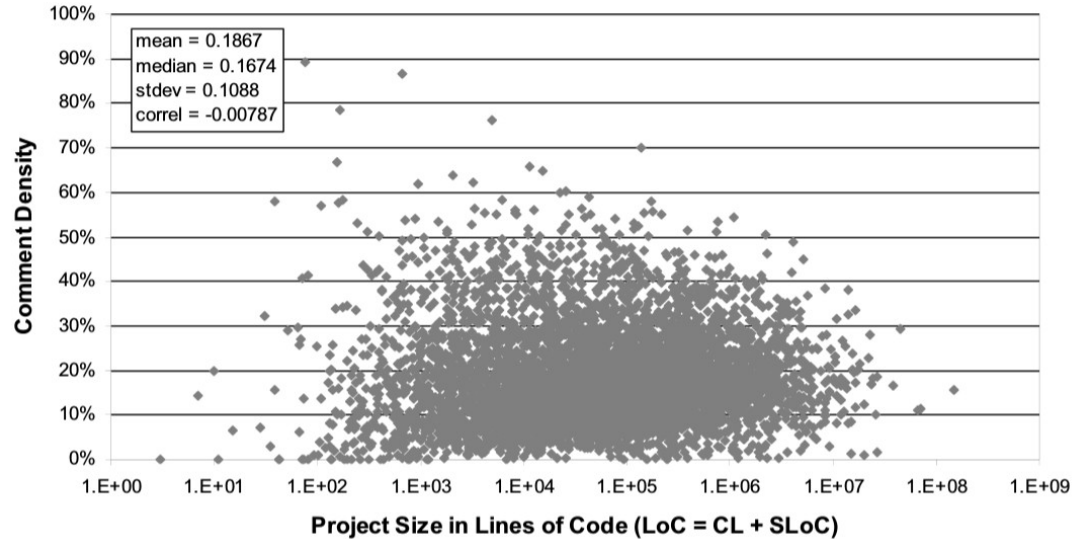
```
[01] * multiples. Repeat until there are no more multiples
[02] * in the array.
[03] */
[04] public class PrimeGenerator
[05] {
[06]     private static boolean[] crossedOut;
[07]     private static int[] result;
[08]     public static int[] generatePrimes(int maxValue){
[09]         if (maxValue < 2){
[10]             return new int[0];
[11]         }else{
[12]             uncrossIntegersUpTo(maxValue);
[13]             crossOutMultiples();
[14]             putUncrossedIntegersIntoResult();
[15]             return result;
[16]         }
[17]     }
[18] }
```

Measures of Size

- Size can be used for **normalization of existing measures**

~ E.g. instead of CLOC, we can use Comments Density (CD)

$$CD = \frac{CLOCs}{LOCs} = \frac{3}{18} = 0.16$$

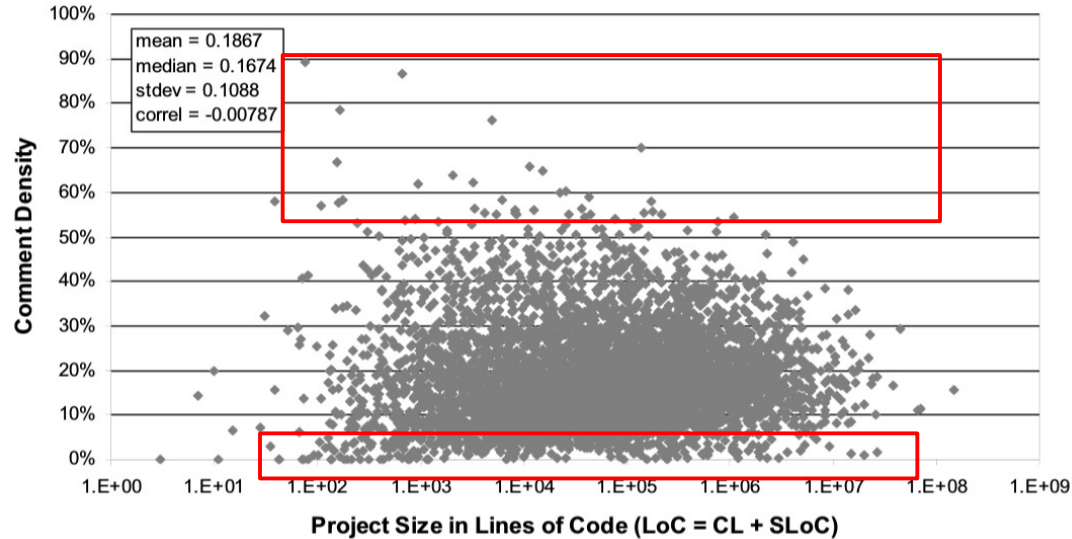


Measures of Size

- Size can be used for **normalization of existing measures**

E.g. instead of CLOC, we can use Comments Density (CD)

$$CD = \frac{CLOCs}{LOCs} = \frac{3}{18} = 0.16$$



Measures of Complexity

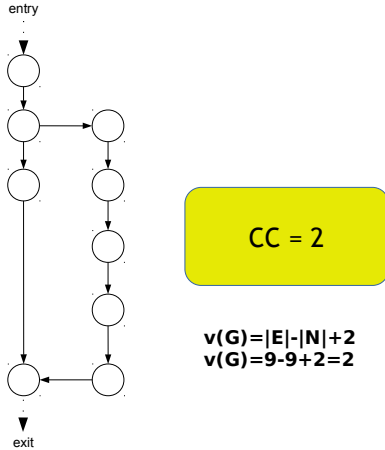
- McCabe's Cyclomatic Complexity (CC)
 - $v(G) = |E| - |N| + 2P$

```
[01] * multiples. Repeat until there are no more multiples
[02] * in the array.
[03] */
[04] public class PrimeGenerator
[05] {
[06]     private static boolean[] crossedOut;
[07]     private static int[] result;
[08]     public static int[] generatePrimes(int maxValue){
[09]         if (maxValue < 2){
[10]             return new int[0];
[11]         }else{
[12]             uncrossIntegersUpTo(maxValue);
[13]             crossOutMultiples();
[14]             putUncrossedIntegersIntoResult();
[15]             return result;
[16]         }
[17]     }
[18] }
```

Measures of Complexity

- McCabe's Cyclomatic Complexity (CC)

- $v(G) = |E| - |N| + 2P$

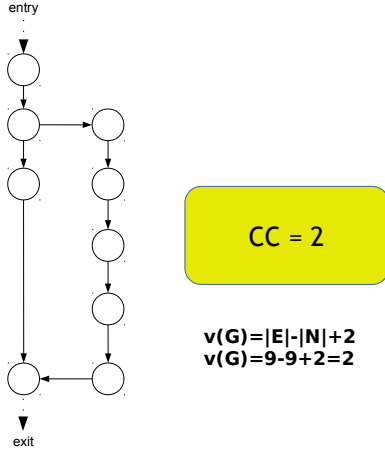


```
[01] * multiples. Repeat until there are no more multiples
[02] * in the array.
[03] */
[04] public class PrimeGenerator
[05] {
[06]     private static boolean[] crossedOut;
[07]     private static int[] result;
[08]     public static int[] generatePrimes(int maxValue){
[09]         if (maxValue < 2){
[10]             return new int[0];
[11]         }else{
[12]             uncrossIntegersUpTo(maxValue);
[13]             crossOutMultiples();
[14]             putUncrossedIntegersIntoResult();
[15]             return result;
[16]         }
[17]     }
[18] }
```

Measures of Complexity

- McCabe's Cyclomatic Complexity (CC)

- $v(G) = |E| - |N| + 2P$



Q: How can be the Cyclomatic Complexity useful?

```
[01] * multiples. Repeat until there are no more multiples
[02] * in the array.
[03] */
[04] public class PrimeGenerator
[05] {
[06]     private static boolean[] crossedOut;
[07]     private static int[] result;
[08]     public static int[] generatePrimes(int maxValue){
[09]         if (maxValue < 2){
[10]             return new int[0];
[11]         }else{
[12]             uncrossIntegersUpTo(maxValue);
[13]             crossOutMultiples();
[14]             putUncrossedIntegersIntoResult();
[15]             return result;
[16]         }
[17]     }
[18] }
```

Measures of Complexity

- CC is more about syntactic complexity, NOT semantic complexity

```
[04] public class PrimeGenerator
[05] {
[06]     private static boolean[] crossedOut;
[07]     private static int[] result;
[08]
[09]     public static int[] generatePrimes(int maxValue){
[10]         if (maxValue < 2){
[11]             return new int[0];
[12]         }else{
[13]             uncrossIntegersUpTo(maxValue);
[14]             crossOutMultiples();
[15]             putUncrossedIntegersIntoResult();
[16]             return result;
[17]         }
[18] }
```

```
[04] public class A
[05] {
[06]     private static boolean[] c;
[07]     private static int[] b;
[08]
[09]     public static int[] generate(int m){
[10]         if (m < 2){
[11]             return new int[0];
[12]         }else{
[13]             methodOne(m);
[14]             methodTwo();
[15]             methodThree();
[16]             return b;
[17]         }
[18] }
```

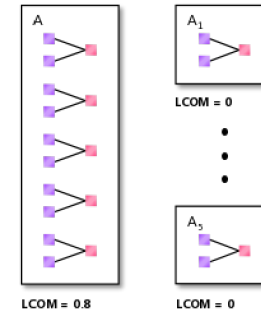
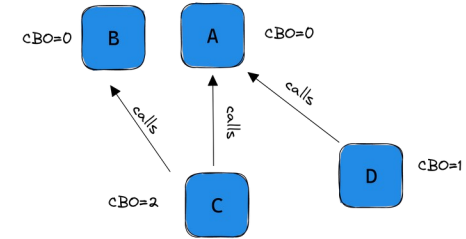
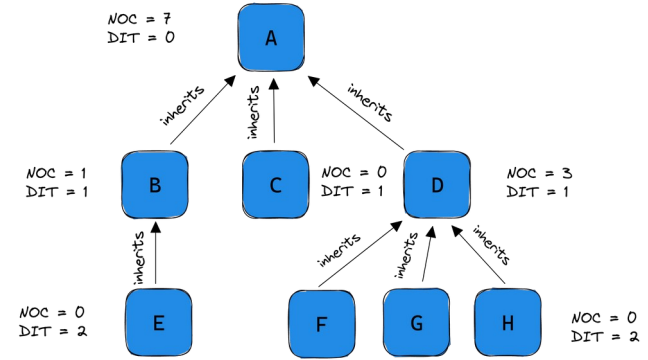
Q: Which code fragment is easier to understand?

- Be careful when comparing projects in terms of average complexity

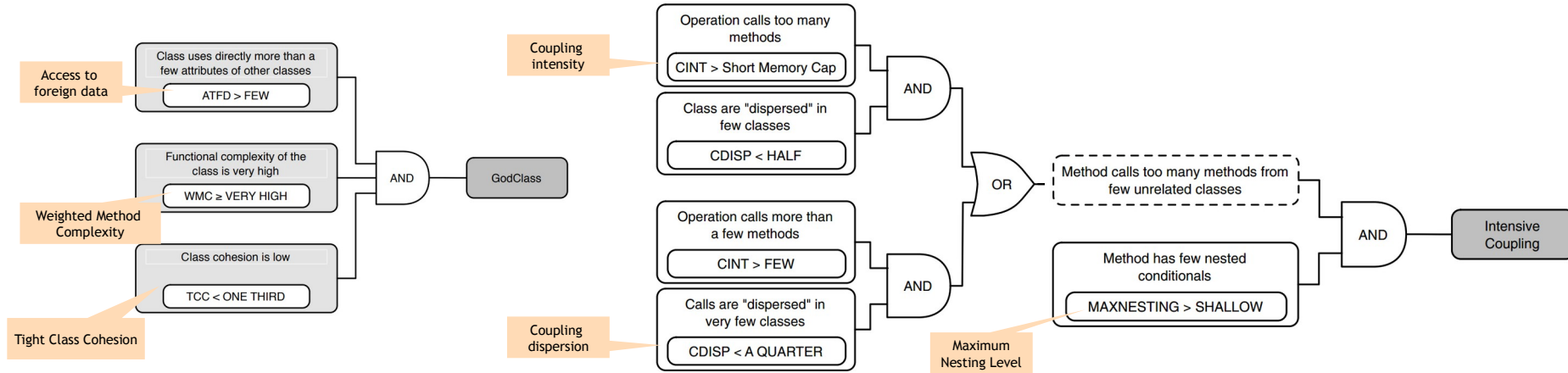
Object-oriented Metrics

Chidamber & Kemerer Suite

- **WMC**: Weighted methods per class
- **DIT**: Depth of Inheritance Tree
- **NOC**: Number of Children
- **CBO**: Coupling between object classes
- **RFC**: Response for a class
- **LCOM**: Lack of cohesion in methods



Object-oriented Metrics – Code Smells



Software Measurement Models

Software Measurement Pitfalls

Collecting meaningless measurements

- Measurement must be goal-driven

Not analysing measurements

- Numbers need detailed analysis

Setting unrealistic targets

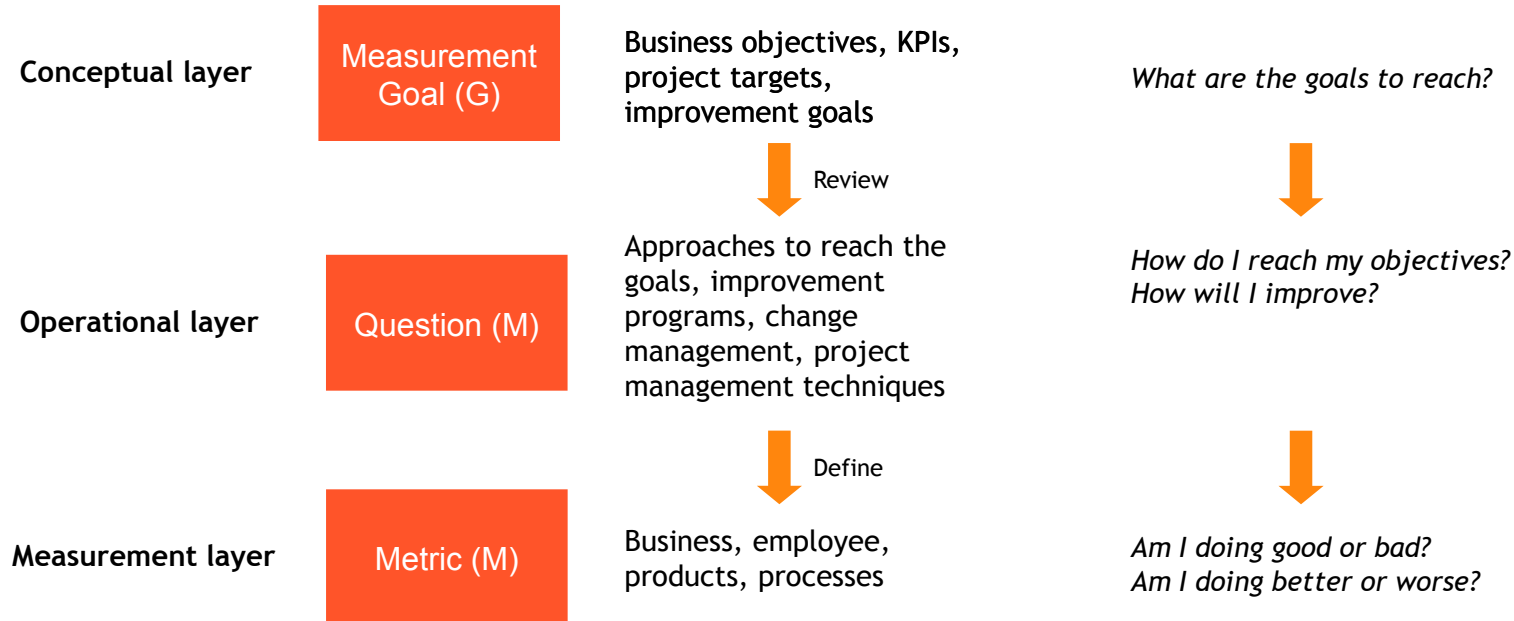
- Targets should not be uniquely defined based on the numbers

Paralysis by analysis

- Measurement is a key activity in management, not a separate activity

The Goal-Question-Metric (GQM)

- Introduced in 1986 by Rombach and Basili
- Top-down deductive instrument to derive suitable measure from prescribed goals



GQM - Examples

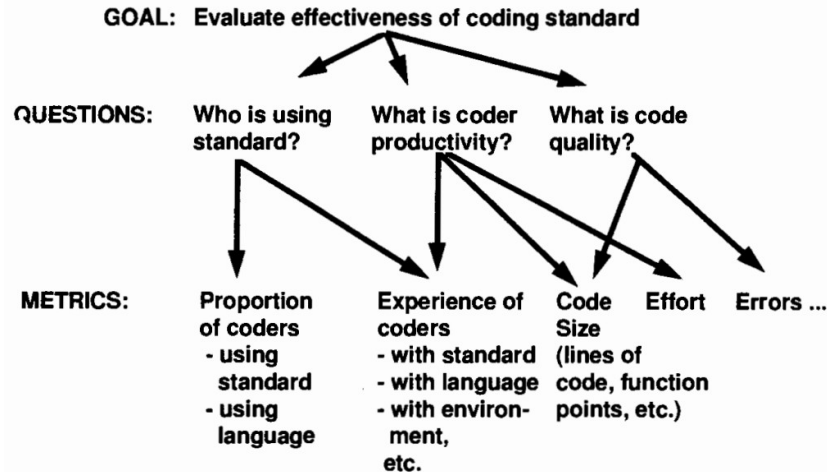


Figure 3.2: Example of deriving metrics from goals and questions

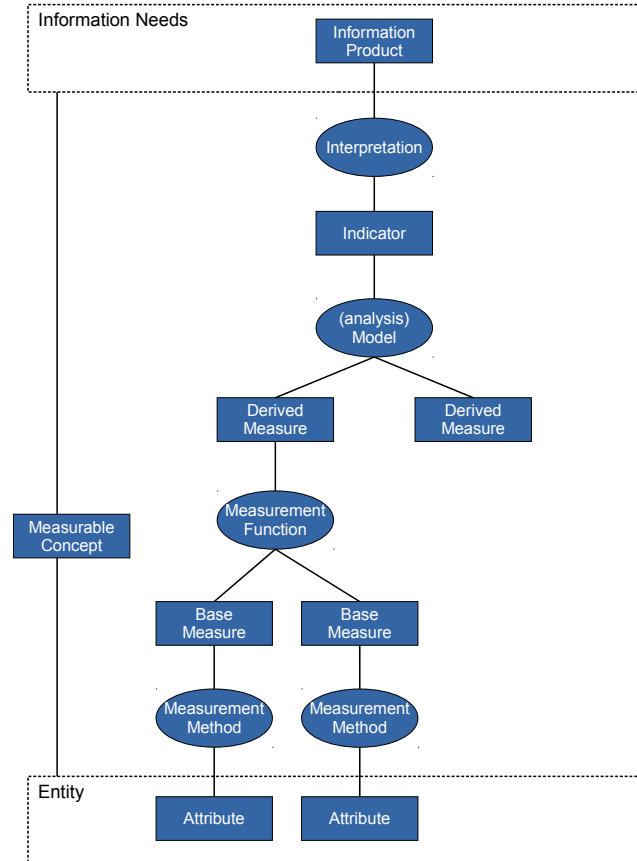
Fenton, Norman, and James Bieman. Software metrics: a rigorous and practical approach. CRC press, 2014.

Table 3.3: Examples of AT&T goals, questions and metrics (Barnard and Price, 1994)

Goal	Questions	Metrics
Plan	How much does the inspection process cost? How much calendar time does the inspection process take?	Average effort per KLOC Percentage of reinspections Average effort per KLOC Total KLOC inspected
Monitor and control	What is the quality of the inspected software? To what degree did the staff conform to the procedures? What is the status of the inspection process?	Average faults detected per KLOC Average inspection rate Average preparation rate Average inspection rate Average preparation rate Average lines of code inspected Percentage of reinspections Total KLOC inspected
Improve	How effective is the inspection process? What is the productivity of the inspection process?	Defect removal efficiency Average faults detected per KLOC Average inspection rate Average preparation rate Average lines of code inspected Average effort per fault detected Average inspection rate Average preparation rate Average lines of code inspected

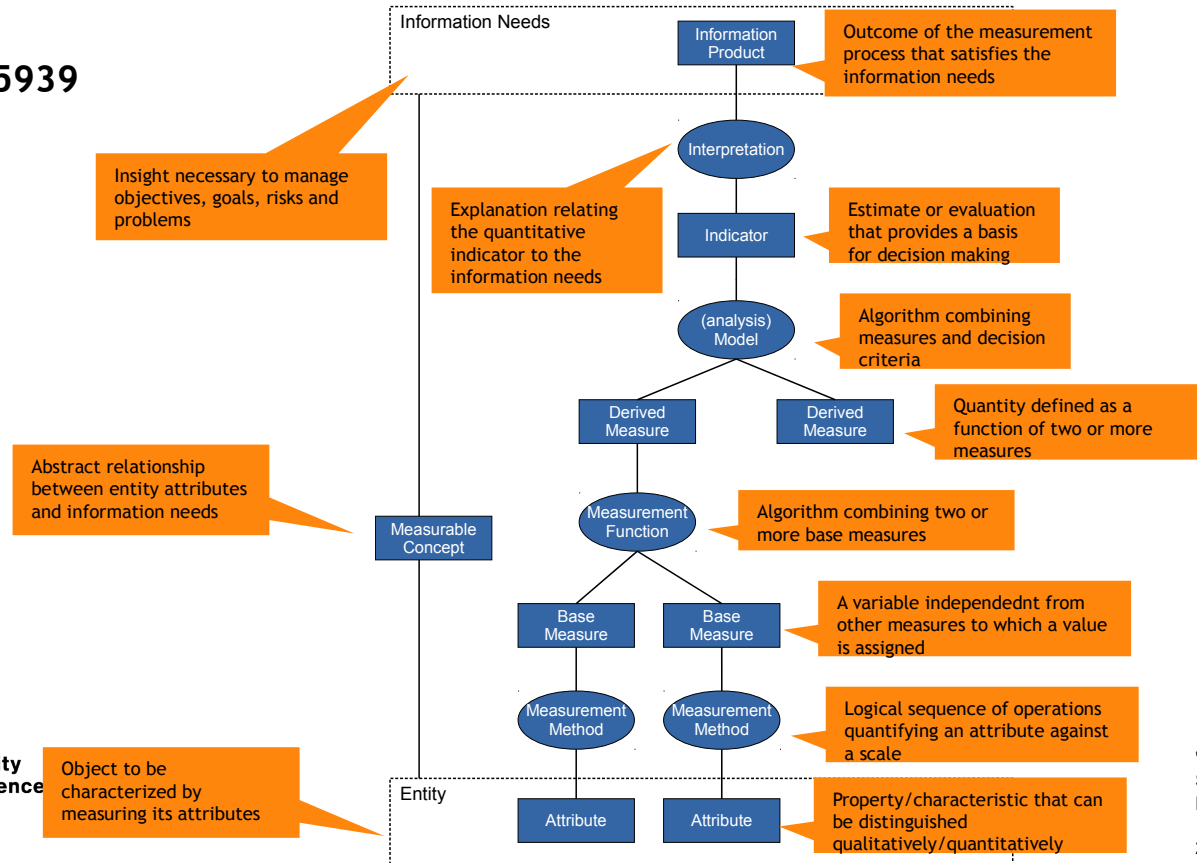
Measurement Information Model

- ISO/IEC 15939



Measurement Information Model

- ISO/IEC 15939

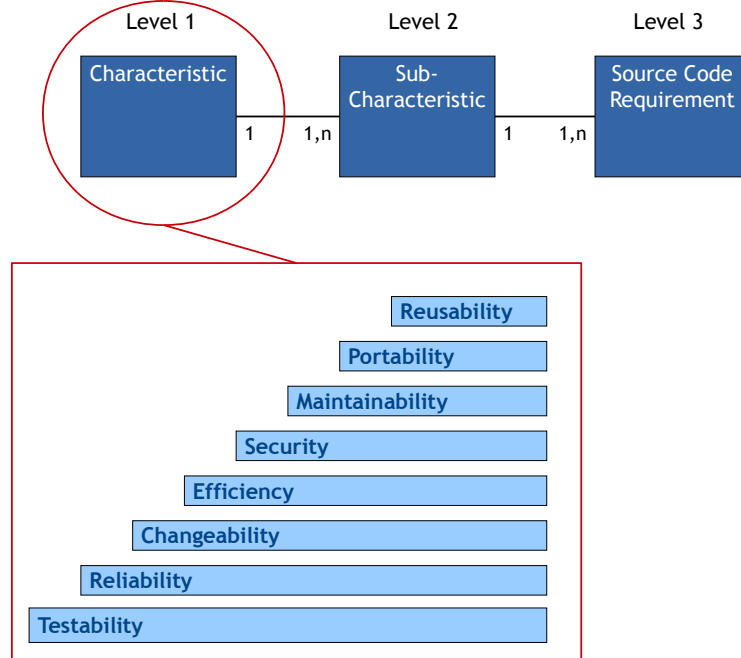


ISO/IEC 15939 - Example

Information Need	Estimate productivity of future project
Measurable Concept	Project productivity
Relevant Entities	1. Requirements implemented by past projects 2. Effort expended by past projects
Attributes	1. Shall Statements 2. Timecard entries (recording effort)
Base Measures	1. Project X Requirements 2. Project X Hours of Effort
Measurement Method	1. Count "Shalls" in Requirements Specification 2. Add timecard entries together for Project X
Type of Measurement Method	1. Objective 2. Objective
Scale	1. Integers from zero to infinity 2. Real numbers from zero to infinity
Type of Scale	1. Ratio 2. Ratio
Unit of Measurement	1. Line 2. Hour
Derived Measure	Project X Productivity
Measurement Function	Divide Project X Requirements Implemented by Project X Hours of Effort
Indicator	Average productivity
Model	Compute mean and standard deviation of all project productivity values.
Decision Criteria	Computed confidence limits based on the standard deviation indicate the likelihood that an actual result close to the average productivity will be achieved. Very wide confidence limits suggest a potentially large departure and the need for contingency planning to deal with this outcome.

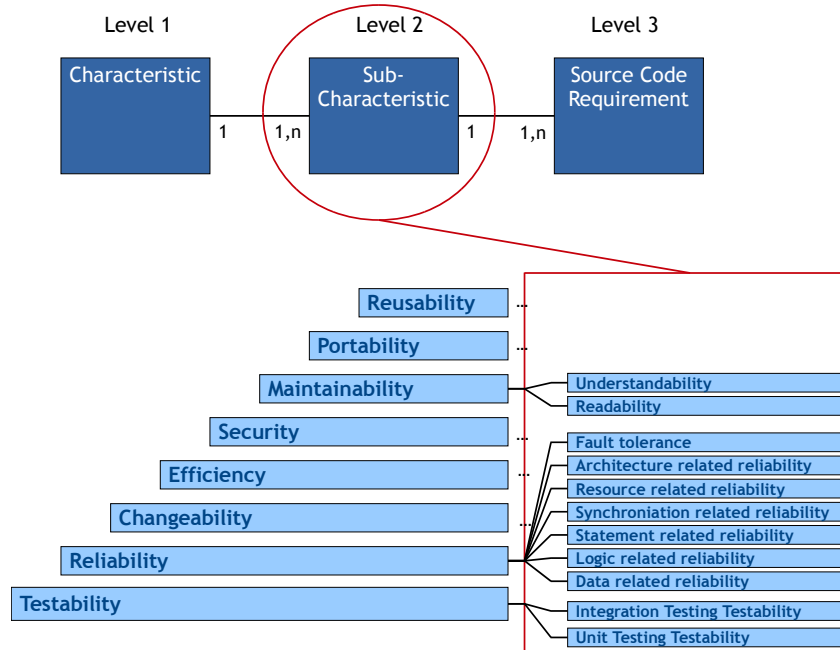
SQALE

- SQALE (Software Quality Assessment Based on Lifecycle Expectations) is a quality method to **evaluate technical debts** in software projects **based on the measurement of software characteristics**



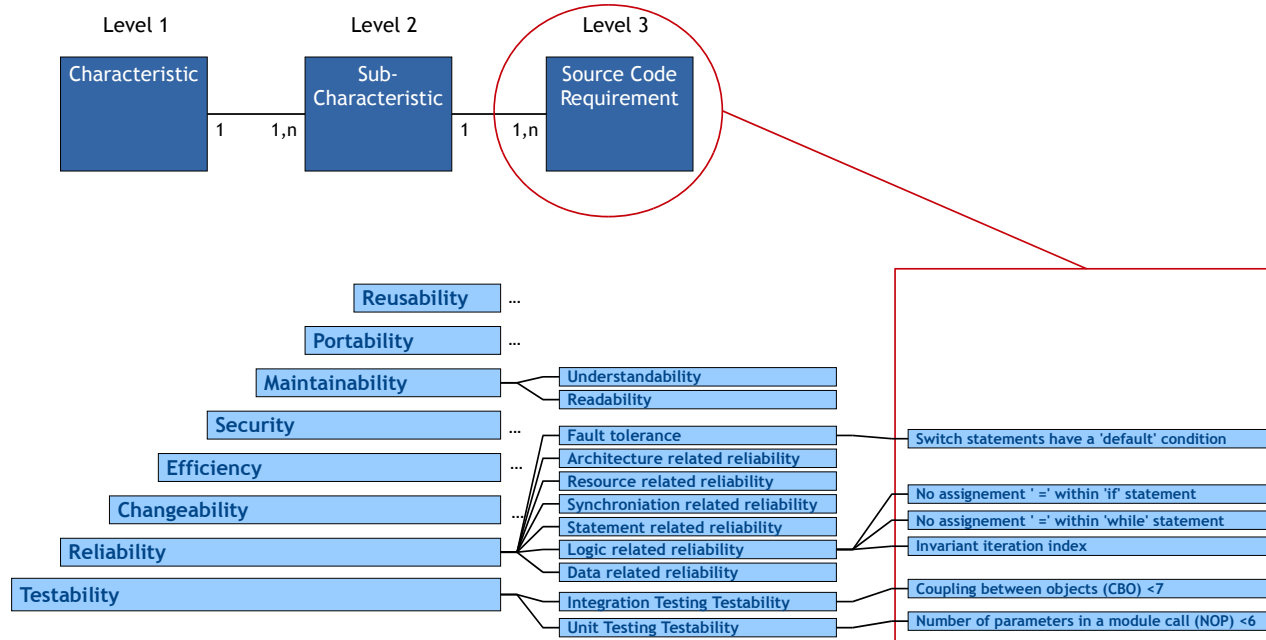
SQALE

- SQALE (Software Quality Assessment Based on Lifecycle Expectations) is a quality method to **evaluate technical debts** in software projects **based on the measurement of software characteristics**



SQALE

- SQALE (Software Quality Assessment Based on Lifecycle Expectations) is a quality method to **evaluate technical debts** in software projects **based on the measurement of software characteristics**



SQALE – (Non-)Remediation Function

- For each of the source code requirements we need to associate a **remediation function** that translates the non-compliances into remediation costs

NC Type Name	Description	Sample	Remediation Factor
Type1	Corrigible with an automated tool, no risk	Change in the indentation	0.01
Type2	Manual remediation, but no impact on compilation	Add some comments	0.1
Type3	Local impact, need only unit testing	Replace an instruction by another	1
Type4	Medium impact, need integration testing	Cut a big function in two	5
Type5	Large impact, need a complete validation	Change within the architecture	20

- Non-remediation functions represent the cost to keep a non-conformity so a negative impact from the business point of view

NC Type	Description	Sample	Non-Remediation Factor
Blocking	Will or may result in a bug	Division by zero	5 000
High	Will have a high/direct impact on the maintainance cost	Copy and paste	250
Medium	Will have a medium/potential impact on the maintainance cost	Complex logic	50
Low	Will have a low impact on the maintainance cost	Naming convention	15
Report	Very low impact, it is just a remediation cost report	Presentation issue	2

SQALE – Indexes and Rating

- Sums of all the remediation costs associated to a particular hierarchy of characteristics constitute an index:

- SQALE Testability Index: STI
- SQALE Reliability Index: SRI
- SQALE Changeability Index: SCI
- SQALE Efficiency Index: SEI
- SQALE Security Index: SSI
- SQALE Maintainability Index: SMI
- SQALE Portability Index: SPI
- SQALE Reusability Index: SRul
- SQALE Quality Index: SQI (overall index)

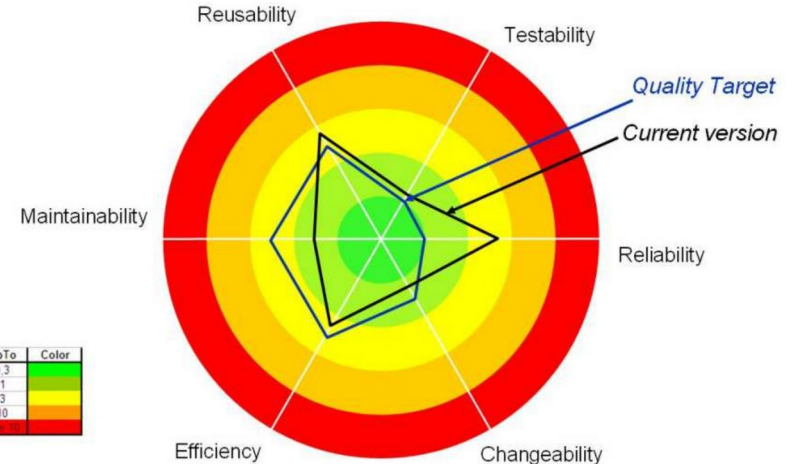
- Indexes can be used to build a rating value:

$$Rating = \frac{\text{estimated remediation cost}}{\text{estimated development cost}}$$

Rating	Up to	Color
A	1%	Green
B	2%	Light Green
C	4%	Yellow
D	8%	Orange
E	∞	Red

- Example
An artefact that has an estimated development cost of 300 hours and a STI of 8.30 hours, using the reference table on the left

$$Rating = \frac{8.30 \text{ h}}{300 \text{ h}} = 2.7 \% \rightarrow C$$



Rating	Up To	Color
A	0.3	Green
B	1	Light Green
C	3	Yellow
D	10	Orange
E	∞	Red

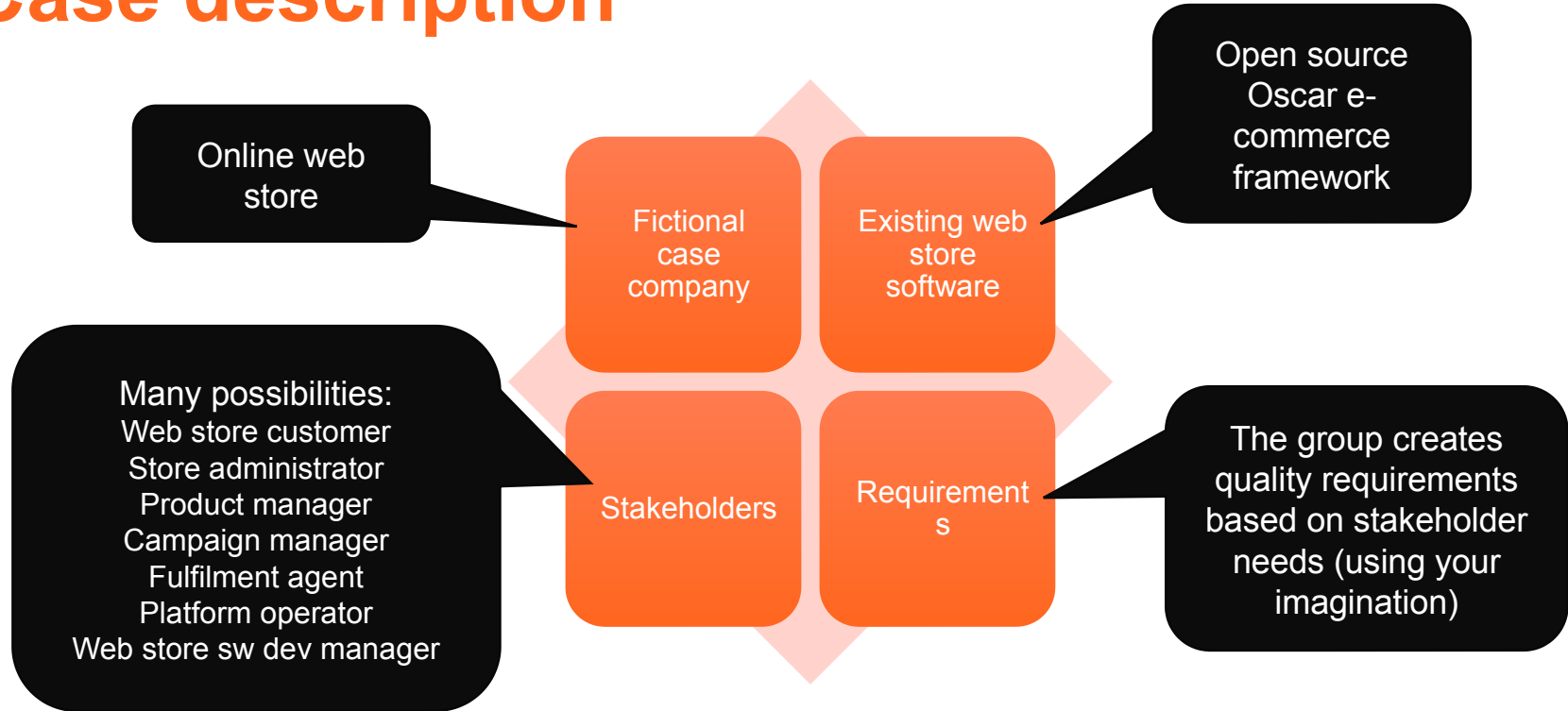
Week	Lecture	Topic	Assignment deadlines (Tuesdays 10:00 unless otherwise specified)
36	3.9.2024	Introduction and practicalities	
37	10.9.2024	Software quality	Individual assignments 1
38	17.9.2024	Software testing: levels, test case analysis and design	Group registration (DL: 20.9.)
39	24.9.2024	Testing techniques: Black-box, white-box testing	Individual assignments 2, Group assignment 1
40	1.10.2024	Testing techniques: Manual testing	Individual assignments 3
41	8.10.2024	Testing techniques: Black-box, white-box, manual testing	Individual assignments 4
42	15.10.2024	(No lecture)	
43	22.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 5
44	29.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 6
45	5.11.2024	Guest lecture	Individual assignments 7, Group assignment 2
46	12.11.2024	Continuous integration and Continuous Delivery/Deployment, Test management, and Course summary	Individual assignments 8
47	19.11.2024	(No lecture)	Individual assignments 9
48	26.11.2024	(No lecture)	Individual assignments 10, Group assignment 3

Group assignment: Final report



Aalto University
School of Science

Case description



Tasks and deliverables

Prepare	Create a quality model	Define quality control activities	Define a test plan	Execute a few tests	Make quality assessment
• Read the project case description • Choose two stakeholders to focus on • <i>Document as report introduction</i>	• Analyse stakeholder needs • Create a quality model according to ISO 25010 • <i>Document as a table + definitions</i>	• Tests • Inspections • Reviews • Metrics • Other appropriate quality control activities • <i>Document as table + text</i>	• At least 10 test cases (of which at least 1 automated unit test case, 1 automated acceptance test case, 1 manual test case) • <i>Document with inputs and expected outputs</i>	• Results from running the tests • You can reuse tests that you have already carried out in the course – but they must be properly documented • <i>Document test results</i>	• What is the current quality of the SUT for different stakeholders? • <i>Document as text</i>

Next steps

- Individual assignments in MyCourses
 - Static code analysis 2: Sonarqube
- Group assignments
 - Team testing

Deadline: before next lecture (Tuesday by 10:00)

- Software measurement

Deadline: before lecture in two weeks (12.11. by 10:00)

- Group assignments opening
 - Final report (DL: 26.11)

Getting help

Ask in the course chat, see Communication section in MyCourses

Personal questions by email

Use the related reading
for more details.
Reserve enough time to
set up your environment.



Study smarter, not harder!

- Plan: when and where
- Go deep at your own pace
- Get some rest
- Practice makes perfect
- Enjoy it ◀◀



Maintaining Quality When Moving Fast

2024



Today's topics

- Introductions: Jussi and Unity
- Continuous delivery and testing

BREAK?

- Expanding the testing horizon
- Discussion on selected topics

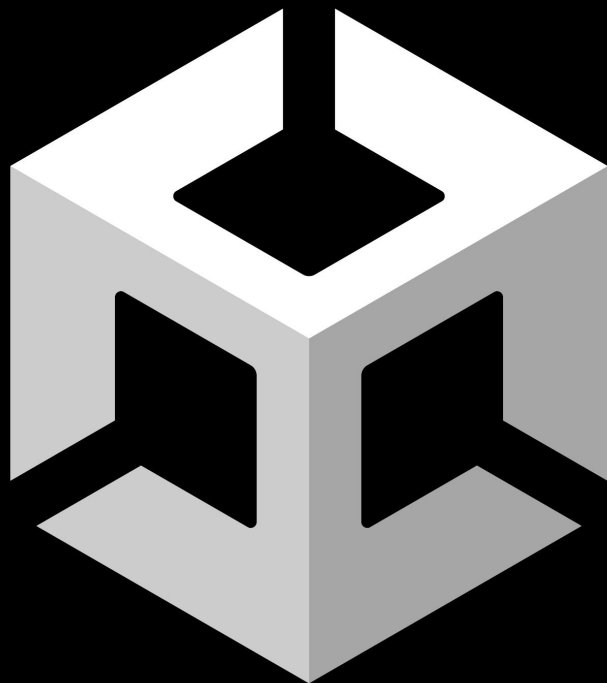


Nice to meet you!



Jussi Kuosa

Senior Software Engineer @Unity

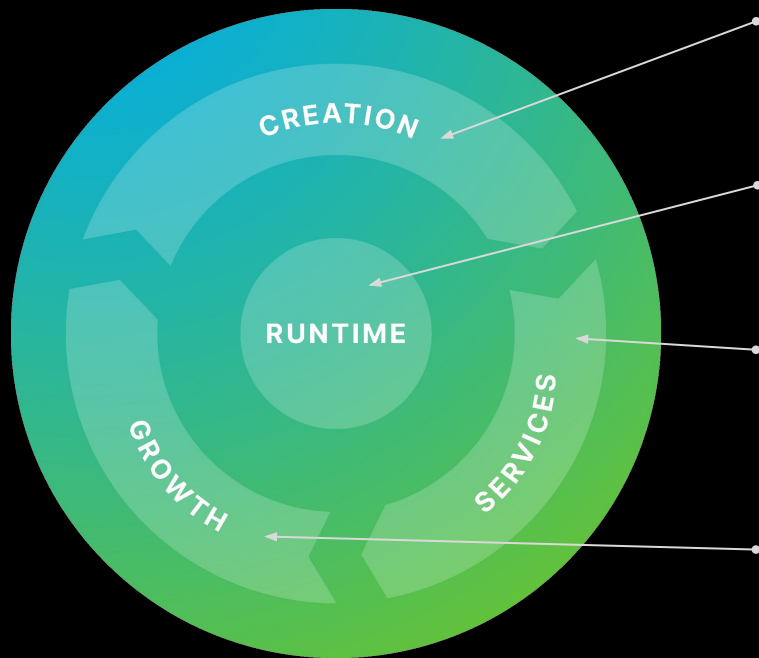




**“We believe the world is a
better place with more
creators in it.”**



SOLUTIONS AT EVERY STEP FOR CREATORS



UNITY ENGINE AND EDITOR

The Unity Editor is highly customizable and provides a visual interface for designing levels, creating game logic, and managing assets. The Unity Engine empowers creators to craft games, apps, or immersive experiences, featuring multiplatform support, with AI integration options.

UNITY RUNTIME

Unity Runtime is our proprietary software that enables games to run across 20+ platforms seamlessly. It processes game logic, renders graphics, manages inputs, and supports physics, animations, and networking directly on devices.

UNITY CLOUD

Unity Cloud provides cloud-based and connected services for developers to efficiently develop, launch, and scale games. It facilitates collaboration on real-time 3D (RT3D) projects, automates builds, tests across environments, and supports multiplayer features on over 20+ platforms.

UNITY GROW

Unity Grow offers tools for monetizing and acquiring users, featuring LevelPlay, ad networks, TapJoy Offerwall, and Supersonic for scaling mobile games into successful businesses.



TIME GHOST

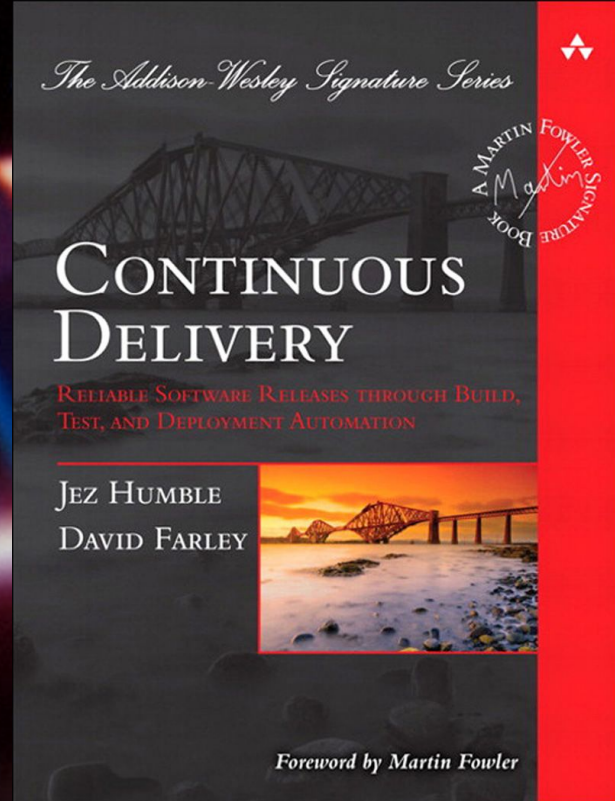


NOKIA 3310

the Chuck Norris of mobile phones



Moving Fast with Continuous Delivery



Drivers for continuous delivery

Ultimately, only the customer is able to tell the value and quality of your service

- We want to get product increments to customers as soon as possible
 - Fast feedback and time-to-market, TTM

Large releases carry a lot of inherent risk

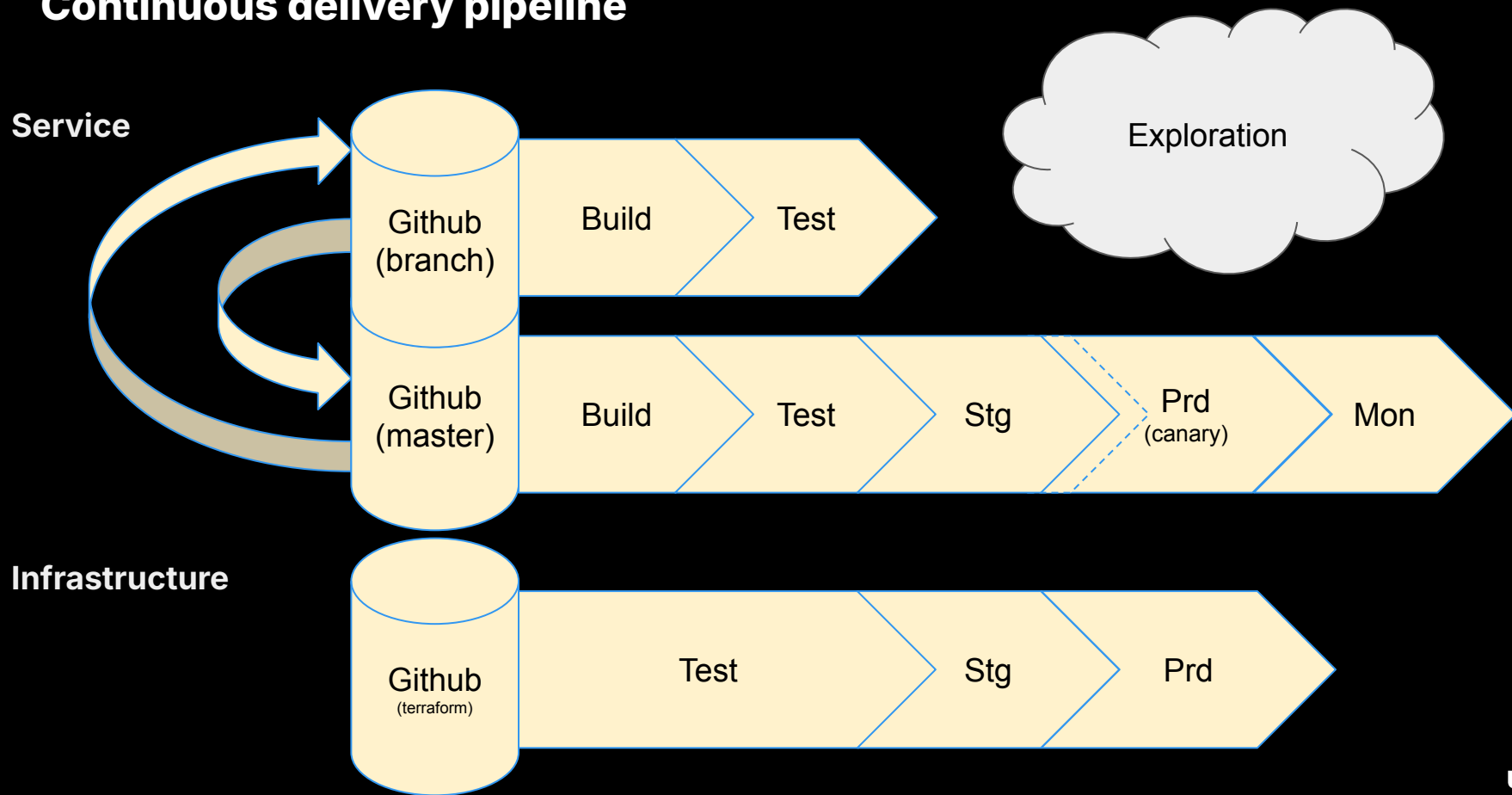
- Features end up sitting on the to-be-shipped shelf for long time
- All-or-nothing approach, long turnaround time if fixes needed
- Too often, development slips and release date is carved in stone

As delivery frequency increases, less time for massive release test phase

- Fully automated build and delivery pipeline for consistent releases
- Need a fast (=automated) regression test suite
- Currently the slowest component takes about 30 minutes to deploy fully



Continuous delivery pipeline





DORA Metrics

Deployment Frequency

How often an organization successfully releases to production

Lead Time for Changes

The amount of time it takes a commit to get into production

Change Failure Rate

The percentage of deployments causing a failure in production

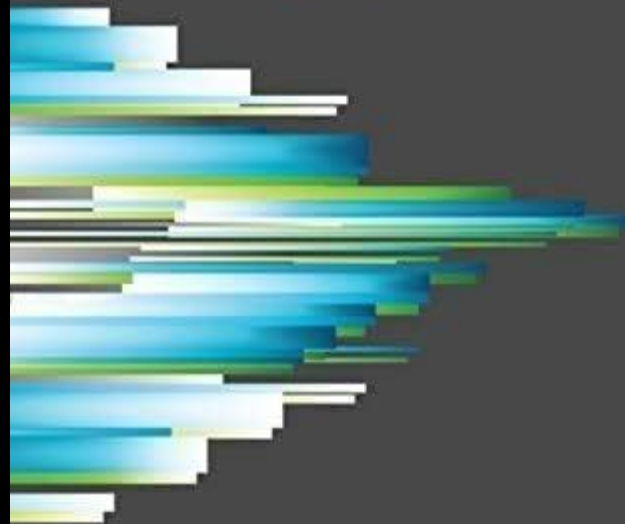
Time to Restore Service

How long it takes an organization to recover from a failure in production

+Reliability

THE SCIENCE OF DEVOPS ACCELERATE

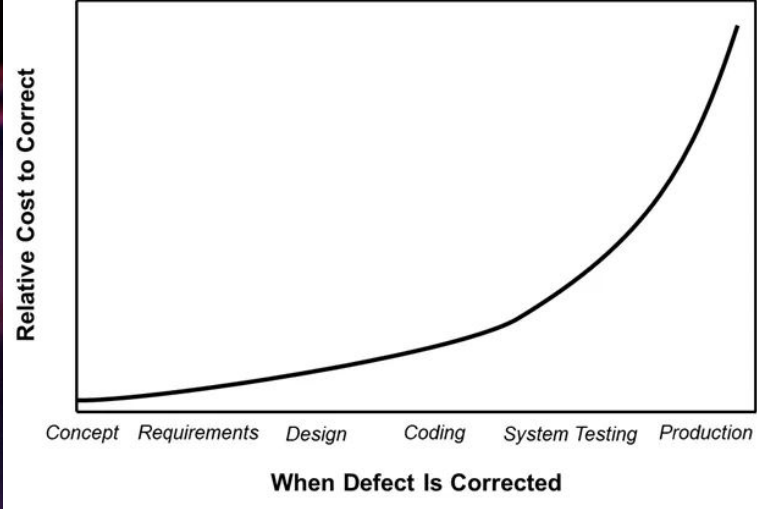
Building and Scaling High Performing
Technology Organizations



Nicole Forsgren, PhD
Jez Humble *and* Gene Kim



Continuous Testing



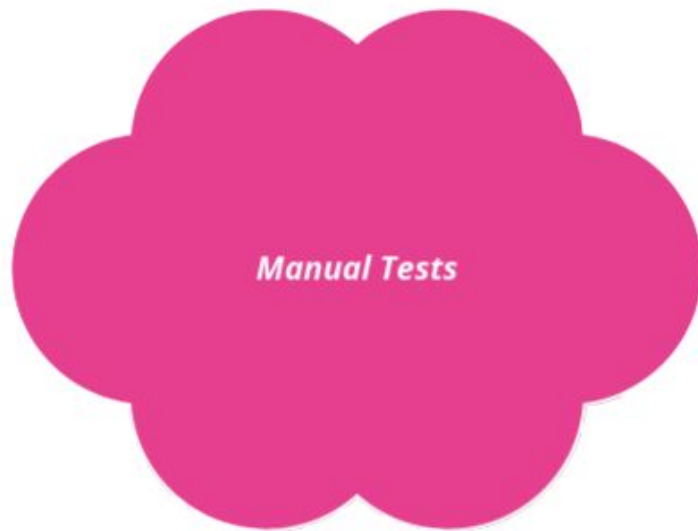


**"Teams own the quality
of their services."**



Quality Fundamentals

- Everything in code repository
- Pull requests and code reviews
- CI/CD pipeline
- Linters
- Unit tests
- Manual testing

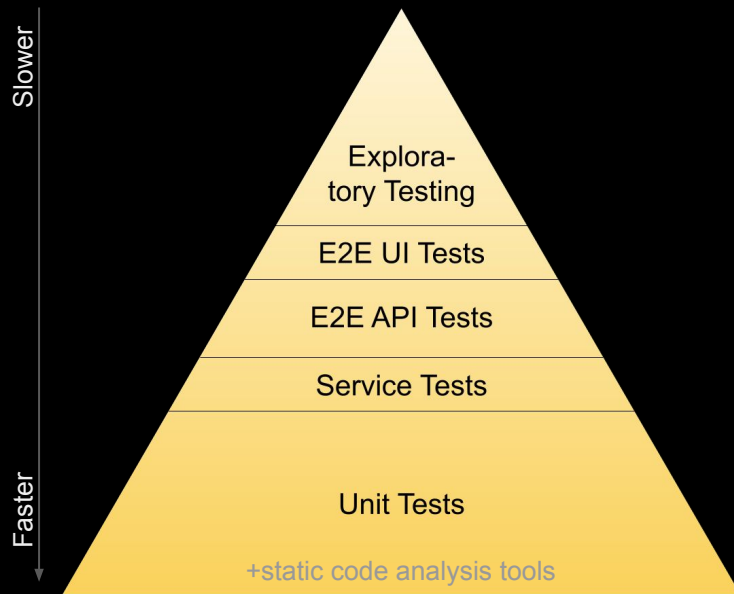


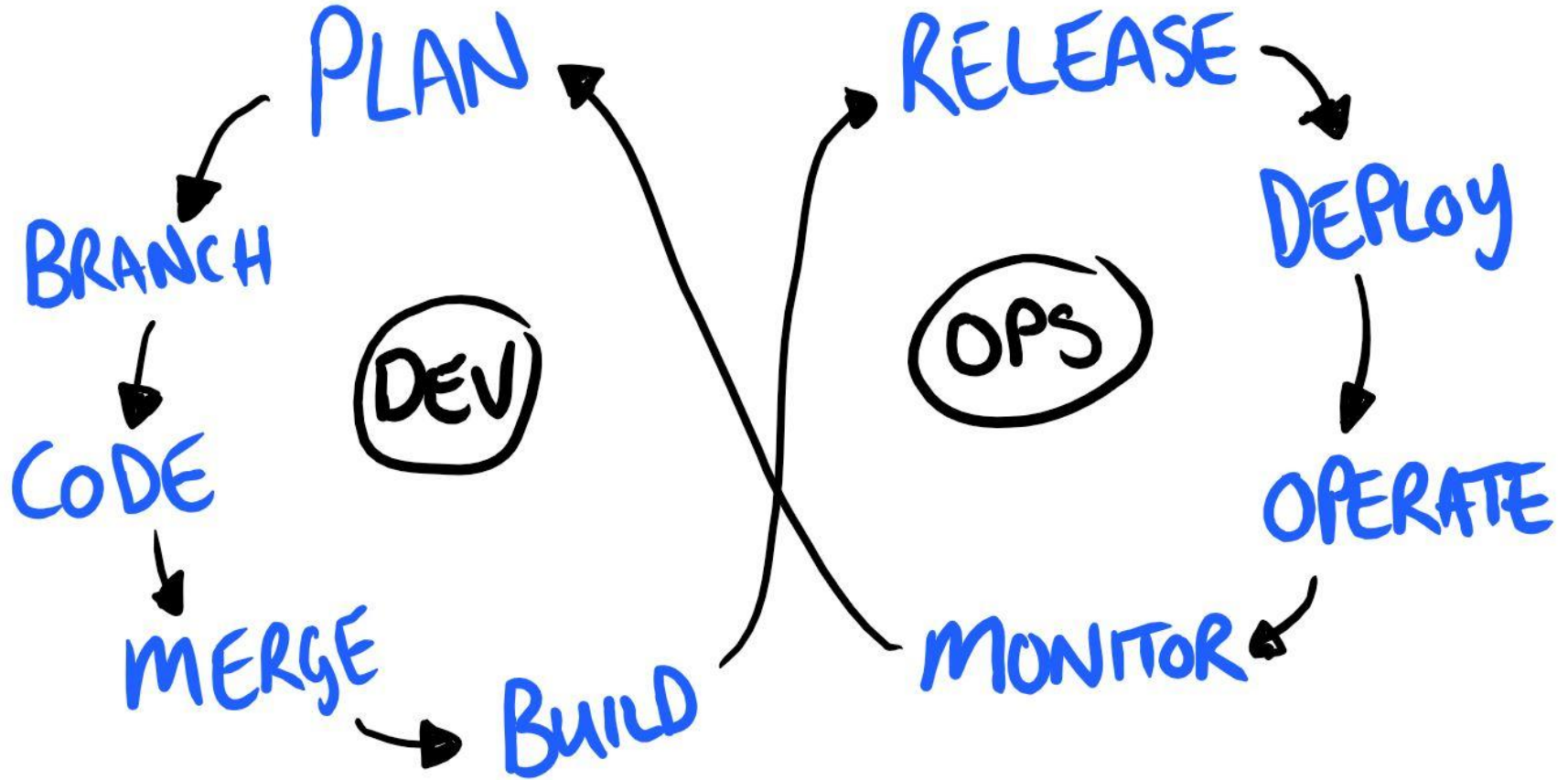


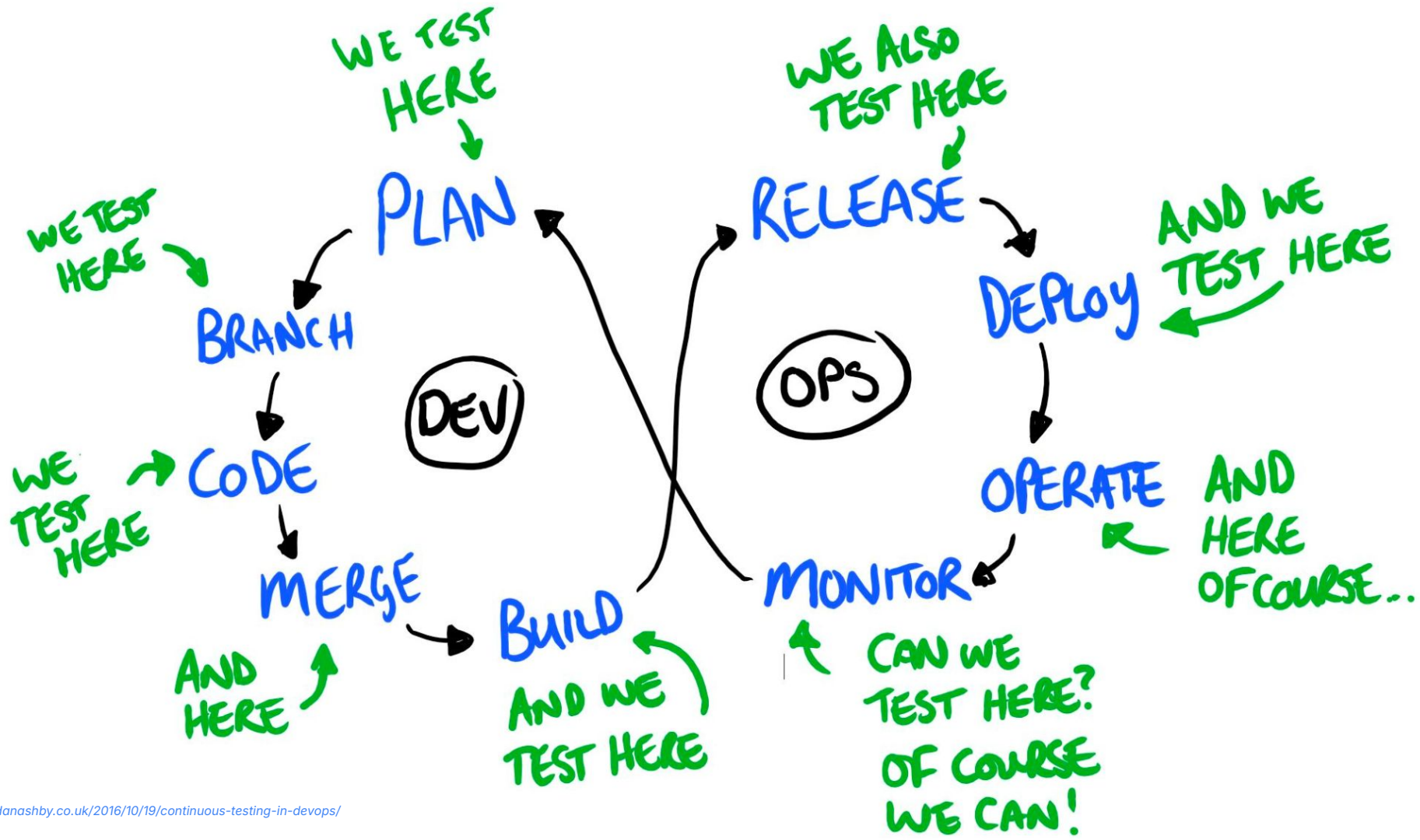
Quality Fundamentals

- Everything in code repository
 - *infrastructure as code* as well
- Pull requests and code reviews
- CI/CD pipeline
- *Formatters*
- Linters
- Unit tests with known *coverage*
- *API/integration tests*
- *UI/system tests*
- *Exploratory testing*

(+maybe load/fuzz/lln/i18n/a11y)









**"As a software engineer
I've noticed that one of our
biggest challenges isn't the
code
but understanding each other."**

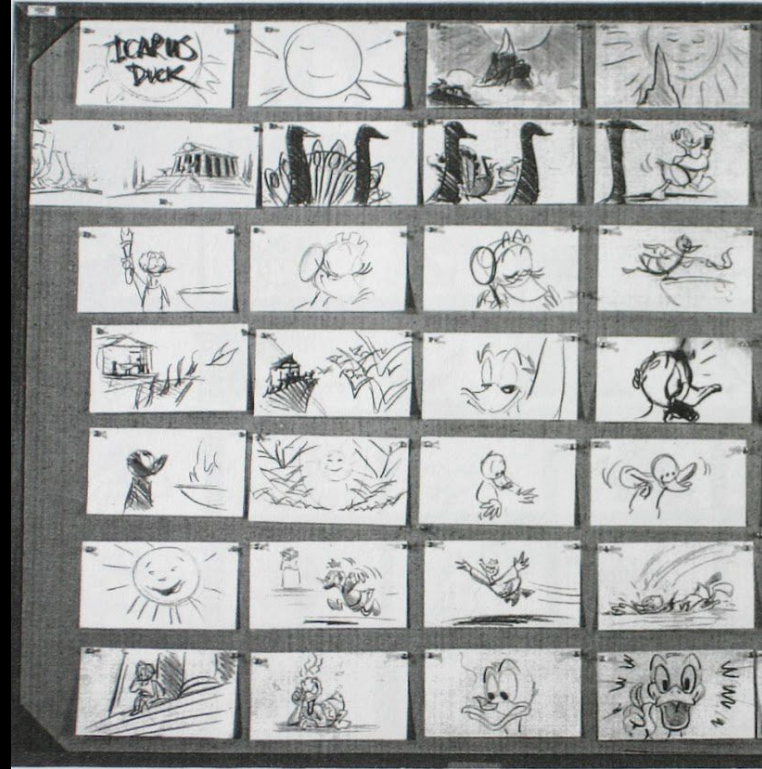
Łukasz Roszak



Shift-left testing

As Unity grew, we started using feature design documents to tie together work from numerous teams:

- Writing down the intent helps clarify the scope (to fight nodding-syndrome)
- Facilitates **discussion** across teams across multiple time zones
- Extract **customer journeys**
- Great opportunity to catch **idea bugs**
- Fast comment-correction cycle



Validation - Are we building the right product?



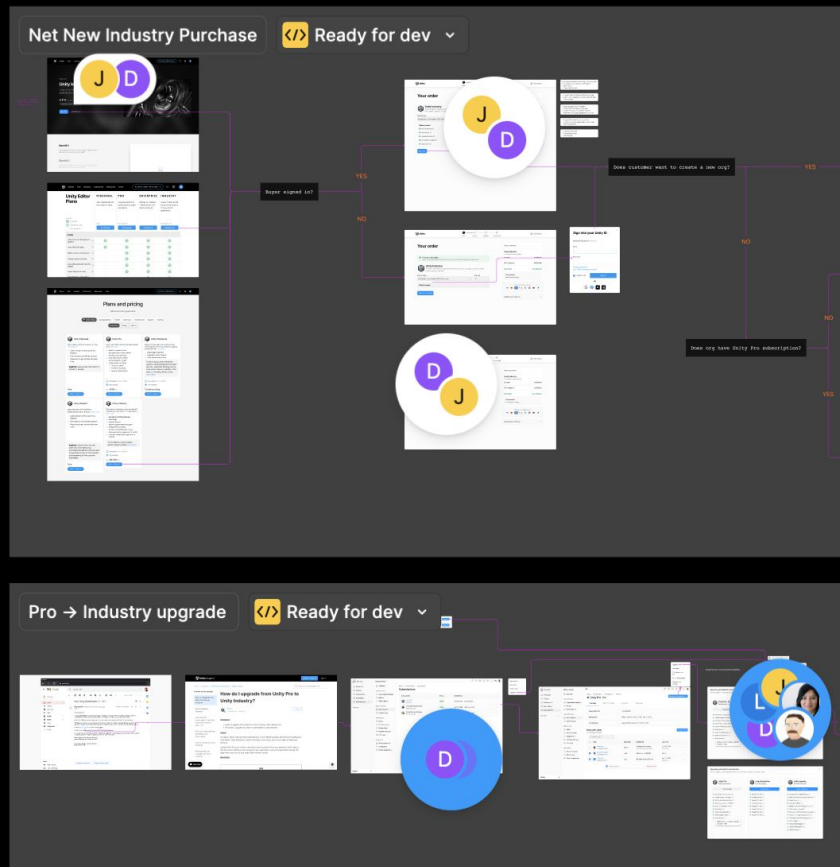
Shift-left testing

- Splitting bigger features to smaller increments (milestones)
- Validating customer journeys on UX wireframes
- Prototyping (achieve golden flow)
 - Company hackweeks
 - Cross-team workshops

The goal is to reduce risk as early as possible by removing ambiguity and misalignment and forming a clear done/acceptance criteria.

Unity Industry

PRD





Shift-right testing

Testing in production

- Observability
 - Logs, metrics, alerts, tracing
- Feature flags
- A/B testing
- Canary releases

Consequences

- Incidents

Verification - Are we building the product right?



PRE-PRODUCTION

- UNIT TESTS
- FUNCTIONAL TESTS
- COMPONENT TESTS
- FUZZ TESTS
- STATIC ANALYSIS
- PROPERTY BASED TESTS
- COVERAGE TESTS
- BENCHMARK TESTS
- REGRESSION TESTS
- CONTRACT TESTS
- LINT TESTS
- ACCEPTANCE TESTS
- MUTATION TESTS
- SMOKE TESTS
- UI/UX TESTS
- USABILITY TESTS
- PENETRATION TESTS
- THREAT MODELLING

TESTING IN PRODUCTION

← DEPLOY → ← RELEASE → ← POST RELEASE →

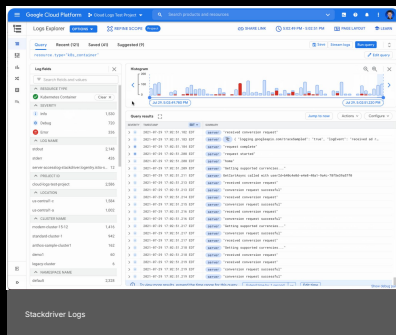
- INTEGRATION TESTS
- TAP COMPARE
- LOAD TESTS
- SHADOWING
- CONFIG TESTS

- CANARYING
- MONITORING
- TRAFFIC SHAPING
- FEATURE FLAGGING
- EXCEPTION TRACKING

- TEEING
- PROFILING
- LOGS/EVENTS
- CHAOS TESTING
- MONITORING
- A/B TESTS
- TRACING
- DYNAMIC EXPLORATION
- REAL USER MONITORING
- AUDITING
- ONCALL EXPERIENCE



Observability



Logs

Silent by default

Add logs when needed



Metrics

Actionable metrics

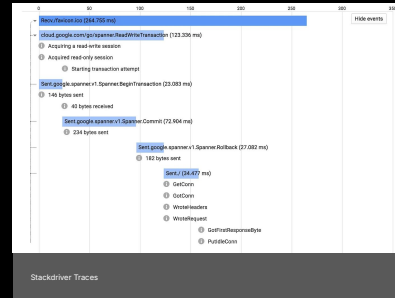
Avoid vanity metrics



Alerts

24/7 rotation

Runbooks for known scenarios



Traces

Drill-down to failing component

Combined view with logs



Feature Flags

Distributed Feature Flags

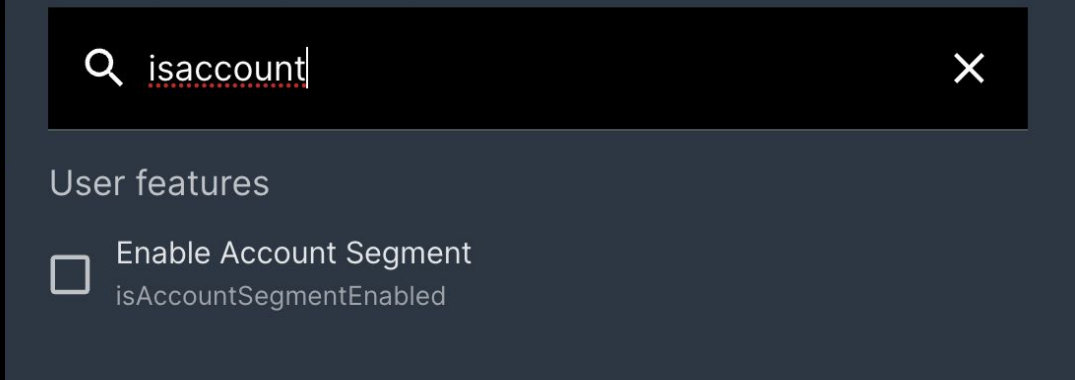
- Both front and backends
- Work across microservices

Enable gradual roll-out

- Enable first to beta-customers
- Remove after successful roll-out

Enable graceful degradation

- Disable the problematic feature, and run other features normally



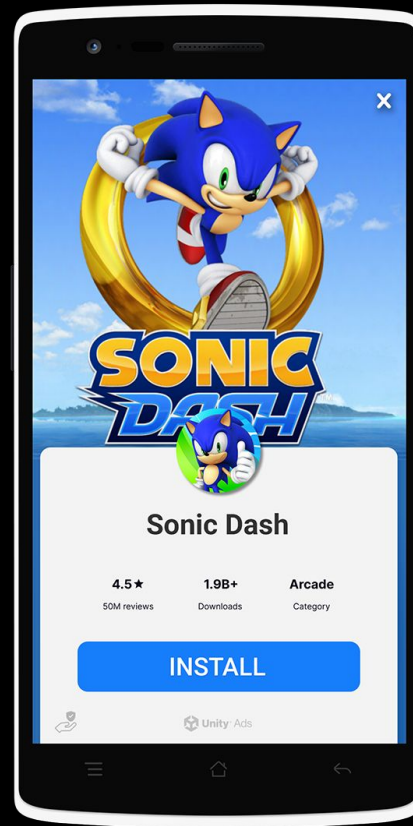
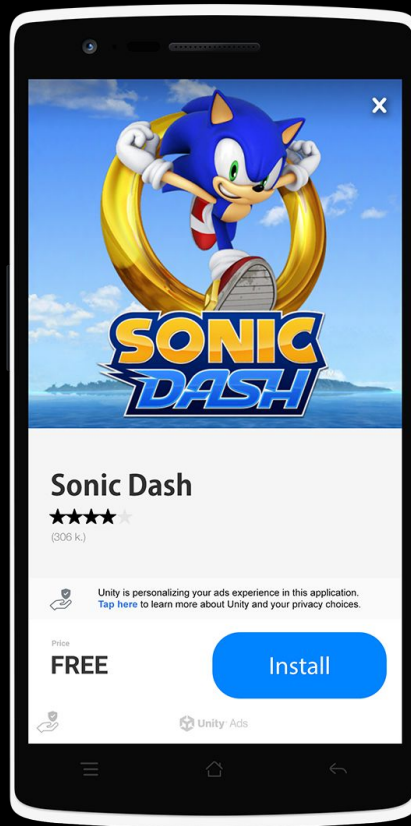
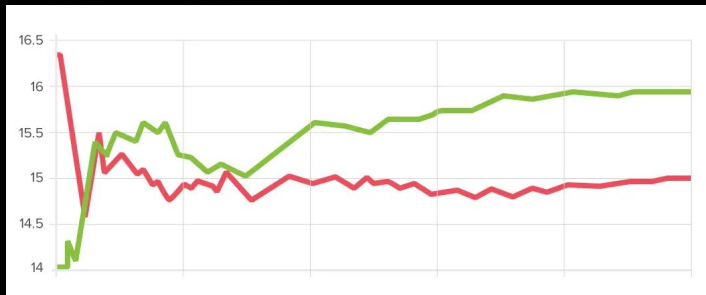
A/B Testing

Design an experiment with control and variant

Show control and variant to different groups

- Duration must be long enough for statistical significance (~1week)

Compare analytics between alternatives for desired outcome





Canary Releases

Split production to 2 (or more) segments

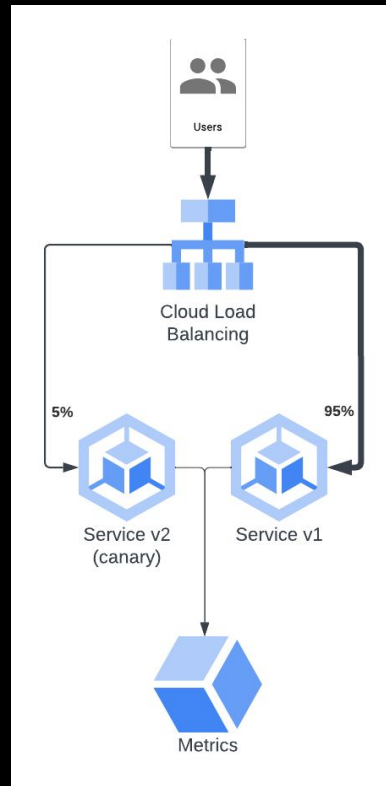
First, deploy new version to small canary area

Observe metrics for expected impact

- Less errors (bug fix)
- Reduced latency (optimization)

Roll out to full production

- OR, roll back with minimal impact





Incidents

Sometimes, we fail...

- Alerts help trigger incidents

Incidents have highest priority

- Metrics and logs allow fast pinpointing
- Tracing helps with layered systems

Always look for roll back possibility

- Stop new A/B test
- Disable feature if just exposed
- Investigate and fix in peace

Retrospective for continuous improvement



03:05 **OpsBot** APP Incident [#incident-2023-10-27](#) started by [@bipen](#) (edited)

Description

ads brand pods in crash loop backoff during deployment

Priority

P1: Service Degradation

Business unit

monetize

Latest update

This incident is not resolved. I still see CrashLoops on the latest rollout.

Resolved, waiting for postmortem



[16 replies](#) Last reply 12 days ago



Group Assignments

1. 100% unit test coverage - are we done?
 - a. What does it mean?
 - b. What defects might slip?
2. Exploratory testing when moving fast?
 - a. How can exploration be added to the mix?
3. How can the team own the quality?
 - a. What has to be done differently?
 - b. What are the benefits and downsides?
4. Performance



We have an internship program!

careers.unity.com



Thank you

2023



Software Testing and Quality Assurance

Lecture 10

Fabian Fagerholm

fabian.fagerholm@aalto.fi



Aalto University
School of Science



Week	Lecture	Topic	Assignment deadlines (Tuesdays 10:00 unless otherwise specified)
36	3.9.2024	Introduction and practicalities	
37	10.9.2024	Software quality	Individual assignments 1
38	17.9.2024	Software testing: levels, test case analysis and design	Group registration (DL: 20.9.)
39	24.9.2024	Testing techniques: Black-box, white-box testing	Individual assignments 2, Group assignment 1
40	1.10.2024	Testing techniques: Manual testing	Individual assignments 3
41	8.10.2024	Testing techniques: Black-box, white-box, manual testing	Individual assignments 4
42	15.10.2024	(No lecture)	
43	22.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 5
44	29.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 6
45	5.11.2024	Guest lecture	Individual assignments 7, Group assignment 2
46	12.11.2024	Continuous Integration and Continuous Delivery/Deployment, Test management, and Course summary	Individual assignments 8
47	19.11.2024	(No lecture)	Individual assignments 9
48	26.11.2024	(No lecture)	Individual assignments 10, Group assignment 3



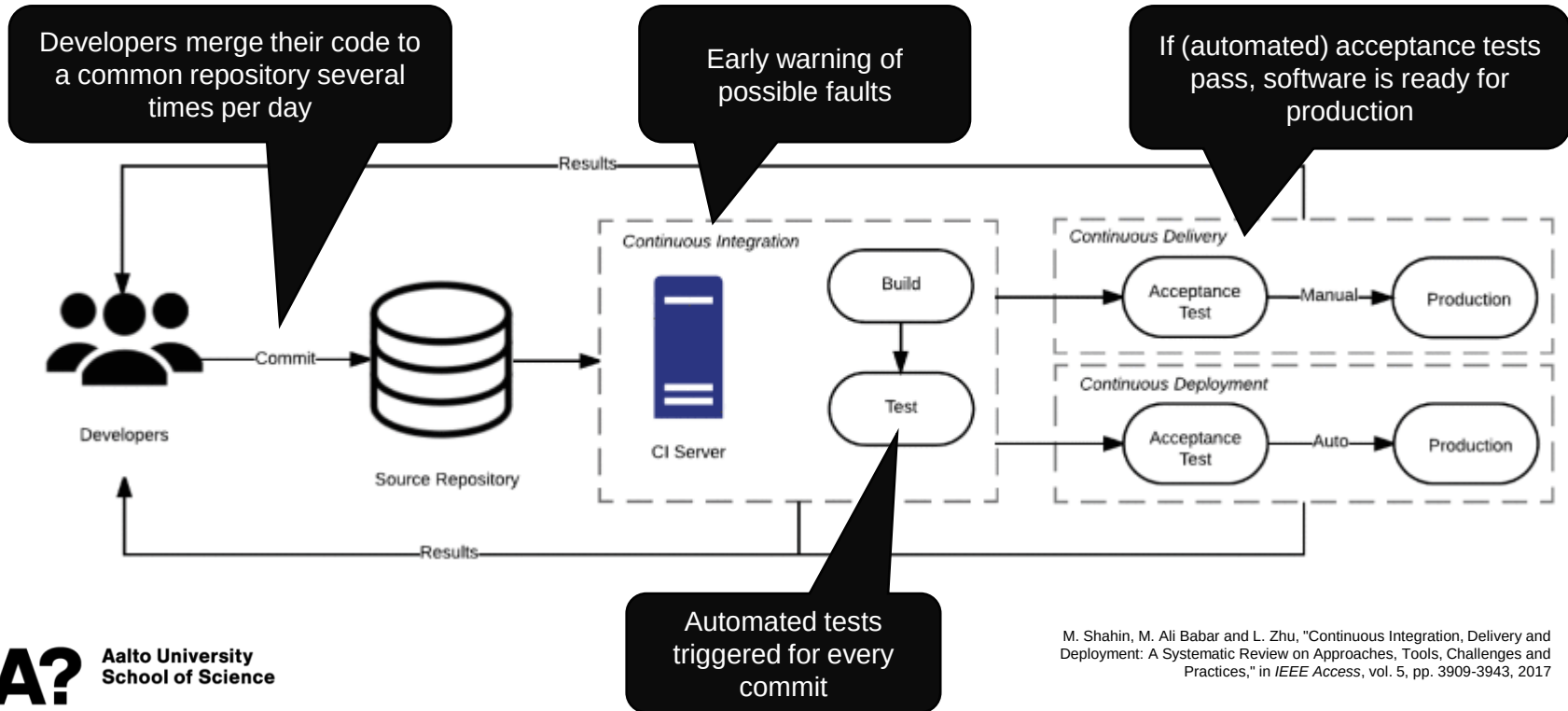
Continuous software engineering

- Conventional ways to manage testing
 - Full requirements & specifications up front
 - **Test-last: testing is performed after implementation**
 - Stricter division between testing and development personnel
 - Challenge: Time until testing is long → may have to spend lot of effort to fix, causing delays
- Agile / continuous ways to manage testing
 - Only a sprintful of requirements & specifications at a time
 - **Test-first: tests are defined with the requirements and drive development**
 - Cross-functional teams: close collaboration between testers and developers
 - Challenge: Testing fast enough as the test suite grows
 - Often supported by test automation



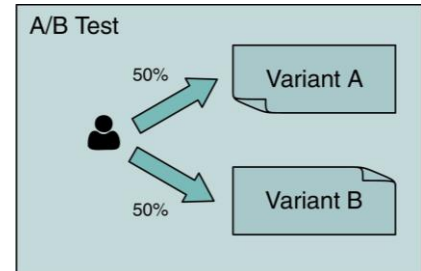
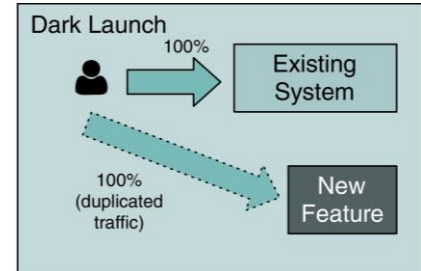
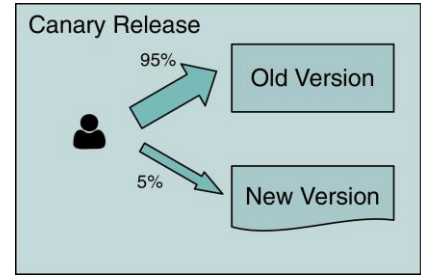
Picture licensed under [CC BY-SA-NC](#)

Continuous Integration – Delivery – Deployment



CI/CD enables several continuous software engineering practices

- Canary releases
 - Deploy new version of software to a small percentage of users
 - If there are failures, roll back
 - If everything is ok, deploy to all users
- Gradual roll-out
 - Like canary releases but deploy to more and more users
- Continuous Experimentation
 - Run field tests with real users to try how they react to new features
 - Allows finding causal relationships between features and user behaviour
 - Simple, e.g., A/B testing
 - More complicated, e.g., multi-armed bandit methods
 - Example: Which leads to more playing in a platform game?
 - A) Having the character return to the start when they die.
 - B) Having the character continue from the place they died.
 - Experiment: A/B test with 10% of players each, measure play time.



Managing software testing

Metrics for software quality

Product metrics

- Describe the characteristics of the product (size, complexity, quality level, ...)

Process metrics

- Can be used to improve software development and maintenance (effectiveness of defect removal, pattern of testing defect arrival, response time of the fix process)

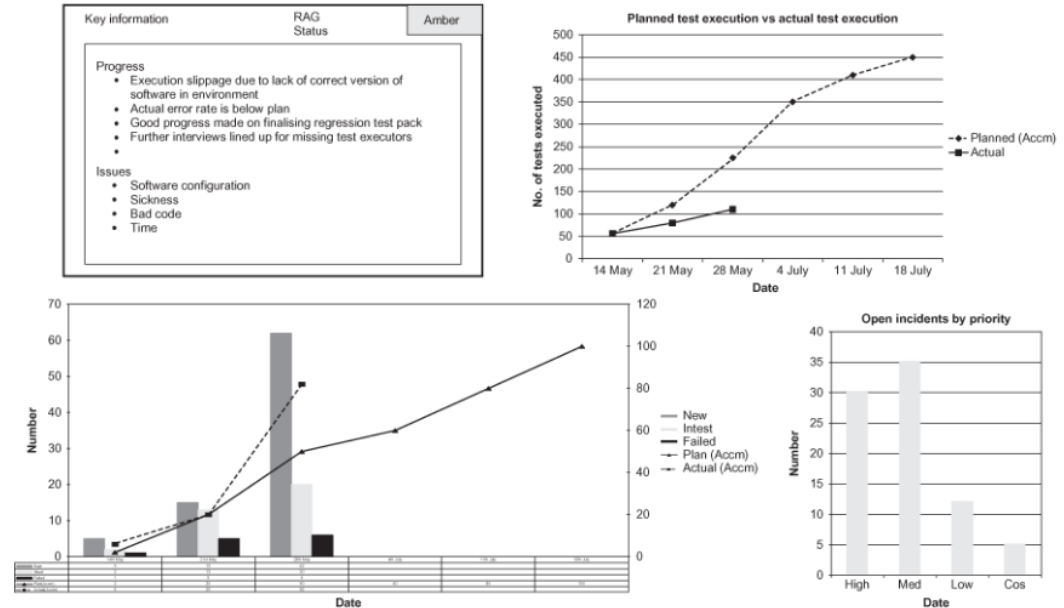
Project metrics

- Describe project characteristics and execution (number of software developers, cost, schedule, ...)

Monitoring progress

- Metrics allow us to monitor how the test plan is executed
- Feedback and visibility on how the testing is progressing
- Allows prioritizing testing effort
- Can also include monitoring of operations metrics

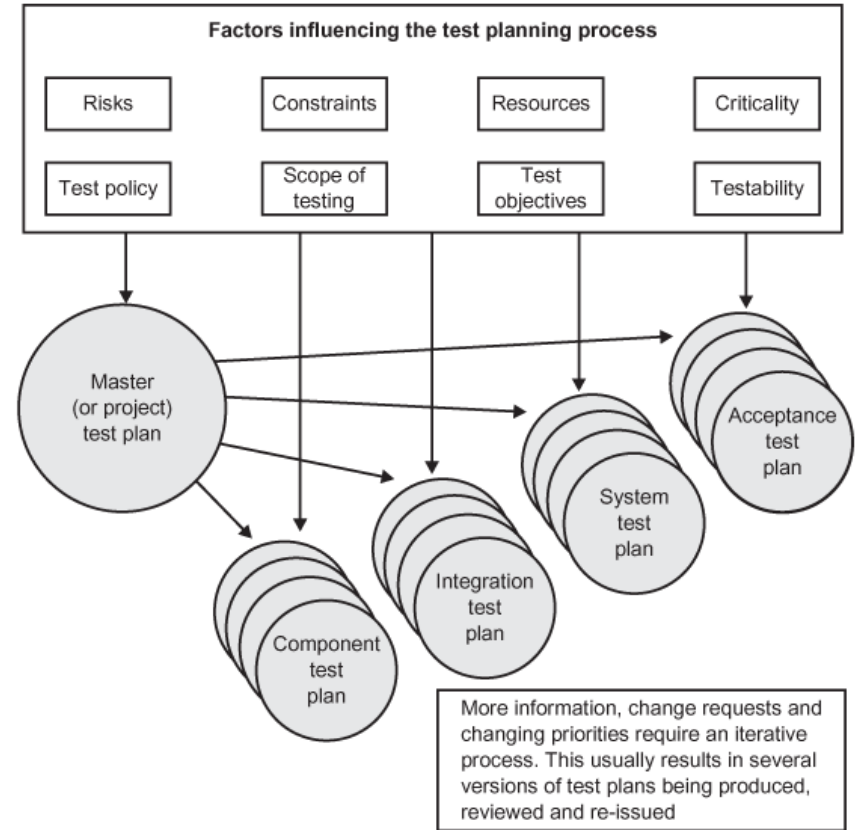
Figure 5.3 iTesting Executive Dashboard



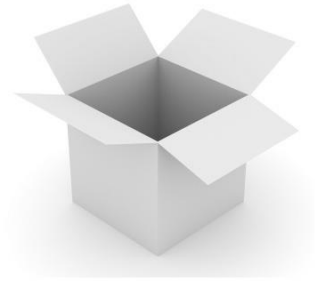
Test plans

- Steer the testing
 - What to test (scope)
 - How to test it (scope, objectives)
 - When to test it (schedule)
 - When will testing finish (effort, schedule)
 - Who does the testing (resource)
- Tasks or milestones for testing
- Track progress
- Define the direction and size of the effort
- In iterative development (e.g. agile), the plan will be continuously updated

Figure 5.2 Test plans in the V-model

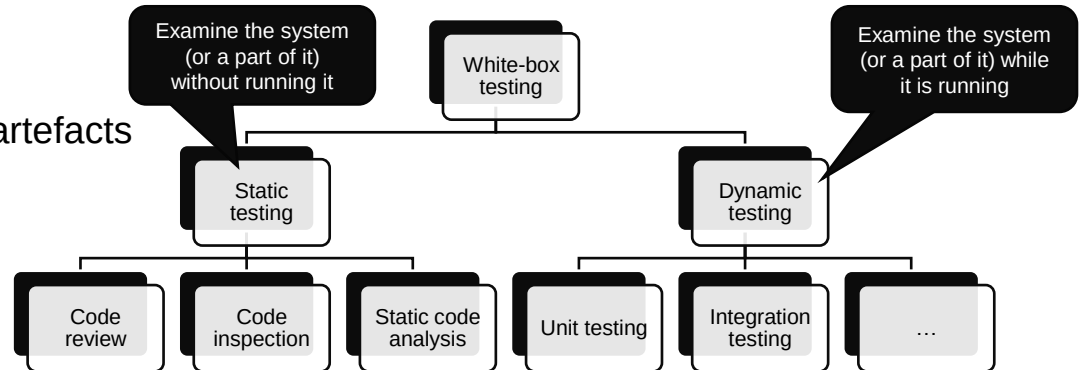


Reviews and inspections



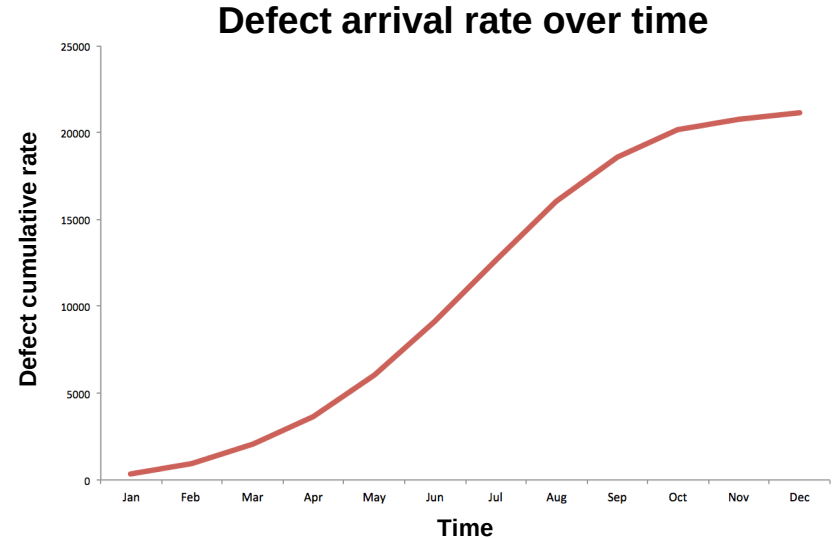
[Photo CC BY-SA-NC](#)

- Code reviews
 - Developers walk through code and try to find faults and (often) improve maintainability
 - Can be done asynchronously
- Inspections
 - Formal sessions
 - Uses a predefined protocol
 - Can be used to test non-code artefacts



Managing the testing process

- Testing usually results in more reported faults
- Development usually results in more faults in the SUT
- By monitoring the **defect arrival rate**, we can estimate how many faults are likely left in the software
- This allows us to make informed choices about the readiness of the software (e.g., for release decisions)



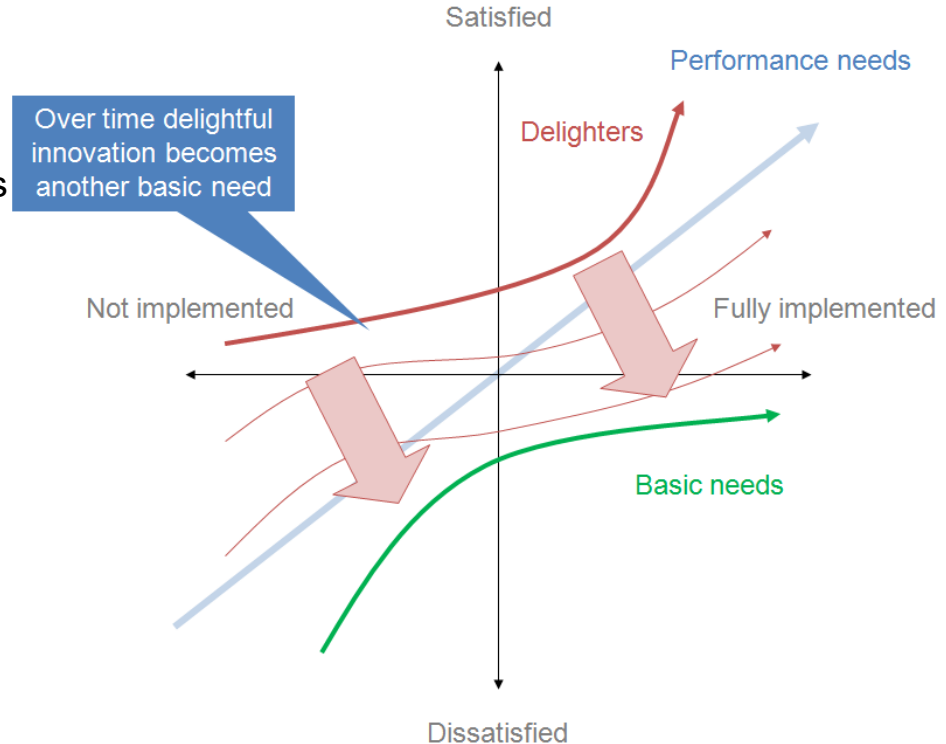
Discussion

How much quality is enough?
(Does your software have to be “perfect”?)

How much quality is enough?

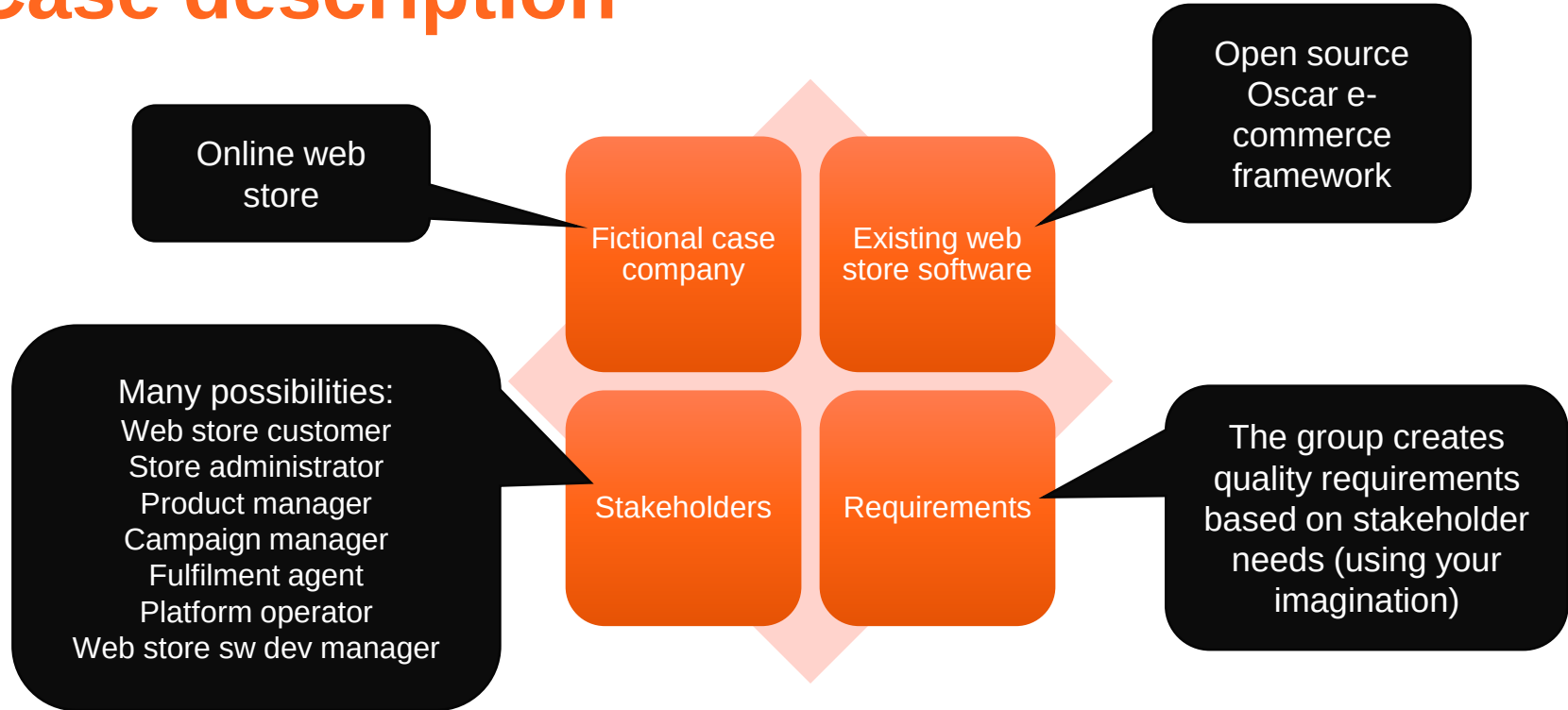
- Kano model
 - A theory for product development and customer satisfaction
 - Emphasises that customers' expectations change over time

→ Need to reassess stakeholder expectations from time to time



Group assignment: Final report

Case description



Tasks and deliverables

Prepare

- Read the project case description
- Choose two stakeholders to focus on
- *Document as report introduction*

Create a quality model

- Analyse stakeholder needs
- Create a quality model according to ISO 25010
- *Document as a table + definitions*

Define quality control activities

- Tests
- Inspections
- Reviews
- Metrics
- Other appropriate quality control activities
- *Document as table + text*

Define a test plan

- At least 10 test cases (of which at least 1 automated unit test case, 1 automated acceptance test case, 1 manual test case)
- *Document with inputs and expected outputs*

Execute a few tests

- Results from running the tests
- You can reuse tests that you have already carried out in the course – but they must be properly documented
- *Document test results*

Make quality assessment

- What is the current quality of the SUT for different stakeholders?
- *Document as text*

Final deliverable: a single PDF. A report template is provided in the assignment

Next steps

Use the related reading
for more details.
Reserve enough time to
set up your environment.

- Individual assignments in MyCourses
 - CD/CD 1: Project setup (DL: 19.11)
 - Test management (DL: 19.11)
 - CD/CD2: Pipeline configuration and execution (DL: 26.11)
- Group assignments in MyCourses
 - Final report (DL 26.11)
 - Peer review (DL 3.12)
- Getting help
 - Ask in the course chat, see Communication section in MyCourses
 - Personal questions by email



Study smarter, not harder!

- Plan: when and where
- Go deep at your own pace
- Get some rest
- Practice makes perfect
- Enjoy it ☺

Group summary: Interactive activity



Aalto University
School of Science